



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

PROGRAMOZÁSELMÉLET ÉS SZOFTVERTECHNOLÓGIAI
TANSZÉK

Webalapú Hotelmenedzsment és Foglalási Rendszer

Témavezető:

Brányi László

Mesteroktató

Szerző:

Csapó Benedek István

programtervező informatikus BSc

Budapest, 2025

SAKDOLGOZAT TÉMABEJELENTŐ

Figyelem! Nyári záróvizsga esetén a módosítás határideje február 1., őszi záróvizsga esetén augusztus 31.

Hallgató adatai:

Név: Csapó Benedek István

Neptun kód: LFUYG3

Képzési adatok:

Szak: programtervező informatikus, alapképzés (BA/BSc/BProf)

Tagozat : Nappali

Belső témavezetővel rendelkezem

Témavezető neve: Brányi László

munkahelyének neve, tanszéke: ELTE IK, Információs rendszerek Tanszék

munkahelyének címe: 1117, Budapest, Pázmány Péter sétány 1/C.

beosztás és iskolai végzettsége: Mesteroktató

A szakdolgozat címe: Webalapú Hotelmenedzsment és Foglalási Rendsze

A szakdolgozat témája:

(A témavezetővel konzultálva adja meg 1/2 - 1 oldal terjedelemben szakdolgozat témájának leírását)

Ez a szakdolgozat egy átfogó, webalapú szállodai menedzsment rendszer tervezését és fejlesztését tűzi ki célul, amely a szálloda munkatársai számára biztosít platformot a szálloda napi működésének digitális támogatására.

A rendszer célja, hogy gyors, felhasználóbarát platformot nyújtson a szálloda belső folyamatainak hatékony digitális kezelésére.

A téma iránti érdeklődésem személyes háttere, hogy családban van, aki hotelmenedzsmenttel foglalkozik, így úgy vélem, valamilyen egyedi rálátásom van a szállodai rendszerek működésére és igényeire.

A fejlesztendő rendszer tervezése és funkciói:

Backend oldali funkciók:

Autentikáció és autorizáció – különböző hozzáférési szintek létrehozása a szálloda alkalmazottai számára (recepció, menedzser, housekeeping, stb.).

Foglalási adatok kezelése, Szobák elérhetőségének és állapotának kezelése.

Értesítési rendszerek.

Statisztikai modul.

Adatbázis-kezelés – a foglalások és szállodai adatok tárolása és optimális elérése.

Frontend oldali funkciók:

Frontdesk opciók – vendégadatok kezelése, check-in/check-out folyamatok, foglalások adminisztrálása és szobák listázása szűrési lehetőségekkel.

Housekeeping funkciók – szobák állapotának kezelése, takarítási feladatok ütemezése és nyomon követése.

Szálloda adatok kezelése – alapvető szállodai információk, szobatípusok, árak és szolgáltatások módosítása.

Statisztikai modul – jelentések és statisztikák megtekintése, foglaltsági adatok elemzése, nyomtatási funkciók különböző dokumentumokhoz.

Számlázási modul.

Reszponzív design.

Budapest, 2025. 08. 24.

Tartalomjegyzék

1. Bevezetés	4
1.1. Feladat bevezetése	4
1.2. A dolgozat szerkezete	5
2. Felhasználói dokumentáció	6
2.1. Bemutakozás	6
2.2. Célközönség	6
2.2.1. Kiknek készült?	6
2.2.2. Mire?	6
2.3. Követelmények	7
2.3.1. Minimális követelmények	7
2.3.2. Optimális követelmények	7
2.4. Telepítés és első indítás	7
2.4.1. A rendszer elérése	7
2.4.2. Első bejelentkezés	8
2.4.3. Biztonsági beállítások első használat után	9
2.4.4. Hibaelhárítás	10
2.4.5. Technikai támogatás	11
2.5. Felületek és műveletek	11
2.5.1. Vezérlőpult	11
2.5.2. Alapadatok kezelése	12
2.5.3. Recepció és foglalások kezelése	21
2.5.4. Számla műveletek kezelése	27
2.5.5. Takarítások kezelése	32
2.6. Hibaiüzenetek, figyelmeztetések és visszajelzések	34
2.6.1. Űrlap validációs hibák	34
2.6.2. Műveletek végrehajtási üzenetek	35

3. Fejlesztői dokumentáció	37
3.1. Bevezetés	37
3.2. Rendszer architektúra	37
3.3. Technológiai stack	39
3.3.1. Frontend technológiák	39
3.3.2. Backend technológiák	40
3.3.3. Adatbázis technológia	40
3.3.4. Fejlesztői eszközök	41
3.3.5. Technológiai döntések indoklása	41
3.4. Adatbázis felépítése	41
3.4.1. Adatbázis konfiguráció	41
3.4.2. Entitás-kapcsolat diagram (ERD)	42
3.4.3. Entitások áttekintése	43
3.4.4. Kapcsolótáblák	44
3.5. Modul- és osztályszerkezet	45
3.5.1. Architektúra áttekintő diagram	45
3.5.2. Főbb osztályok/modulok	45
3.5.3. Osztályok közötti kapcsolatok példa	48
3.5.4. UI-tervek és navigáció	50
3.5.5. Felhasználói felület tervei	50
3.5.6. Navigációs diagram	54
3.6. Kiemelendő kódrészletek	55
3.6.1. Közös algoritmusok	55
3.6.2. Nem triviális megoldások	61
3.7. Telepítési útmutató	69
3.7.1. Rendszerkövetelmények	69
3.7.2. Docker telepítése	69
3.7.3. Projekt letöltése és indítása	70
3.7.4. Gyakori hibák és megoldásuk	71
3.8. Tesztek	71
3.8.1. Tesztelési módszertan	71
3.8.2. Tesztesetek	73
4. Teszt jegyzőkönyv	76

4.0.1.	Bejelentkezési képernyő	76
4.0.2.	Regisztrációs képernyő	77
4.0.3.	Foglalások oldal	78
4.0.4.	Vendégek oldal	86
4.0.5.	Számlák oldal	89
4.0.6.	Takarítás oldal	92
5.	Összegzés	96
6.	Függelék	97
6.1.	Szerepkörök	97
6.2.	Részletes entitás leírások	98
6.2.1.	User tábla	98
6.2.2.	Guest tábla	98
6.2.3.	Booking tábla	99
6.2.4.	Room tábla	100
6.2.5.	Invoice tábla	100
6.3.	Osztályok és modulok	102
6.3.1.	Controller osztályok	102
6.3.2.	Service osztályok	102
6.4.	Docker konfiguráció	104
6.4.1.	Docker Compose konfiguráció	104
	Irodalomjegyzék	107
	Ábrajegyzék	107
	Táblázatjegyzék	109
	Forráskódjegyzék	110

1. fejezet

Bevezetés

1.1. Feladat bevezetése

A legtöbb iparhoz hasonlóan az elmúlt évtizedben a szállodaipar is egy jelentős digitális átalakuláson ment keresztül. Manapság egy szálloda működése szinte elképzelhetetlen digitalizált menedzsment rendszerek nélkül. A foglalás kezelés, vendégadatok tárolása, szobakiosztás optimalizálása és sok egyéb más kulcs fontosságú területen nyújtanak hatékony megoldást ezen programok.

Azonban a piacon elérhető megoldások általában túlzottan bonyolultak vagy drágák kisebb, közepes méretű szállodák speciális igényeinek. Így előfordul, hogy ezen szállodák még mindig manuálisan, táblázatkezelőkkel vagy rosszabb esetben papíron vezetik mindennapi munkájukat.

Szakdolgozatom célja egy modern, egyszerű, de hatékony webalapú hotelmenedzsment és foglalási rendszer készítése. A weblap kifejezetten kisebb szállodák igényeit tartja szem előtt. Webalapja miatt minden internettel rendelkező eszközről elérhető helytől függetlenül. Felületet biztosít szobák, foglalások, vendégek és egyéb hotelmenedzsment területek kezelését, miközben a hotel adatait biztonságosan tárolja.

A témaválasztást családi kapcsolat befolyásolta: családom tagja több mint 20 éve van az iparban és rajta keresztül tapasztaltam, hogy mennyire elavult, túlkomplikált szoftvert használnak több helyen is a hotel menedzselésére. Így célként kitűztem egy egyszerű, modern rendszer kiépítését.

1.2. A dolgozat szerkezete

1. Felhasználói dokumentáció: Ismertetem hogyan kell használni és telepíteni az alkalmazást.
2. Fejlesztői dokumentáció: A rendszer belső működését beleértve a tervezési döntéseket, adatstruktúrákat és programozási megoldásokat mutatom be.
3. Tesztesetek tárháza.
4. Összefoglalás

2. fejezet

Felhasználói dokumentáció

2.1. Bemutakozás

Webalapú szállodamenedzsment rendszert szeretnék bemutatni mely felületet biztosít kisebb vagy közepes méretű hotelek mindennapi feladatainak egyszerűsítésére, megoldására és nyomon követésére.

Modern, egyszerűbb alternatívát kínál a drágább, bonyolultabb vagy elavultabb rendszerekkel szemben.

2.2. Célközönség

2.2.1. Kiknek készült?

Elsősorban a szálloda recepcióinak készült, de mellettük a menedzserek, vezetők és a szállodai gondnokság dolgozói (housekeeping) is hasznát vehetik mindennapi munkájuk során.

2.2.2. Mire?

- Foglalások kezelésére, vizualizálására.
- Szálloda jelenlegi statisztikai megtekintésére.
- Vendég adatok tárolására, szerkesztésére.
- Szálloda szervizeinek menedzselésére.
- Különféle ÁFA-ráták nyilvántartására
- Számlák megtekintésére és műveleteire

- Housekeeping feladatok kezelésére
- Felhasználók, jogosultságok alakítására
- Szobák adatainak módosítására
- Különböző alap adatok módosítására (Szoba, Ágy típusok ...)

2.3. Követelmények

2.3.1. Minimális követelmények

Internetkapcsolattal rendelkező eszköz (PC, laptop, tablet, telefon). Modern webböngésző (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+). 1280x720 képernyőfelbontás.

2.3.2. Optimális követelmények

Asztali számítógép vagy laptop. 4GB+ RAM. Stabil, széles sávú internetkapcsolat (min. 5 Mbps). 1920x1080 vagy nagyobb felbontás. Ajánlott böngésző: Chrome.

2.4. Telepítés és első indítás

2.4.1. A rendszer elérése

A rendszer egy webalapú alkalmazás, amely böngészőből érhető el. **Nem szükséges semmilyen szoftver telepítése az Ön számítógépére** a rendszer használatához

Helyi hálózati elérés

Amennyiben a rendszert a szálloda helyi hálózatán futtatják, az alapértelmezett elérési cím:

`http://localhost:3000`

Fontos: Az elérési cím változhat attól függően, hogy a rendszergazda hogyan konfigurálta a szerveret. A pontos címet a rendszer üzembe helyezésekor kapja meg az IT osztálytól vagy a rendszergazdától.

Online demo verzió

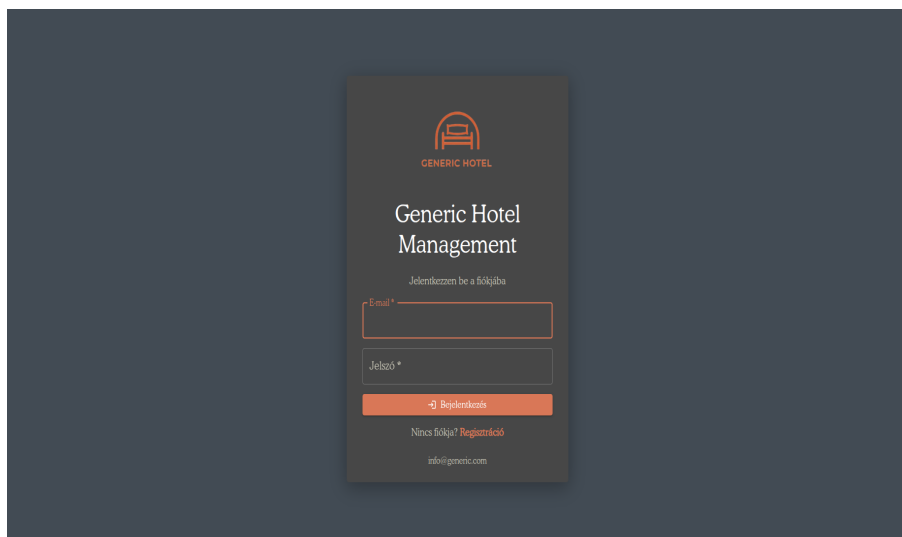
A rendszer egy működő demo verziója nyilvánosan elérhető az alábbi címen:

`https://hotelpmsdemo.up.railway.app`

A demo verzió ugyanazokkal a funkciókkal rendelkezik, mint az éles rendszer, és tesztelési célokra használható. **Figyelem:** A demo verzióban tárolt adatok nem valósak, rendszeresen törlésre kerülnek és alkalmazás ingyenes infrastruktúrát használ, ami lassabb működést eredményezhet. Türelmét köszönjük!

2.4.2. Első bejelentkezés

A rendszer megnyitásakor a bejelentkezési képernyő fogadja Önt:



2.1. ábra. Alapértelmezetten megjelenő bejelentkezési képernyő

Alapértelmezett belépési adatok

A frissen telepített rendszer az alábbi alapértelmezett bejelentkezési adatokkal rendelkezik tesztelési célokra:

E-mail cím	Jelszó
admin@hotel.com	admin123

2.1. táblázat. Alapértelmezett adminisztrátori fiók

Biztonsági figyelmeztetés: Ez a fiók csak kezdeti beállításhoz és teszteléshez használható!

Kezdeti adatok

A rendszer első indításakor alapértelmezett demo adatokkal rendelkezik, amelyek megkönnyítik a rendszer kipróbálását:

- Minta szobák (pl. Szoba 101, 102, 103)
- Szoba típusok
- Ágy típusok
- ÁFA kulcsok
- Egyéb alapadatok

Ezeket az adatokat **a rendszer használata előtt törölnie vagy módosítania kell** a szálloda tényleges adataival.

2.4.3. Biztonsági beállítások első használat után

FONTOS! Az első bejelentkezés után az alábbi biztonsági lépéseket **kötelezően végre kell hajtani** (Kivéve Demo weblap használata során):

1. Új adminisztrátori felhasználó létrehozása

Navigáljon a Felhasználók menüpontba, és hozzon létre egy új adminisztrátori jogosultságú felhasználót a saját e-mail címével és erős jelszóval.

(a) Felhasználók menü kiválasztva

(b) Felhasználót létrehozó ablak

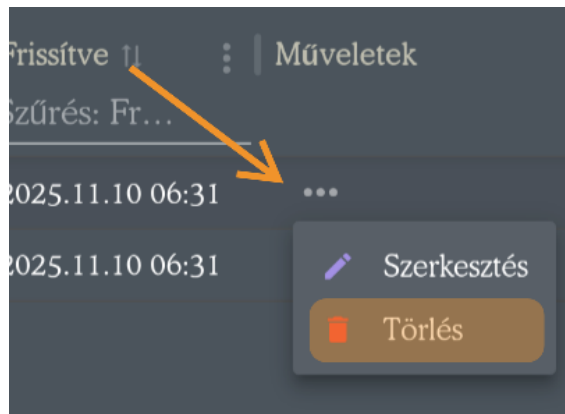
2.2. ábra. Új felhasználó létrehozását dokumentáló képek

2. Jelentkezzen ki, majd jelentkezzen be az új fiókkal

Ellenőrizze, hogy az új fiók megfelelően működik és rendelkezik adminisztrátori jogosultságokkal.

3. Törölje az alapértelmezett admin@hotel.com fiókot

A Felhasználók menüben válassza ki az alapértelmezett admin fiókot és törölje véglegesen.



2.3. ábra. Alapértelmezett admin fiók törlése

4. Törölje vagy módosítsa a demo adatokat

Navigáljon végig a Szobák, Vendégek és Beállítások menüpontokban, és cserélje le a demo adatokat a szálloda valós adataival.

2.4.4. Hibaelhárítás

Nem érem el a rendszert

Ha a megadott címen nem érhető el a rendszer:

- Ellenőrizze, hogy helyesen írta-e be a címet (http:// vagy https://)
- Próbáljon meg más böngészőt használni
- Ellenőrizze, hogy a számítógépe kapcsolódik-e a hálózathoz
- Ellenőrizze, hogy a szerver fut-e (kérdezze meg a rendszergazdát)

Nem tudok bejelentkezni

Ha a bejelentkezési adatok nem működnek:

- Ellenőrizze, hogy helyesen írta-e be az e-mail címet és jelszót
- Figyeljen a kis- és nagybetűk különbségére
- Próbálja meg az alapértelmezett admin fiókot (csak új telepítésnél)

- Ha elfelejtette jelszavát, forduljon a rendszergazdához

2.4.5. Technikai támogatás

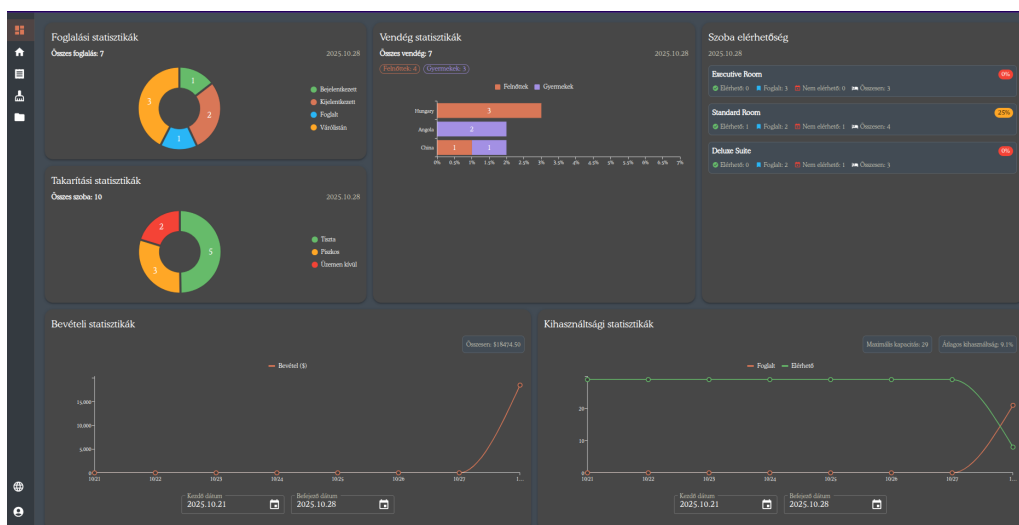
Ha a fenti megoldások nem segítenek, vagy bármilyen egyéb technikai problémája merül fel, kérjük, forduljon a rendszergazdához vagy az IT osztályhoz.

A rendszer telepítésével, konfigurálásával és szerver oldali karbantartásával kapcsolatos információk a **3.7. részben** találhatók (Elsősorban informatikai szakemberek részére).

2.5. Felületek és műveletek

2.5.1. Vezérlőpult

Bejelentkezés után a vezérlőpult fogad, amely a szálloda aktuális állapotáról nyújt gyors áttekintést. Itt megtekinthetők a mai foglalásokat, vendégstatisztikákat és a bevételi adatokat.



2.4. ábra. Vezérlőpult megjelenése

A következő statisztikák jelennek meg balról jobbra:

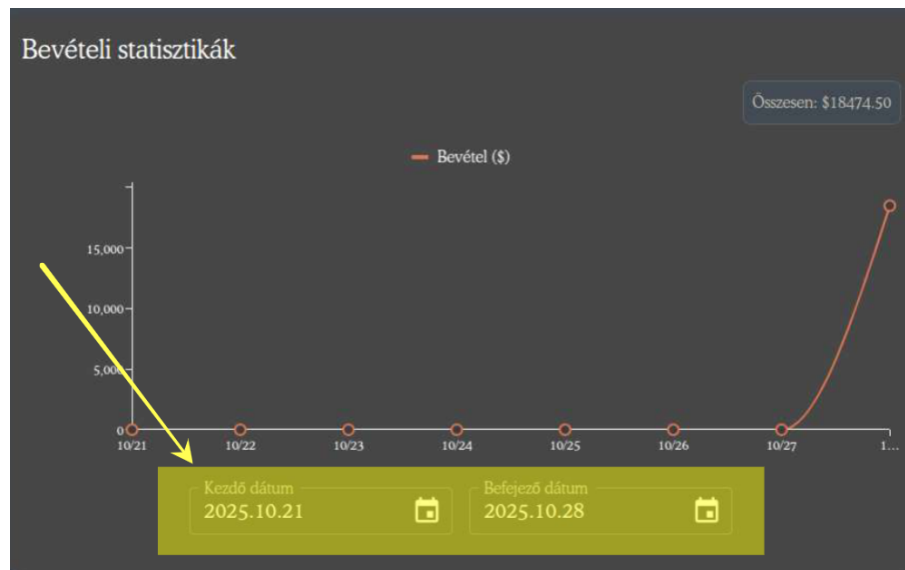
1. **Foglalási statisztikák:** Mai érkezések, távozások és aktív foglalások száma
2. **Vendégstatisztikák:** Jelenleg tartózkodó vendégek száma
3. **Elérhető szobátípusok:** Szabad szobák típusonként
4. **Takarítási statisztikák:** Takarítandó és kész szobák
5. **Bevételi gráf:** Napi bevételek alakulása

6. Kihasználtsági gráf: Szobakihasználtság

Időszak beállítása a gráfoknál

A bevételi és kihasználtsági gráfok esetében beállíthatom a megjeleníteni kívánt időszakot. Alapértelmezetten az elmúlt egy hétre szűr a rendszer.

Fontos: Minél hosszabb időszakot választok, annál több időt vesz igénybe a gráf betöltése.



2.5. ábra. Bevételi gráf időszak beállítása

2.5.2. Alapadatok kezelése

Ezekon a felületeken tárolódnak a szálloda alapvető adatai szűrhető táblákban. A táblázatok minden sornál megtekinthető a létrehozás és az utolsó módosítás dátumát. A táblázatokat minden felhasználó megtekintheti, azonban az adatok létrehozása és módosítása Admin vagy Data maintaner jogosultsághoz kötött (lásd: 6.1. függelék).

Megjegyzés: Minden művelet után a rendszer sikeres mentésről vagy hibaüzenettel tájékoztat.

Szűrők és táblaműveletek A táblázatok jobb felső sarkában találhatóak a következő tábla műveletek (lásd: 2.6. ábra, balról jobbra):

1. Oszlopszűrők megjelenítése
2. Oszlopok megjelenítése vagy elrejtése

3. Táblázat sűrűségének beállítása

4. Teljes képernyős mód

Megjegyzés: Alapértelmezetten léteznek elrejtett oszlopok!



2.6. ábra. Táblaműveletek

Szoba szolgáltatás neve ↓	⋮	Létrehozva ↑	⋮	Frissítve ↑	⋮
Szűrés: Szoba szolgáltatás neve alapján		Szűrés: Létrehozva al...		Szűrés: Frissítve alapján	

2.7. ábra. Oszlopszűrés és rendezés felület

Alapadatok közös kezelési műveletek

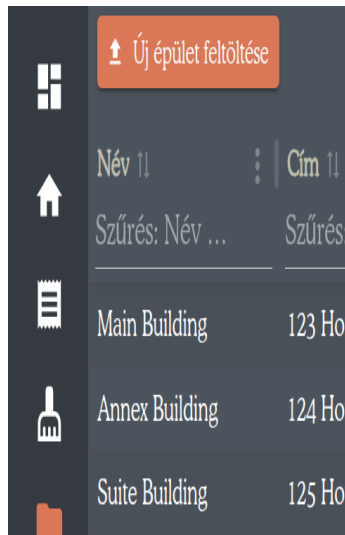
A következő alapadat-táblák azonos módon kezelhetők:

- Épületek
- Szoba szolgáltatások
- Vendég címkék
- Ágytípusok
- Szerepkörök
- Felhasználók
- Szoba típusok
- Szobák

Mindegyik felületen az alábbi három alapművelet érhető el alapból a megfelelő jogosultsággal:

Új elem hozzáadása:

1. Megnyomom az **Új [elem] feltöltése** gombot
2. Megnyílik a létrehozó űrlap
3. Kitöltöm a kötelező mezőket
4. Elmentem



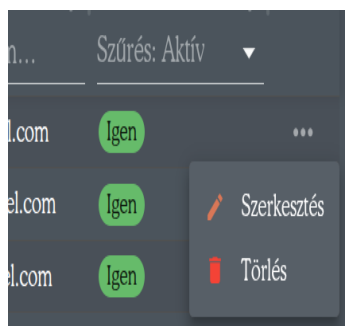
(a) Épület hozzáadása gomb

(b) Épület létrehozó űrlap

2.8. ábra. **Példa:** Új épület hozzáadásának lépései

Meglévő elem módosítása:

1. A módosítandó elem sorának végén megnyomom a ... gombot
2. Kiválasztom a **Szerkesztés** menüpontot
3. Módosítom a kívánt mezőket
4. Elmentem



(a) Épület műveletek menü

(b) Épület szerkesztő űrlap

2.9. ábra. **Példa:** Épület szerkesztésének lépései

Elem törlése:

1. A törölni kívánt elem sorának végén megnyomom a ... gombot
2. Kiválasztom a **Törlés** menüpontot
3. Megerősítem a törlést

2.10. ábra. **Példa:** Épület törlése gomb

Fontos: Az elemek törlése csak akkor lehetséges, ha nincsenek hozzájuk kapcsolódó rekordok más táblákban. De erről majd pontosabban az egyes felületek ismertetésénél kitérek.

Épületek felület

Mikor használok? Az épületek kezelése oldalon állíthatom be, hogy a szálloda mely épületeiben találhatóak szobák. Ez akkor hasznos, ha a szálloda több épületből áll (pl. főépület, melléképület).

Specifikus mezők: Név, Cím, Város, Irányítószám, Ország, Telefonszám, Email, Aktív-e

Elérés a menüből: Adatbank => Épületek

Törlési megszorítás: Az épületet csak akkor törölhetem, ha nincsenek hozzá tartozó szobák. Ellenkező esetben a rendszer hibaüzenetet jelenít meg.

+ Új épület felvétele						
Név	Cím	Város	Irányítószám	Ország	E-mail	Műveletek
Main Building	123 Hotel Street	Hotel City	12345	Hotel Country	main@hotel.com	...
Annex Building	124 Hotel Street	Hotel City	12345	Hotel Country	annex@hotel.com	...
Suite Building	125 Hotel Street	Hotel City	12345	Hotel Country	suites@hotel.com	...

2.11. ábra. Épületek felület

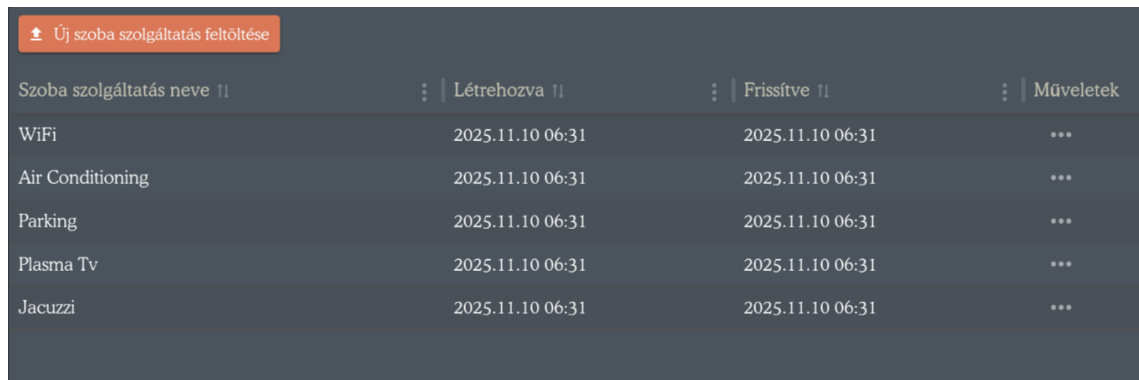
Szoba szolgáltatások felület

Mikor használok? Ezen az oldalon állíthatom be, hogy a szálloda szobatípusai milyen szolgáltatásokkal rendelkezhetnek (pl. WiFi, légkondicionáló, minibar).

Specifikus mező: Szoba szolgáltatás neve

Elérés a menüből: Adatbank => Szolgáltatások

Törlési megszorítás: A szolgáltatást csak akkor törölhetem, ha nincsenek olyan szobatípusok, amelyek használják az adott szolgáltatást. Ellenkező esetben a rendszer hibaüzenetet jelenít meg.



Szoba szolgáltatás neve	Létrehozva	Frissítve	Műveletek
WiFi	2025.11.10 06:31	2025.11.10 06:31	...
Air Conditioning	2025.11.10 06:31	2025.11.10 06:31	...
Parking	2025.11.10 06:31	2025.11.10 06:31	...
Plasma Tv	2025.11.10 06:31	2025.11.10 06:31	...
Jacuzzi	2025.11.10 06:31	2025.11.10 06:31	...

2.12. ábra. Szoba szolgáltatások felület

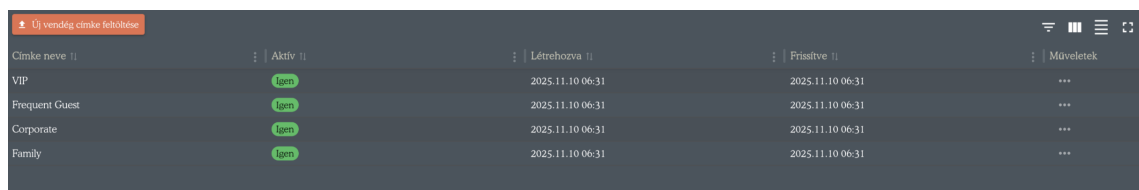
Vendég címkék felület

Mikor használok? Ezen az oldalon állíthatom be, hogy milyen címkéket adhatok egyes vendégeknek (pl. törzsvendég, mozgáskorlátozott, VIP). Így a megfelelő címkézéssel könnyíthetem a recepciós munkatársaim munkáját.

Specifikus mezők: Címke neve, Aktív-e

Elérés a menüből: Adatbank => Vendég címkék

Törlési megszorítás: A címkét csak akkor törölhetem, ha nincs egyetlen vendéghez sem hozzárendelve.



Címke neve	Aktív	Létrehozva	Frissítve	Műveletek
VIP	Igen	2025.11.10 06:31	2025.11.10 06:31	...
Frequent Guest	Igen	2025.11.10 06:31	2025.11.10 06:31	...
Corporate	Igen	2025.11.10 06:31	2025.11.10 06:31	...
Family	Igen	2025.11.10 06:31	2025.11.10 06:31	...

2.13. ábra. Vendég címkék felület

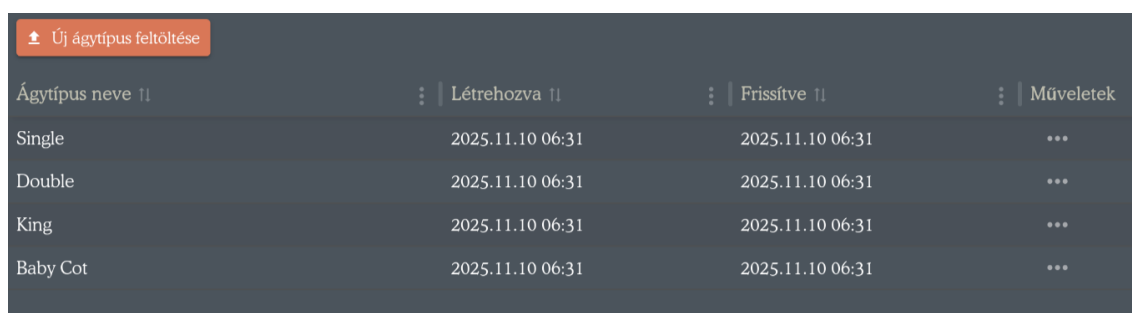
Ágytípusok felület

Mikor használok? Ezen az oldalon állíthatom be, hogy milyen ágytípusokat használok a rendszerben (pl. egyszemélyes, franciaágy, kétszemélyes). Ezeket később a szobatípusok felületen rendelhetem hozzá az egyes szobatípusokhoz megadott darabszámmal.

Specifikus mezők: Ágytípus neve

Elérés a menüből: Adatbank => Ágy típusok

Törlési megszorítás: Az ágytípust csak akkor törölhetem, ha nincs egyetlen szobatípushoz sem hozzárendelve.



Ágytípus neve	Létrehozva	Frissítve	Műveletek
Single	2025.11.10 06:31	2025.11.10 06:31	...
Double	2025.11.10 06:31	2025.11.10 06:31	...
King	2025.11.10 06:31	2025.11.10 06:31	...
Baby Cot	2025.11.10 06:31	2025.11.10 06:31	...

2.14. ábra. Ágytípusok felület

Szerepkörök felület

Mikor használok? Ezen az oldalon állíthatom be, hogy milyen felhasználói szerepkörök léteznek a rendszerben (pl. recepció, menedzser, housekeeping, adminisztrátor).

Specifikus mezők: Szerepkör neve, Aktív-e

Elérés a menüből: Adatbank => Szerepkörök

Törlési megszorítás: A szerepkört csak akkor törölhetem, ha nincs egyetlen felhasználóhoz sem hozzárendelve.

Új szerepkör feltöltése					
Név	Aktív	Létrehozva	Frissítve	Műveletek	
Admin	Igen	2025.11.10 06:31	2025.11.10 06:31	...	
Receptionist	Igen	2025.11.10 06:31	2025.11.10 06:31	...	
Housekeeping	Igen	2025.11.10 06:31	2025.11.10 06:31	...	
Invoice Manager	Igen	2025.11.10 06:31	2025.11.10 06:31	...	
Data maintainer	Igen	2025.11.10 06:31	2025.11.10 06:31	...	
Default User	Igen	2025.11.10 06:31	2025.11.10 06:31	...	

2.15. ábra. Szerepkörök felület

Megjegyzés: A szerepkörökhöz tartozó jogosultságokat a 6.1. függelékben részletezem.

Felhasználók felület

Mikor használok? Ezen az oldalon kezelhetem a rendszerben lévő felhasználók adatait, módosíthatom a szerepköreiket, valamint aktiválhatom vagy deaktiválhatom őket. A rendszerbe csak aktív felhasználó képes belépni.

Specifikus mezők: Keresztnév, Vezetéknév, Email, Telefonszám, Szerepkörök, Aktív-e

Elérés a menüből: Adatbank => Felhasználók

Törlési megszorítás: A felhasználó törölhető, függetlenül attól, hogy vannak-e hozzá tartozó műveletek a rendszerben. Önmagamat nem tudom törölni.

Új felhasználó létrehozása						
Keresztnév	Vezetéknév	E-mail	Telefon	Szerepkörök	Aktív	Műveletek
Szűrés: Keres...	Szűrés: Vezet...	Szűrés: E-mail alapján	Szűrés: Telefo...	Szűrés: Szerepkörök ala...	Szűrés: ...	
Admin	User	admin@hotel.com	+1-555-23123	Admin	Igen	...
Basic	User	basic@hotel.com	+1-555-123122	Default User	Igen	...

2.16. ábra. Felhasználók felület

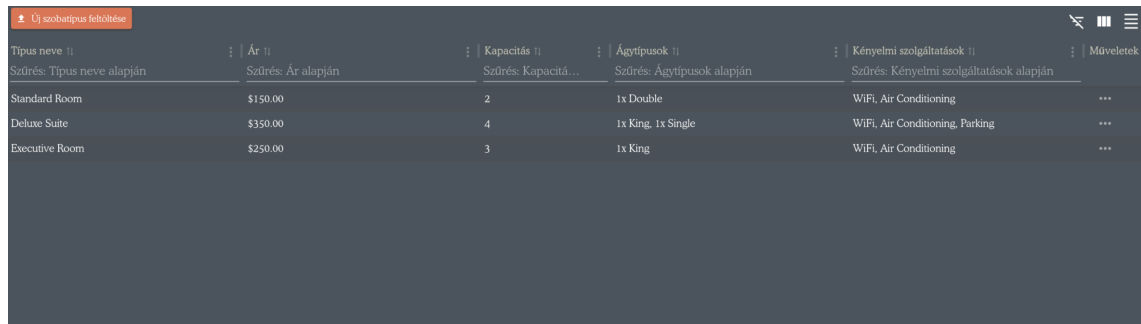
Szobatípusok felület

Mikor használok? Ezen az oldalon kezelhetem a szobák típusait, amikor új szobatípust szeretnék létrehozni (pl. Standard kétágyas, Deluxe lakosztály), vagy módosítani szeretném egy meglévő típus paramétereit. Megadhatom, hogy egy adott típus hány és milyen ágyakkal, valamint milyen szoba szolgáltatásokkal rendelkezik. Az egy éjszakás alapárat is itt állíthatom be.

Specifikus mezők: Típus neve, Ár, Kapacitás, Ágytípusok (darabszám x ágy-típus), Szoba szolgáltatások, Aktív-e

Elérés a menüből: Adatbank => Szoba Típusok

Törlési megszorítás: A szobatípust csak akkor törölhetem, ha nincs egyetlen szoba sem az adott típushoz rendelve.



Típus neve	Ár	Kapacitás	Ágytípusok	Kényelmi szolgáltatások	Műveletek
Standard Room	\$150.00	2	1x Double	WiFi, Air Conditioning	...
Deluxe Suite	\$350.00	4	1x King, 1x Single	WiFi, Air Conditioning, Parking	...
Executive Room	\$250.00	3	1x King	WiFi, Air Conditioning	...

2.17. ábra. Szobatípusok felület

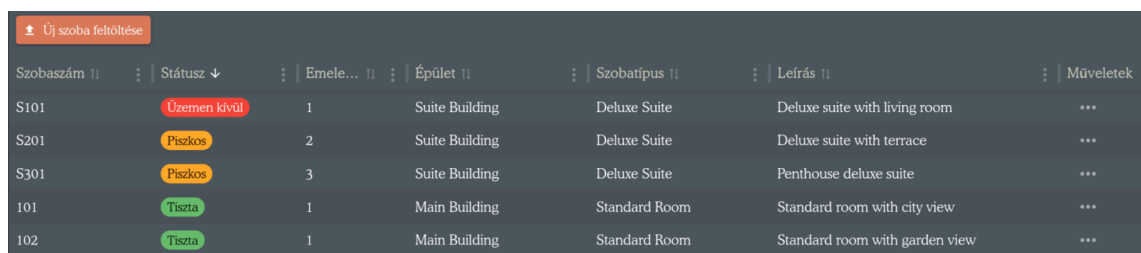
Szobák felület

Mikor használok? Ezen az oldalon kezelhetem a rendszerben létező szobákat, amikor új szobát szeretnék létrehozni vagy módosítani szeretném egy meglévő szoba paramétereit vagy tisztasági státuszát.

Specifikus mezők: Szobaszám, Státusz, Emelet, Épület, Szobatípus, Leírás

Elérés a menüből: Adatbank => Szobák

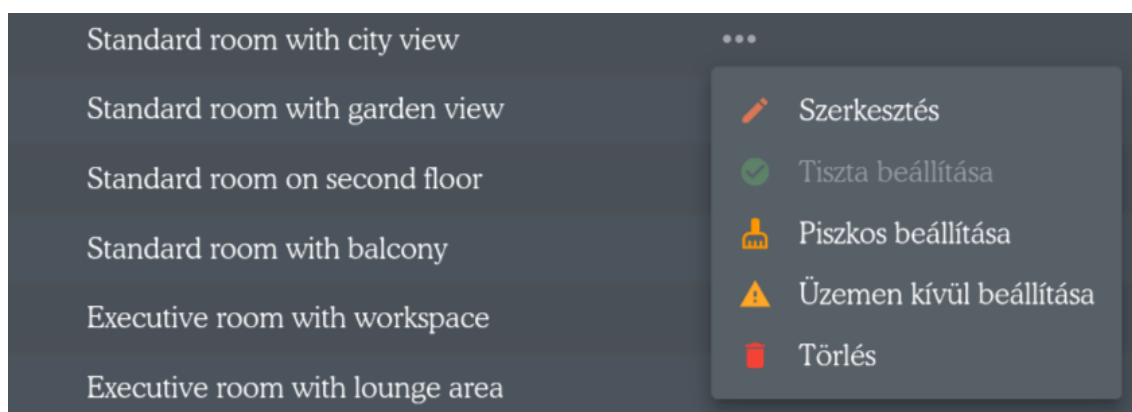
Törlési megszorítás: A szobát csak akkor törölhetem, ha nincs hozzá kapcsolt foglalás. Ha van hozzákapcsolt takarítási kérvény, a szoba törlésével a kérvény is megszűnik.



Szobaszám	Státusz	Emelet	Épület	Szobatípus	Leírás	Műveletek
S101	Üzemen kívül	1	Suite Building	Deluxe Suite	Deluxe suite with living room	...
S201	Piszkos	2	Suite Building	Deluxe Suite	Deluxe suite with terrace	...
S301	Piszkos	3	Suite Building	Deluxe Suite	Penthouse deluxe suite	...
101	Tiszta	1	Main Building	Standard Room	Standard room with city view	...
102	Tiszta	1	Main Building	Standard Room	Standard room with garden view	...

2.18. ábra. Szobák felület

Szoba specifikus műveletek: A sorok végén lévő ... gombra kattintva a megszokott funkciók mellett találók 3 gyors gombot az adott szoba tisztasági státuszának beállítására



2.19. ábra. Szoba műveletek

Változásnapló felület

Megjegyzés: A változásnapló kizárólag olvasható felület, nem rendelkezik létrehozás, módosítás vagy törlés funkciókkal.

Mikor használok? Itt tudom megnézni a rendszerben történt változtatások vagy fejlesztések listáját.

Specifikus mezők: Verzió, Leírás

Elérés a menüből: Adatbank => Változásnapló

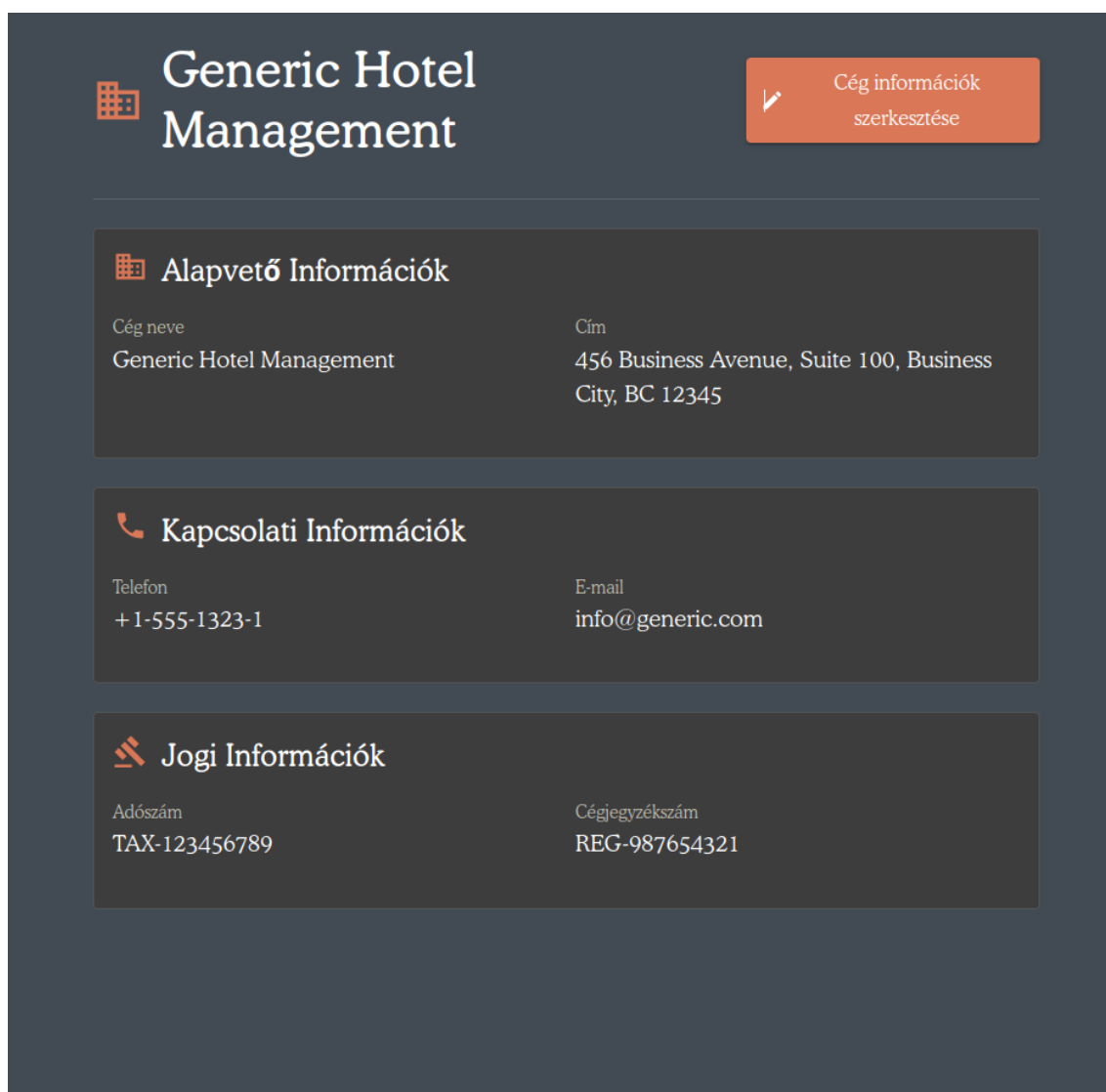
Cég információ felület

Megjegyzés: Ez a felület lényegesen eltér a többi alapadatok felülettől, mivel nem táblázatos megjelenítésű. Nincs rajta törlési vagy új létrehozása funkció, kizárólag módosítani lehet a meglévő cég adatokat.

Mikor használok? Ha szeretném állítani a cégem információit.

Specifikus mezők: Cégnév, Cím, Telefonszám, Vendégeknek szóló weboldal (ha van), Adószám, Cégjegyzék szám

Elérés a menüből: Adatbank => Cég információk



Generic Hotel Management

Cég információk szerkesztése

Alapvető Információk

Cég neve	Cím
Generic Hotel Management	456 Business Avenue, Suite 100, Business City, BC 12345

Kapcsolati Információk

Telefon	E-mail
+1-555-1323-1	info@generic.com

Jogi Információk

Adószám	Cégjegyzékszám
TAX-123456789	REG-987654321

2.20. ábra. Cég információ felület

2.5.3. Recepció és foglalások kezelése

A recepció menüpont három feleletül rendelkezik: Vendégek, Foglalások, és szoba tükkör. Ezen felületek megtekintéséhez és szerkesztéséhez szükséges megfelelő jogosultság lásd: 6.1. függelék.

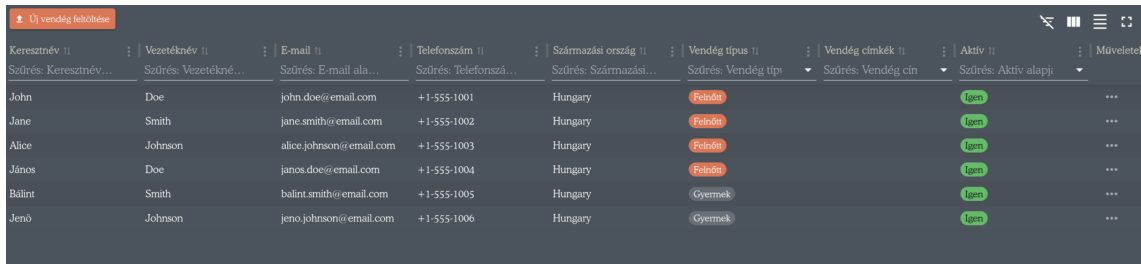
Vendégek felület

Mire használok? Ezen az oldalon kezelhetem a rendszerben tárolt vendégeket. Új vendéget vehetek fel, módosíthatom a meglévő vendégek adatait, vagy deaktiválhatom őket. Foglalások felvétele során tudom majd őket hozzájuk rendelni.

Táblázat és szűrők Ez a felület megjelenítésben és funkciókban hasonlít az alapadatokon belül található felületekre. Ugyanúgy egy szűrhető táblázat jelenik meg különböző táblázat műveletekkel (lásd 2.5.2.).

Specifikus mezők: Keresztnév, Vezetéknév, Email, Telefonszám, Származási ország, Vendég típus (Felnőtt, gyerek), Vendég címkék, Aktív-e

Elérés a menüből: Recepció => Vendégek



Keresztnév	Vezetéknév	E-mail	Telefonszám	Származási ország	Vendég típus	Vendég címkék	Aktív	Műveletek
John	Doe	john.doe@email.com	+1-555-1001	Hungary	Felnőtt		Igen	...
Jane	Smith	jane.smith@email.com	+1-555-1002	Hungary	Felnőtt		Igen	...
Alice	Johnson	alice.johnson@email.com	+1-555-1003	Hungary	Felnőtt		Igen	...
János	Doe	janos.doe@email.com	+1-555-1004	Hungary	Felnőtt		Igen	...
Bálint	Smith	balint.smith@email.com	+1-555-1005	Hungary	Gyerek		Igen	...
Jenő	Johnson	jeno.johnson@email.com	+1-555-1006	Hungary	Gyerek		Igen	...

2.21. ábra. Vendégek felület

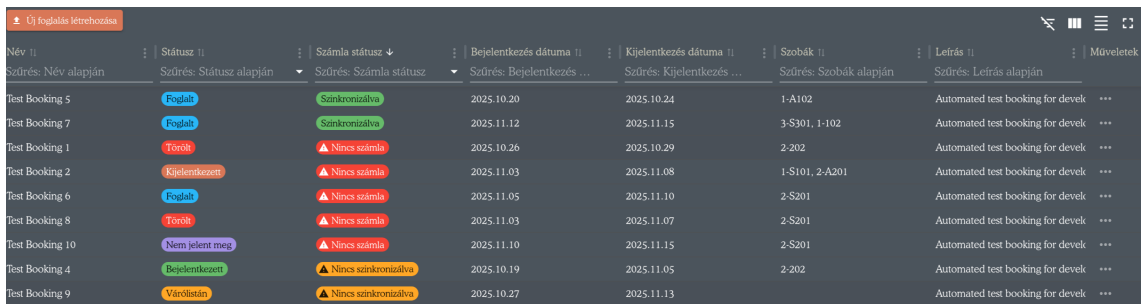
Műveletek:

- **Új vendég felvétele:** A táblázat feletti **Új vendég felvétele** gombbal hozhatók létre új vendégek.
- **Szerkesztés:** A sorok végén lévő ... gombra kattintva módosíthatom a meglévő vendégek adatait.
- **Törlés:** Szintén a ... gombon keresztül érhető el. Megerősítés után a vendég nem törlődik véglegesen, hanem inaktív státuszba kerül, így megmaradnak a korábbi foglalásai és adatai a rendszerben.

Foglalások felület

Mire használok? Ezen az oldalon kezelhetem a rendszerben tárolt foglalásokat.

Elérés a menüből: Recepció => Foglalás



Név	Státusz	Számla státusz	Bejelentkezés dátuma	Kijelentkezés dátuma	Szobák	Leírás	Műveletek
Test Booking 5	Foglalt	Szinkronizálva	2025.10.20	2025.10.24	1-A102	Automated test booking for delevk	...
Test Booking 7	Foglalt	Szinkronizálva	2025.11.12	2025.11.15	3-S301, 1-102	Automated test booking for delevk	...
Test Booking 1	Törölt	Nincs számla	2025.10.26	2025.10.29	2-202	Automated test booking for delevk	...
Test Booking 2	Kijelentkezett	Nincs számla	2025.11.03	2025.11.08	1-S101, 2-A201	Automated test booking for delevk	...
Test Booking 6	Foglalt	Nincs számla	2025.11.05	2025.11.10	2-S201	Automated test booking for delevk	...
Test Booking 8	Törölt	Nincs számla	2025.11.03	2025.11.07	2-S201	Automated test booking for delevk	...
Test Booking 10	Nem jelent meg	Nincs számla	2025.11.10	2025.11.15	2-S201	Automated test booking for delevk	...
Test Booking 4	Bejelentkezett	Nincs szinkronizálva	2025.10.19	2025.11.05	2-202	Automated test booking for delevk	...
Test Booking 9	Vároltán	Nincs szinkronizálva	2025.10.27	2025.11.13		Automated test booking for delevk	...

2.22. ábra. Foglalások felület

Táblázat és szűrők Ez a felület megjelenítésben és funkciókban hasonlít az alapadatokon belül található felületekre. Ugyanúgy egy szűrhető táblázat jelenik meg különböző táblázat műveletekkel (lásd 2.5.2.).

Specifikus mezők:

- Név: a foglalás neve
- Aktiv-e
- Státusz: Foglalt, bejelentkezett, kijelentkezett, várólistán, blokkolt, nem jelent meg, törölt
- Számla státusz: nincsen számla, szinkronizált, nincs szinkronizálva
- Bejelentkezési dátum (Check-in-date)
- Kijelentkezési dátum (Check-out-date)
- Szobák: vesszővel elválasztva egymás után
- Leírás

Műveletek:

- Új foglalás létrehozása
- Meglévő foglalás információs ablak megnyitása
- Meglévő foglalás szerkesztése
- Meglévő foglalás státusz váltása
- Számla művelete: létrehozás, szinkronizálás és megtekintés
- Törlés

Új foglalás létrehozása művelet:

1. Rákattintok a táblázatt feletti **Új foglalás létrehozása** gombra Ekkor egy többlépcsős űrlap fogad.
2. **1. lépés - Alapadatok megadása:**
 - Megadom a foglalás nevét
 - Kiválasztom a bejelentkezési és kijelentkezési dátumot. Ha szabálytalan dátumokat adnék meg, a rendszer értesít
 - Opcionálisan megjegyzést is írhatok
 - (lásd 2.23. (a) ábra)
3. **2. lépés - Vendégek kiválasztása:**

- Kiválasztom a rendszerben már meglévő vendégeket
- Ha olyan vendéget szeretnék felvenni, aki még nincs a rendszerben, azt megtehetem ezen lépés során az **Új vendég hozzáadása** opcióval
- Vendégek felvételéről és megtekintéséről lásd: 2.5.3
- (lásd 2.23. (b) ábra)

4. **3. lépés - Szobák kiválasztása:**

- Az elérhető szobák listájából választok. Csak azok a szobák jelennek meg, amelyek elérhetők a tartózkodás időtartama során és más foglalás nem foglalja őket
- Szükség esetén leellenőrizhetem, hogy az adott szobának megvan-e minden kért funkciója
- Szoba nélküli foglalás esetén a rendszer automatikusan **Várólistán** státuszt állít be a **Foglalt** státusz helyett
- (lásd 2.23. (c) ábra)

5. Ellenőrzöm az adatokat és elmentem a foglalást a **Mentés** gombbal

Új foglalás létrehozása

Alapadatok / Vendégek / Szobák

Név
Proba új Foglalás

Bejelentkezés dátuma
2025-11-04

Kijelentkezés dátuma
2025-11-14

Leírás
Minta leírás

Mégse Következő

(a) 1. lépés: Alapadatok

Új foglalás létrehozása

Alapadatok / Vendégek / Szobák

Vendégek

Vendégek kiválasztása
Bálint Smith Jenő Johnson Keresés név vagy e-mail szerint...

Válasszon ki egy vagy több vendéget ehhez a foglaláshoz.

Mégse Vissza Következő

(b) 2. lépés: Vendégek

Új foglalás létrehozása

Alapadatok / Vendégek / Szobák

3 kiválasztva 6 sorból Kiválasztás törlése

»	»	Épület	Szobaszám	Szobatípus	Státusz										
✓	✓	Main Building	101	Standard Room	Tiszta										
✓	□	Main Building	201	Standard Room	Tiszta										
✓	□	Main Building	202	Standard Room	Tiszta										
✓	✓	Annex Building	A101	Executive Room	Tiszta										
<table border="1"> <thead> <tr> <th>Type Name</th> <th>Price</th> <th>Capacity</th> <th>Bed Configuration</th> <th>Amenities</th> </tr> </thead> <tbody> <tr> <td>Executive Room</td> <td>\$250</td> <td>3 guests</td> <td>1x King</td> <td>Air Conditioning WiFi</td> </tr> </tbody> </table>						Type Name	Price	Capacity	Bed Configuration	Amenities	Executive Room	\$250	3 guests	1x King	Air Conditioning WiFi
Type Name	Price	Capacity	Bed Configuration	Amenities											
Executive Room	\$250	3 guests	1x King	Air Conditioning WiFi											
✓	✓	Annex Building	A102	Executive Room	Tiszta										
✓	□	Suite Building	S101	Deluxe Suite	Üzemen kívül										

Sorok oldalanként 10 1-6 összesen 6

Mégse Vissza Létrehozás

(c) 3. lépés: Szobák

2.23. ábra. Új foglalás létrehozásának lépései

Meglévő foglalás információs ablak megnyitása A sorok végén lévő ... gombra kattintva megnyomhatom az **Információ** gombot, ami megnyit egy részletes leírást az adott foglalásról. Itt megtekinthető a foglaláshoz kapcsolódó vendégek, szobák és számla. Emellett a foglalás alap- és metainformációi is megjelennek.

Meglévő foglalás szerkesztése A sorok végén lévő ... gombra kattintva megnyomhatom a **Szerkesztés** gombot, amivel a foglalás adataival előre kitöltött többlépcsős űrlap nyílik meg. Ugyanazokkal a lépésekkel rendelkezik, mint az **Új foglalás létrehozása** űrlap (lásd 2.23. ábra).

Meglévő foglalás státusz váltása A sorok végén lévő ... gombra kattintva megnyomhatom a **Státusz módosítás** gombot, amivel megnyílik egy egyetlen mezővel rendelkező űrlap, ahol kiválaszthatom a kívánt státuszt (foglalt, bejelentkezett, kijelentkezett, várólistán, blokkolt, nem jelent meg, törölt).

Nem minden esetben fogadja el a rendszer a státusz módosítást. Például ha egy kijelentkezett státuszból próbálok foglaltra visszaváltani, de a foglalásban lévő szobákat már valaki más lefoglalta, ilyenkor a rendszer tájékoztat a hibáról. Emellett ha egy foglalást várólistás státuszba helyezek, az megszünteti a szobáival való kapcsolatot.

Számla műveletek: létrehozás, szinkronizálás és megtekintés A sorok végén lévő ... gombra kattintva találom meg az alábbi gombokat:

- **Számla létrehozása:** Létrehoz egy új számlát és hozzárendeli ehhez a foglaláshoz
- **Számla szinkronizálás:** Szinkronizálja a számlához kiszámolt teljes és szobánkénti árat a foglalás adataival. Akkor van rá szükség, ha a foglalás adatai változtak, pl. még két napot szeretnének maradni a vendégek
- **Számla megtekintése:** Átnavigál a számlák felületre (lásd 2.5.4. oldal)

Törlés A sorok végén lévő ... gombra kattintva megnyomhatom a **Törlés** gombot, amivel megerősítés után törlésre kerül az adott foglalás. Későbbi visszakereshetőség érdekében a foglalás tárolva marad, csak nem lesz többé aktív.

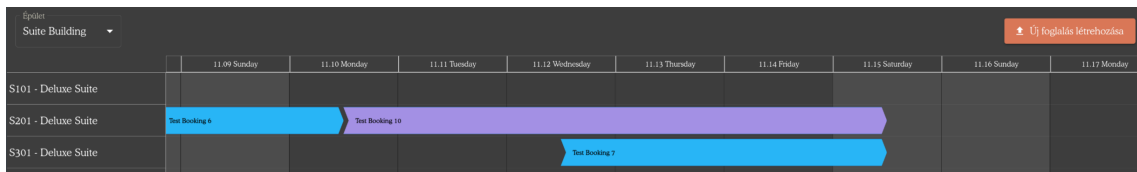
Szoba tükör felület

Mire használok? Ezen az oldalon egy vizuális reprezentációt látok az aktív foglalásokról naptár nézetben, szobánként és épületenként. A szobatükör időrendi sorrendben jeleníti meg a szobák foglaltságát, így gyorsan áttekinthetők a szabad kapacitások és az átfedések.

Elérés a menüből: Recepció => Szoba Tükör

Műveletek A szobatükör felületen elérhető műveletek megegyeznek a Foglalások felületen leírtakkal:

- **Új foglalás létrehozása:** Ugyanaz a többlépcsős folyamat, mint a Foglalások felületen (lásd 2.23. ábra)
- **Foglalás információk megtekintése:** Dupla kattintással megnyílik a részletes információs ablak, ahogy a Foglalások felületen is (lásd 2.5.3)
- **Szűrés:** A naptár felett kiválaszthatom, hogy melyik épület szobáit szeretném megjeleníteni



2.24. ábra. Szobatükör felület

2.5.4. Számla műveletek kezelése

Számla művelet három felületen keresztül tudom kezelni: Szolgáltatások, Számlák, és ÁFA kulcsok felület. Ezen felületek megtekintéséhez és szerkesztéséhez szükséges megfelelő jogosultság lásd: 6.1. függelék.

ÁFA kulcsok felület

Mire használok? Ezen az oldalon kezelhetem a rendszerben használt ÁFA kulcsokat, amelyeket később a szolgáltatások árának kiszámításához használok fel.

Elérés a menüből: Számla Adatok => ÁFA

Specifikus mezők: ÁFA kulcs neve (pl. Normál, Kedvezményes), ÁFA mértéke (%)

Táblázat és szűrők Ez a felület megjelenítésben és funkciókban hasonlít az alapadatokon belül található felületekre. Ugyanúgy egy szűrhető táblázat jelenik meg különböző táblázat műveletekkel (lásd 2.5.2. oldal).

Név	Százalék	Művelet
Standard VAT	10%	...
Reduced VAT	5%	...
Luxury VAT	23%	...

2.25. ábra. ÁFA kulcsok felület

Műveletek:

- **Új ÁFA kulcs felvétele:** A táblázat feletti **Új** gombbal hozhatók létre új ÁFA kulcsok.
- **Szerkesztés:** A sorok végén lévő ... gombra kattintva módosíthatom a meglévő ÁFA kulcsok adatait.
- **Törlés:** Szintén a ... gombon keresztül érhető el, de csak akkor engedélyezett, ha az ÁFA kulcshoz nincs hozzárendelve szolgáltatás. Ellenkező esetben a rendszer hibaüzenetet jelenít meg.

Szolgáltatások felület

Mire használom? Ezen az oldalon kezelhetem a rendszerben elérhető szolgáltatásokat, amelyeket később a számlákhoz rendelek hozzá. Ezek a szolgáltatások építik fel a végső számlát (pl. szobák költsége, spa csomag, stb.).

Elérés a menüből: Számla Adatok => Szolgáltatások

Specifikus mezők: Név, Leírás, Költség, ÁFA kulcs

Táblázat és szűrők Ez a felület megjelenítésben és funkciókban hasonlít az alapadatokon belül található felületekre. Ugyanúgy egy szűrhető táblázat jelenik meg különböző táblázat műveletekkel (lásd 2.5.2.).

Név	Leírás	Költség	ÁFA	Műveletek
Szűrés: Név alapján	Szűrés: Leírás alapján	Szűrés: Költség alapján	Szűrés: ÁFA alapján	
Rooms aggregated cost	Base service model for room costs in invoices	\$0.00	Standard VAT (10%)	...
Spa Package	Relaxing spa treatment	\$120.00	Luxury VAT (23%)	...
Room Service	24/7 room service	\$45.00	Standard VAT (10%)	...
Laundry Service	Express laundry	\$25.00	Standard VAT (10%)	...

2.26. ábra. Szolgáltatások felület

Műveletek:

- **Új szolgáltatás felvétele:** A táblázat feletti **Új** gombbal hozhatók létre új szolgáltatások.
- **Szerkesztés:** A sorok végén lévő ... gombra kattintva módosíthatom a meglévő szolgáltatások adatait.
- **Törlés:** Szintén a ... gombon keresztül érhető el. Megerősítés szükséges a sikeres törléshez. Nem törölhetek olyan szolgáltatást, amit használ egy számla.

Speciális szolgáltatás: Rooms aggregated cost

A "Rooms aggregated cost" szolgáltatás speciális kezelést igényel:

- **Nem törölhető:** Ez a szolgáltatás alapértelmezett, és nem lehet törölni a rendszerből.
- **Korlátozott szerkesztés:** Csak az ÁFA kulcsa módosítható.
- **Automatikus árszámítás:** A költsége automatikusan kiszámítódik az adott számlában szereplő szobák alapján, manuálisan nem állítható.

Számlák felület

Mire használok? Ezen az oldalon kezelhetem a rendszerben lévő számlákat. Megtekinthetek, létrehozhatok, szerkeszthetek, nyomtathatok és törölhetek számlákat. A számlák a foglalásokhoz kapcsolódó szobák és egyéb szolgáltatások költségeit tartalmazzák az alap számlainformációk mellett.

A Számlák felület 2 fő részből áll: bal oldalt egy vékony szűrhető táblázat (lásd 2.5.2. oldal szűrési és beállítási lehetőségeit), amely listázza a számlákat, jobb oldalt pedig a munkafelület. Ha a bal oldali táblázatban kattintással kiválasztok egy számlát, az megnyílik minden adatával a jobb oldalon. A részek mérete állítható az elválasztó húzásával.

(a) Számlák táblázat - felső rész

Név	Leírás	Költség	ÁFA	Művelet
Room Service	24/7 room service	\$45.00	Standard VAT (10%)	
Rooms aggregated cost	Base service model for room costs in invoices	\$450.00	Standard VAT (10%)	
Rooms aggregated cost	Base service model for room costs in invoices	\$1750.00	Standard VAT (10%)	
Rooms aggregated cost	Base service model for room costs in invoices	\$1400.00	Standard VAT (10%)	
		Összesen ÁFA nélkül: \$3645.00	Összesen ÁFA-val: \$4009.50	

Sorok oldalanként 10 1-4 összesen 4

Foglalások

Test Booking 1
2025.10.26 00:00 - 2025.10.29 00:00
CANCELLED

Test Booking 6
2025.11.05 00:00 - 2025.11.10 00:00
RESERVED

Test Booking 8
2025.11.03 00:00 - 2025.11.07 00:00
CANCELLED

+
Foglalás hozzáadása

(b) Számlák táblázat - alsó rész

2.27. ábra. Számlák felület teljes nézet

Specifikus mezők: Név, Fizetési státusz, Leírás, Végösszeg, Címzett adatok, Szolgáltatások listája, Foglalások listája

Műveletek

- Új számla létrehozása
- Meglévő számla alapadatainak szerkesztése
- Meglévő számla címzett adatainak szerkesztése
- Meglévő számla nyomtatása
- Meglévő számla szolgáltatásainak szerkesztése
- Meglévő számla foglalásainak szerkesztése
- Törlés

Új számla létrehozása Rákattintok a bal részen lévő táblázat feletti **Új számla létrehozása** gombra, amivel megnyílik egy egyetlen mezővel rendelkező űrlap, ahol a számla nevét megadhatom. Mentés után létrejön egy új üres számla.

Meglévő számla alapadatainak szerkesztése A jobb oldali munkafelületen az Alapadatok felett rákattintok a **Szerkesztés** gombra, és így már átírhatom a számla alapadatait. Új nevet adhatok neki, leírást, vagy akár a státuszát is módosíthatom. Miután befejeztem a szerkesztést, elmenthetem a változtatásaimat a **Mentés** gombbal, ami a szerkesztés gomb helyett jelent meg, vagy visszaállíthatom az eredeti számla adatait a mellette lévő **Visszaállítás** gombbal.

Meglévő számla címzett adatainak szerkesztése A jobb oldali munkafelületen a Címzett adatok felett rákattintok a **Szerkesztés** gombra, és így módosíthatom a számla címzett adatait (név, cégnév, email, telefon, cím, adószám, ország). Miután befejeztem a szerkesztést, elmenthetem a változtatásaimat a **Mentés** gombbal, vagy visszaállíthatom az eredeti adatokat a **Visszaállítás** gombbal.

Meglévő számla nyomtatása A számla neve mellett lévő **Nyomtatás** gombra kattintva kinyomtathatom a számlát



2.28. ábra. Számla nyomtatás gomb

Meglévő számla szolgáltatásainak szerkesztése A jobb oldali munkafelületen a Szolgáltatások táblázatban (lásd 2.27. (b) ábra) a + gombbal új szolgáltatást adhatok hozzá, amelyet kiválaszthatok a rendszerben lévő szolgáltatások közül. A táblázat sorok végén lévő **X** gombbal törölhetek szolgáltatást.

Fontos: A szoba ár szolgáltatást ("Rooms aggregated cost") nem törölhetem, mert az automatikusan a foglalásokkal jön és megy.

Meglévő számla foglalásainak szerkesztése A jobb oldali munkafelületen a Foglalások táblázatban (lásd 2.27. (b) ábra) a + gombbal új foglalást adhatok a számlához. Kiválaszthatok a rendszerben lévő foglalások közül olyan foglalást, aminek még nincs számlája. Egy számlának lehet 0, 1 vagy több foglalása is.

Egy adott foglalásra ha ráviszem az egeret, tudom eltávolítani a számláról az **X** gombbal. A foglalások módosítása után a teljes ár és a szolgáltatások automatikusan újraszámolódnak.

Törlés A számla neve mellett lévő kuka gombbal, megerősítés után törölhetem a számlát.



2.29. ábra. Számla törlése gomb

2.5.5. Takarítások kezelése

Ezen a felületen vehetek fel egyes szobákra takarítási kérést, rendelhetem hozzá egyes munkatársakhoz az adott kérést, vagy egyes munkatársaim jelezhetik, hogy a kérés végbement és a szoba kitakarítva (ebben az esetben a szoba státusza automatikusan tiszta lesz).

Elérés a menüből: Takarítás => Áttekintés

Feladat létrehozása		Feladataim megjelenítése					
Szoba	Hozzárendelve	Státusz	Prioritás	Megjegyzés	Hozzárendelés dátuma	Befejezés dátuma	Művelet
Szűrő: Szoba ala...	Szűrő: Hozzáren...	Szűrő: Státusz al...	Szűrő: Prioritás a...	Szűrő: Megjegyz...	Szűrő: Hozzárendelés dátu...	Szűrő: Befejezés ...	
Main Building 201	Admin User	Teendő	Közepes		2025.10.30		...
Main Building 202	Admin User	Folyamatban	Magas		2025.10.30		...
Annex Building A102	Admin User	Kész	Magas		2025.10.30	2025.10.30	...

2.30. ábra. Takarítások felület

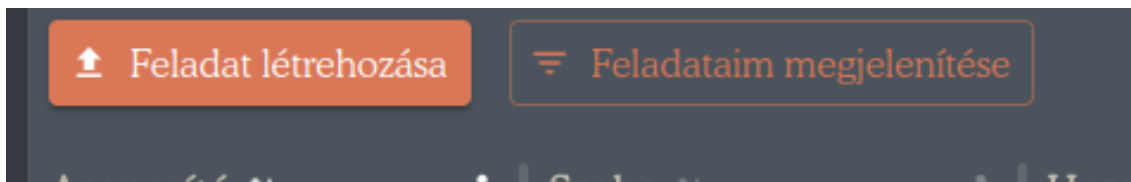
Kérésenként megjelenített mezők a következők:

1. Szoba: Szoba neve és épület
2. Hozzárendelve: Melyik munkatárshoz kell a feladatot elvégeznie
3. Státusz: hogy halad a takarítás? Kész, teendő vagy folyamatban
4. Prioritás: alacsony, közepes, magas vagy sürgős
5. Megjegyzés
6. Hozzárendelés időpontja: mikor osztottam ki a feladatot
7. Befejezés dátuma: mikor lett a feladat befejezve

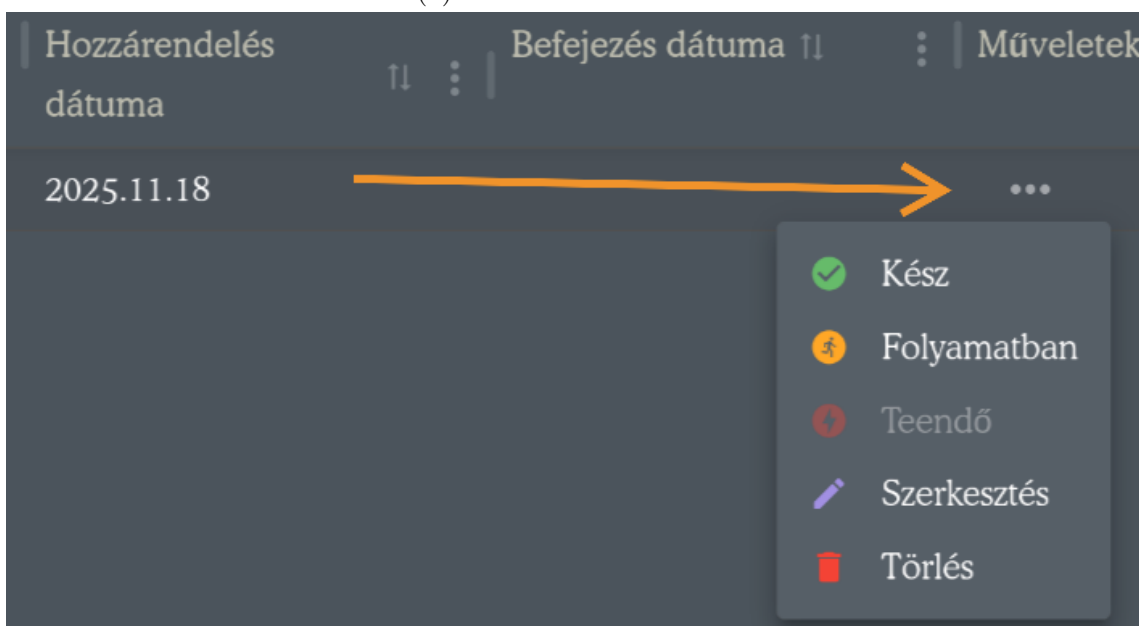
Takarítás felület műveletei A felületen 7 műveletet hajthatok végre:

1. Feladat létrehozása
2. Feladataim megjelenítése

3. Szerkesztés
4. Törlés
5. Kész
6. Folyamatban
7. Teendő



(a) Táblázati műveletek



(b) Soronkénti műveletek

2.31. ábra. Takarítás felület műveletei

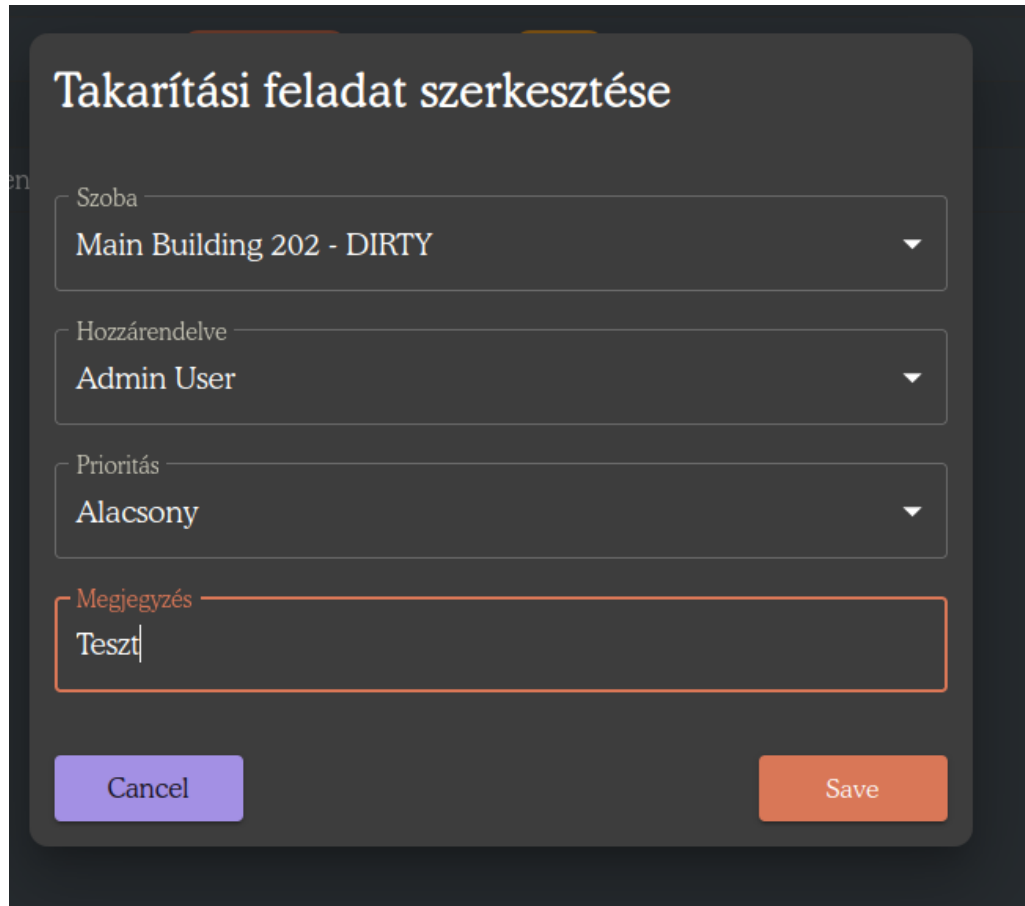
Kész, folyamatban, teendő Kattintásra megerősítés nélkül átállítódik a feladat státusza a kiválasztott értékre. Amennyiben ez az érték kész, akkor a szoba státusza is tisztára módosul, és a feladat kap egy befejezés dátumot.

Törlés Megerősítés után a feladat törlődik.

Feladataim megjelenítése Leszűri a táblázatot csak azon feladatokra, amelyek hozzám tartoznak. Másodszori rákattintásra a szűrést megszünteti.

Szerkesztés és új feladat felvétele A kiválasztott gomb megnyomása után csak címében és kezdeti adataiban eltérő ablakok fogadnak, ahol meg tudom adni a ta-

karítandó szobát, sürgősségét, hozzárendelt munkatársamat és akár opcionális megjegyzést.



Takarítási feladat szerkesztése

Szoba
Main Building 202 - DIRTY

Hozzárendelve
Admin User

Prioritás
Alacsony

Megjegyzés
Teszt

Cancel Save

2.32. ábra. Feladat szerkesztése

Szerkeszteni, törölni vagy új feladatot felvenni csak a megfelelő jogosultággal lehet lásd: 6.1. függelék.

2.6. Hibaüzenetek, figyelmeztetések és visszajelzések

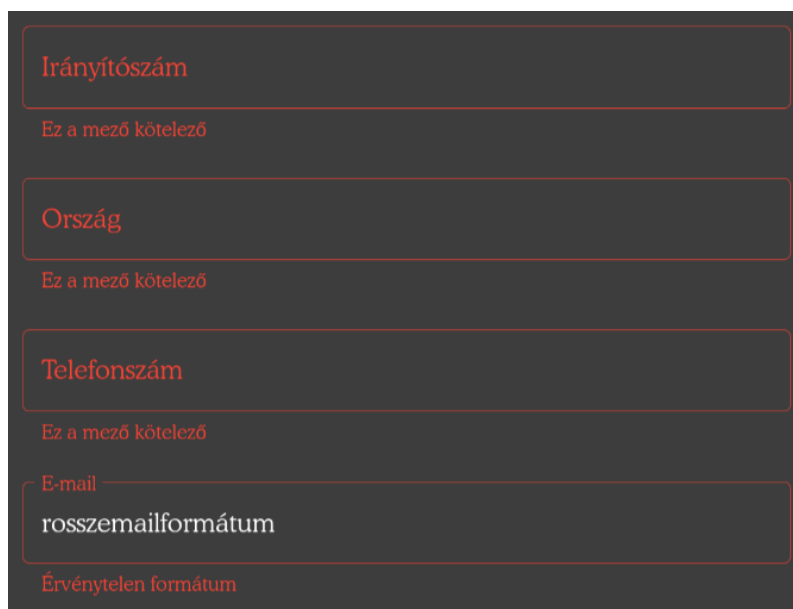
A rendszer különböző helyzetekben hibaüzenetekkel, figyelmeztetésekkel és visszajelzésekkel tájékoztat használat során. Ezek segítenek az űrlapok helyes kitöltésében, műveletek eredményének megjelenítésébe.

2.6.1. Űrlap validációs hibák

Ezek az üzenetek akkor jelennek meg, amikor hibásan vagy hiányosan töltöm ki az űrlapokat. A rendszer nem engedi a mentést, amíg a hibákat nem javítom.

Gyakori példák:

- **Kötelező mező hiánya:** "Ez a mező kötelező"
- **Hibás email vagy telefonszám formátum:** "Érvénytelen formátum"
- **Hibás dátum:** "A kijelentkezés dátumának a bejelentkezés dátuma után kell lennie"

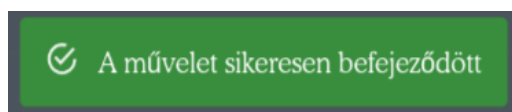


The image shows a dark-themed registration form with four input fields. Each field has a red border and a red error message below it. The fields and their messages are: 1. 'Irányítószám' (Postal code) with the message 'Ez a mező kötelező' (This field is required). 2. 'Ország' (Country) with the message 'Ez a mező kötelező' (This field is required). 3. 'Telefonszám' (Phone number) with the message 'Ez a mező kötelező' (This field is required). 4. 'E-mail' (Email) with the message 'rosszemailformátum' (Wrong email format) and 'Érvénytelen formátum' (Invalid format).

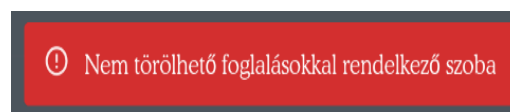
2.33. ábra. Példa űrlap validációs hibákra

2.6.2. Műveletek végrehajtási üzenetek

A rendszer minden művelet után visszajelzést ad arról, hogy a művelet sikeres volt-e vagy hiba történt. A sikeres műveletek zöld háttérrel, a hibák piros háttérrel jelennek meg.



(a) Sikeres művelet üzenete



(b) Sikertelen művelet hibaüzenete

2.34. ábra. Művelet végrehajtási üzenetek

Gyakori hibaüzenetek:

- **Általános hibák:**
 - "Váratlan hiba történt"
 - "Hálózati hiba"
 - "Erőforrás nem található"
 - "Az erőforrás már létezik"

- "Hozzáférs megtagadva"
- **Vendégek kezelésekor:**
 - "Vendég nem található"
 - "Ezzel az e-mail címmel már létezik vendég"
 - "Ezzel a telefonszámmal már létezik vendég"
- **Foglalások kezelésekor:**
 - "Foglalás nem található"
 - "Érvénytelen foglalási státuszváltás"
- **Számlák kezelésekor:**
 - "Számla nem található"
 - "Nem sikerült szinkronizálni a számlát a foglalásokkal"
 - "Ez a foglalás egy másik számlához tartozik"
 - "Hiba a számla összegének kiszámításakor"
- **Szobák kezelésekor:**
 - "Szoba nem található"
 - "A szoba nem elérhető"
 - "Nem törölhető foglalásokkal rendelkező szoba"
- **Alapadatok törlésekor:**
 - "Nem törölhető szobákkal rendelkező épület"
 - "Nem törölhető szobákkal rendelkező szobatípus"
 - "Nem törölhető szolgáltatás modellekkel rendelkező ÁFA"
 - "Nem törölhető vendégekkel rendelkező vendég címke"
 - "Nem törölhető szobatípusokkal rendelkező ágytípus"
- **Bejelentkezésakor és jogosultságok:**
 - "Érvénytelen bejelentkezési adatok"
 - "Elégtelen jogosultságok"
 - "A fiók le van tiltva"

3. fejezet

Fejlesztői dokumentáció

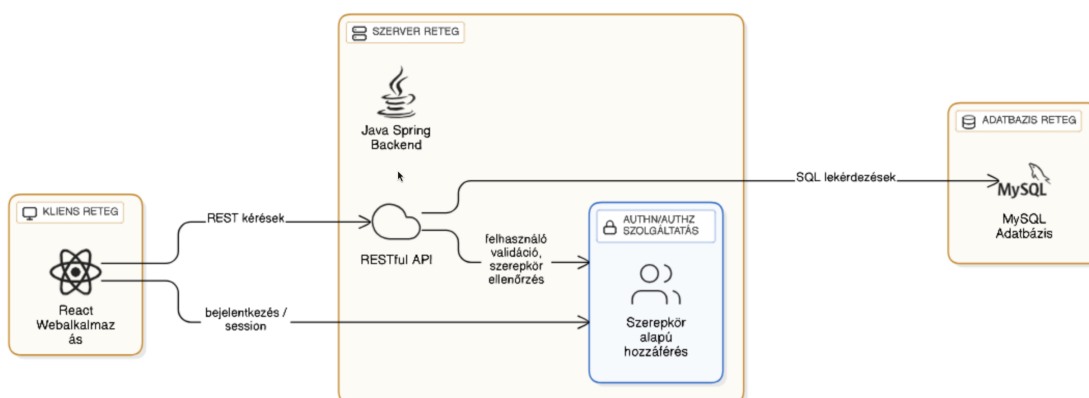
3.1. Bevezetés

A fejlesztői dokumentációban bemutatom a webalapú hotelmendzsmnt és foglalási rendszer tervezési folyamatát, architektúráját, implementációs döntéseit. A dokumentáció alapján a rendszer továbbfejleszthető és karbantartható.

3.2. Rendszer architektúra

A rendszer háromrétegű architektúrát használ:

- **Kliens réteg (Frontend):** React alapú webalkalmazás
- **Szerver réteg (Backend):** Java spring és restful api
- **Adatbázis réteg:** MySQL relációs adatbázis



3.1. ábra. Rendszer háromrétegű architektúrája

Rétegek közötti kommunikáció

- **Frontend → Backend:** HTTP kérések (GET, POST, PUT, DELETE, PATCH) Axios kliens segítségével frontenden
- **Backend → Adatbázis:** JPA/Hibernate ORM
- **Autentikáció:** JWT (JSON Web Token) alapú hitelesítés
- **API prefix:** /api - minden backend endpoint ezzel kezdődik

Backend felépítése

A backend háromrétegű (Controller-Service-Repository) architektúrát követ:

- **Controller réteg:** HTTP kérések fogadása, válaszok küldése
- **Service réteg:** Üzleti logika megvalósítása
- **Repository réteg:** Adatbázis elérés JPA-n keresztül
- **Model réteg:** JPA entitások az adatbázis táblák reprezentálására
- **DTO réteg:** Adatátviteli rétegek közt
- **Security réteg:** JWT szűrők és Spring Security konfiguráció

Frontend felépítése

A frontend moduláris felépítésű:

- **Pages (Oldalak):** 23 oldal-szintű komponens
- **Components (Komponensek):** Újrafelhasználható UI elemek
- **API modulok:** 19 API kliens modul (amenityApi, bookingApi, stb.)
- **State (Állapot):** Jotai atom-ok globális állapotkezelésre
- **Router:** React Router külön auth és fő router-rel
- **Theme:** Material-UI téma testreszabás
- **i18n:** Többnyelvű támogatás (magyar, angol)

Konténerizáció és deployment

A rendszer Docker konténerekben fut:

- **Frontend konténer:** Node.js 22 Alpine, Vite build, port 3000
- **Backend konténer:** OpenJDK 17, Maven build, port 8080
- **Adatbázis konténer:** MySQL 8.0, port 3306, perzisztens volume

3.3. Technológiai stack

3.3.1. Frontend technológiák

Alap keretrendszer és nyelv

- **React 18.3.1:** Komponens alapú UI könyvtár
- **TypeScript 5.8.3:** Típusbiztos JavaScript
- **Vite 7.1.7:** Modern build eszköz SWC-vel

UI keretrendszer és stílusok

- **Material-UI (MUI) 7.3.4:** Átfogó komponens könyvtár
 - @mui/material: Alap komponensek (gombok, táblázatok, dialógusok)
 - @mui/icons-material: Ikon könyvtár
 - @mui/x-date-pickers: Dátum/idő választók
 - @mui/x-charts: Grafikonok (Dashboard használja)
- **Emotion 11.14:** CSS-in-JS stílusrendszer

Állapotkezelés és routing

- **Jotai 2.15.0:** Atomi állapotkezelés (könnyűsúlyú alternative a Reduxnak)
- **React Router 7.9.3:** Kliens oldali navigáció

API kommunikáció és adatkezelés

- **Axios 1.12.2:** HTTP kliens JWT interceptor-okkal
- **date-fns 4.1.0:** Dátum és idő kezelés
- **jwt-decode 4.0.0:** JWT token dekódolás

Táblázatok és vizualizáció

- **material-react-table 3.2.1:** Haladó táblázat komponens szűrésekkel és rendezésekkel
- **react-calendar-timeline 0.28.0:** Szobatükör naptár nézet

Többnyelvűség és kódminőség

- **i18next 25.5.2 + react-i18next 15.7.3:** Többnyelvű támogatás (magyar, angol)
- **Biome 2.2.5:** Kód formázó és linter (ESLint + Prettier egyben)

3.3.2. Backend technológiák

Keretrendszer és nyelv

- **Spring Boot 3.4.3:** Java alkalmazás keretrendszer
- **Java 17:** Programozási nyelv
- **Maven:** Dependency és build management

Spring modulok

- spring-boot-starter-web
- spring-boot-starter-data-jpa
- spring-boot-starter-validation
- spring-boot-starter-security

Biztonság

- **Spring Security 6.4.4:** Hitelesítés és jogosultságkezelés
- **JWT 0.11.5:** JWT token generálás és validáció

Adatbázis elérés

- **Spring Data JPA**
- **MySQL Connector/J**

Segédkönyvtárak

- **Lombok 1.18+:** Boilerplate kód csökkentés (@Getter, @Setter stb.)
- **ModelMapper 3.0.0:** DTO és entitás közötti mapping
- **OpenPDF 2.0.0:** PDF generálás számlákhoz

API dokumentáció

- **springdoc-openapi 2.2.0:** Swagger/OpenAPI UI a /swagger végponton

3.3.3. Adatbázis technológia

- **MySQL 8.0:** Relációs adatbázis
- **Adatbázis név:** hotelpms
- **Hibernate DDL mód:** update (automatikus séma frissítés)

3.3.4. Fejlesztői eszközök

- **Git:** Verziókezelés
- **Node.js:** Frontend fejlesztési környezet
- **Java JDK 17:** Backend fejlesztési környezet
- **Docker Desktop:** Konténer futtatás

3.3.5. Technológiai döntések indoklása

Miért React + TypeScript? A React komponens alapú megközelítése lehetővé teszi az újrafelhasználható UI elemek fejlesztését, miközben a TypeScript típusbiztonsága javítja a kódminőséget és a fejlesztői élményt. Mindkét technológia széles körben elterjedt, ezért gazdag dokumentáció és közösségi támogatás áll rendelkezésemre. Ezen indokok és korábbi tapasztalataim miatt választottam ezt a két technológiát

Miért Spring Boot? A Spring Boot beépített production-ready funkciókat (security, validation stb.) kínál, amelyek gyorsítják a fejlesztést. A JPA/Hibernate integráció egyszerűsíti az adatbázis műveleteket. Ezek mellett fontos szempont volt, hogy elterjedt és jól dokumentált legyen, így egyszerűen megtanulhassam használni.

Miért MySQL? A MySQL relációs adatbázisként ACID tulajdonságokkal rendelkezik, ami elengedhetetlen a komplex kapcsolatok (számlák, foglalások, vendégek) megbízható kezeléséhez. A széles körű elterjedtsége garantálja a stabilitást és a támogatottságot.

3.4. Adatbázis felépítése

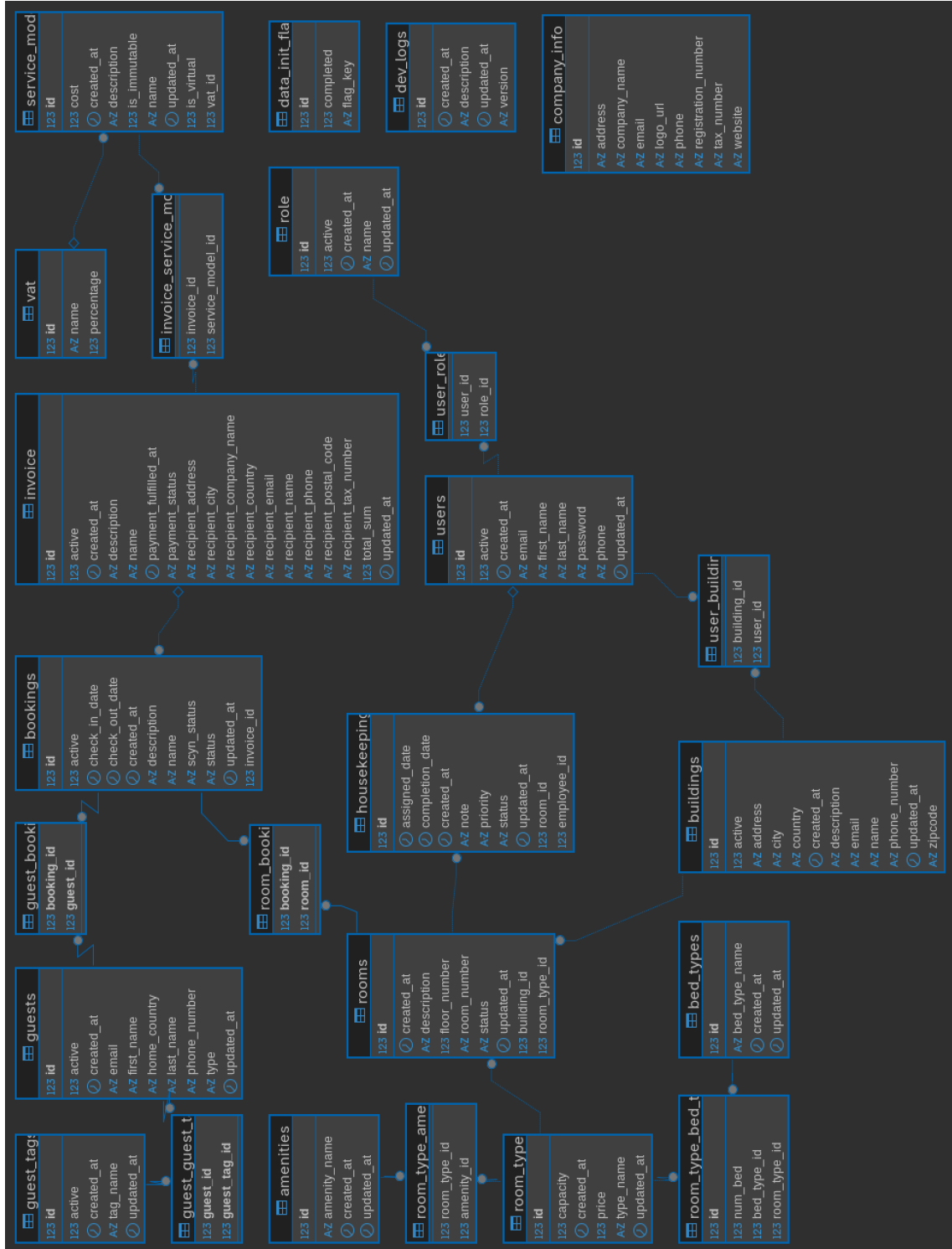
A rendszer relációs adatbázist használ a szálloda adatainak tárolására. Az adatbázis MySQL 8.0-t használ, és 18 fő entitásból áll.

3.4.1. Adatbázis konfiguráció

- **Adatbázis neve:** hotelpms
- **Karakter kódolás:** UTF-8

- **Port:** 3306 (alapértelmezett MySQL port)

3.4.2. Entitás-kapcsolat diagram (ERD)



3.2. ábra. Adatbázis Entitás-Kapcsolat Diagram

3.4.3. Entitások áttekintése

A rendszer 18 entitást tartalmaz, amelyeket a következő csoportokba sorlom:

- **Felhasználói és biztonsági entitások:** A rendszer felhasználóit a **User** entitás tárolja, amely N:M kapcsolatban áll a **Role** entitással, lehetővé téve a rugalmas jogosultságkezelést (recepciósök, adminok, stb.).
- **Vendég kezelés:** A vendégek adatait a **Guest** entitás tartalmazza, amely **GuestTag**-ekkel bővíthető különleges címkével (törzsvendég, VIP).
- **Szálloda struktúra:** Az épület hierarchiát a **Building** és **Room** entitások reprezentálják. A szobák **RoomType** (egyágyas, kétágyas, apartman), **BedType** (egyszemélyes, franciaágy) és **Amenity** (WiFi, klíma, minibar) tulajdonságokkal rendelkeznek.
- **Foglalás kezelés:** A **Booking** entitás kezeli a foglalásokat, amely N:M kapcsolatban áll a vendégekkel (**BookingGuest** kapcsolótáblán át) és a szobákkal (**BookingRoom** kapcsolótáblán át), így egy foglalás több vendéget és több szobát is tartalmazhat.
- **Számlázás:** A számlázási rendszer központi entitása az **Invoice**, amely N:M kapcsolatban áll a foglalásokkal (**InvoiceBooking** kapcsolótábla) és szolgáltatásokkal (**InvoiceService** kapcsolótábla). A **ServiceModel** definiálja az elérhető szolgáltatásokat (szoba ár, spa, minibar), míg a **Vat** entitás az ÁFA kulcsokat tárolja.
- **Takarítás:** A **Housekeeping** entitás tárolja a takarítási feladatokat.
- **Rendszer konfiguráció**
 - **CompanyInfo:** Cég adatok (egyetlen rekord)
 - **DataInitFlag:** Adat inicializálási kapcsoló: minta adatokkal fel kell-e tölteni az adatbázist?
 - **DevLog:** Változáspanló (verzió történet)

Megjegyzés: Néhány fontosabb tábla pontosabb leírását mellékeltem a 6.2 függelékben.

3.4.4. Kapcsolótáblák

A rendszer több N:M kapcsolatot kezel külön kapcsolótáblákon keresztül. Ezek közül kettőt emelek ki példaként: egy általánosot és az egyetlen speciális kapcsolótáblát.

guest_booking kapcsolótábla (általános kapcsolótábla példa) Összeköti a foglalásokat a vendégekkel. Egy foglaláshoz több vendég is tartozhat, és egy vendég több foglalással is rendelkezhet.

3.1. táblázat. guest_booking kapcsolótábla

Mező	Típus	Megszorítás	Leírás
booking_id	BIGINT	PK, FK(Booking.id)	Foglalás azonosító
guest_id	BIGINT	PK, FK(Guest.id)	Vendég azonosító

room_type_bed_type kapcsolótábla Ez a tábla speciális, mert nem csak összeköti a szobatípusokat az ágytípusokkal, hanem tárolja az adott ágytípusból hány darab található az adott szobatípusban. Pl. egy "Családi szoba" lehet 1 db franciaágy + 2 db egyszemélyes ágy.

3.2. táblázat. room_type_bed_type kapcsolótábla szerkezete

Mező	Típus	Megszorítás	Leírás
id	BIGINT	PK, AUTO_INC	Elsődleges kulcs
room_type_id	BIGINT	FK(RoomType.id), NOT NULL	Szobatípus azonosító
bed_type_id	BIGINT	FK(BedType.id), NOT NULL	Ágytípus azonosító
num_bed	INT	NOT NULL	Ágyak száma (mennyiség)

Kapcsolatok:

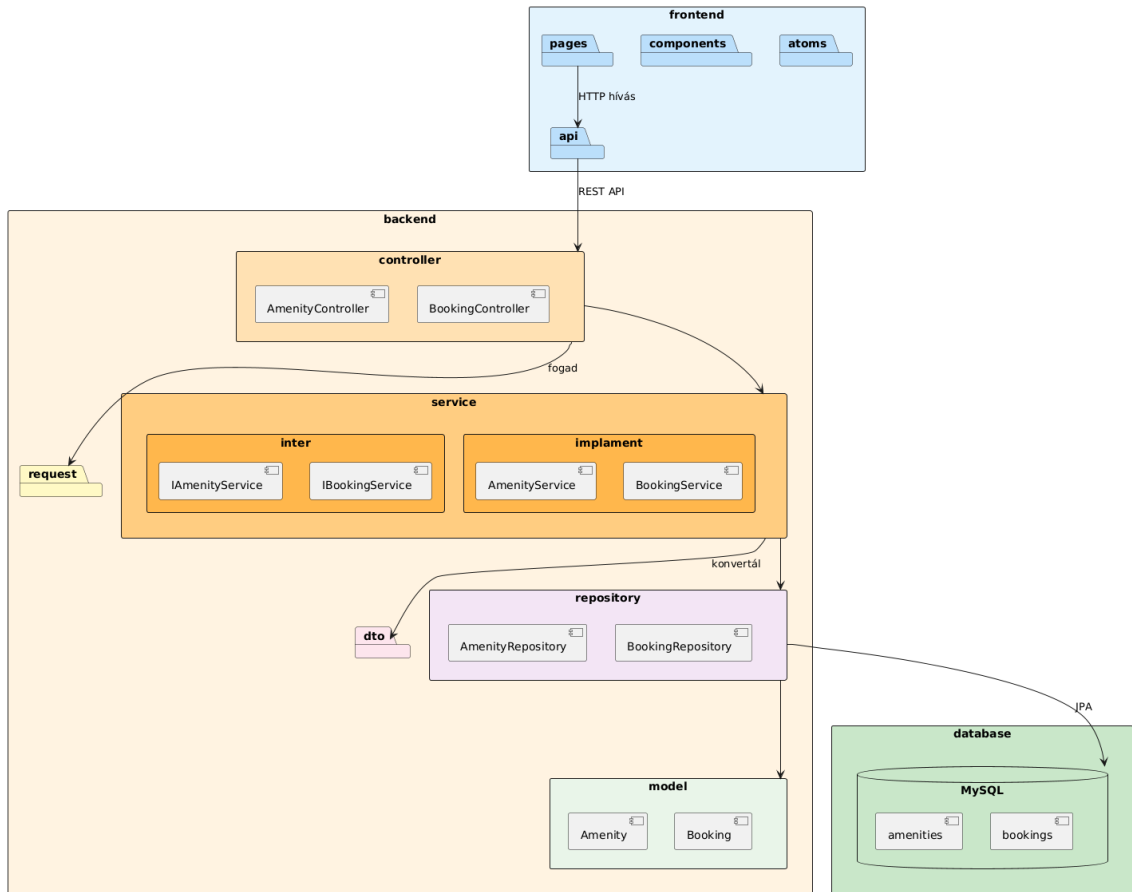
- N:1 kapcsolat RoomType táblával (room_type_id FK)
- N:1 kapcsolat BedType táblával (bed_type_id FK)

Megjegyzés: Ez a tábla saját elsődleges kulccsal rendelkezik (id), szemben az általános kapcsolótáblákkal. Ez a megoldást rugalmasabbnak találtam, mivel lehetővé teszi extra attribútumok (num_bed) tárolását a kapcsolaton.

3.5. Modul- és osztályszerkezet

3.5.1. Architektúra áttekintő diagram

A rendszer három fő rétegből épül fel, amelyek Docker konténerekben futnak:



3.3. ábra. Rendszer architektúra áttekintése

- **Frontend konténer:** React 18 + TypeScript, Vite build eszköz
- **Backend konténer:** Spring Boot 3.4, Java 17, REST API
- **Adatbázis konténer:** MySQL 8.0, perzisztens volume

3.5.2. Főbb osztályok/modulok

Backend osztályok

Controller réteg: A REST API végpontokat kezelő osztályok (`@RestController` annotációval). Strukturálisan a `controller/` mappában találhatóak. Itt fogadom a backend felé érkező kéréseket, majd továbbítom őket a service réteg felé, ha megfelelőek a bejövő adatok, végül visszaküldéskor konvertálom az üzenetet a megfelelő

DTO objektumba. **Megjegyzés:** Összesen 19 controller van. Pontosan felsoroltam őket a 6.3.1 függelékben.

Service réteg: Az üzleti logikát megvalósító osztályok interfészekkel. Strukturálisan a `service/``implement/` és a `service/inter/` mappákban találhatóak. Ez a réteg tartalmazza az alkalmazás központi üzleti szabályait és komplex műveleteit

Repository réteg: JPA Repository interfészek Spring Data JPA-val. Strukturálisan a `repository/` mappában találhatóak. Ez a réteg felelős az adatbázis műveletek végrehajtásáért, emellett Spring Data JPA automatikusan generálja az alapvető CRUD műveleteket.

Megjegyzés: Összesen 18 repository van.

Model réteg (Entitások): JPA entitások az adatbázis táblák reprezentálására (lásd 3.4 fejezet). Strukturálisan a `model/` mappában találhatóak. Ezek az osztályok reprezentálják az adatbázis sémát és tartalmazzák az entitások közötti kapcsolatokat (`@OneToMany`, `@ManyToMany`, stb.).

Megjegyzés: Összesen 18 entitás van.

DTO és Request/Response objektumok: Adatátviteli objektumok a frontend-backend kommunikációhoz. Strukturálisan a `dto/`, `request/` és `response/` mappákban találhatóak. Ezek az osztályok biztosítják, hogy csak a szükséges adatok kerüljenek át a rétegek között, és egységes válasz formátumot (`ApiResponse`) használnak.

Megjegyzés: Minden entitáshoz tartozik DTO, valamint a műveletek típusától függően (`create`, `update`) Request objektumok is.

Security réteg: JWT alapú autentikáció és Spring Security konfiguráció. Strukturálisan a `security/` mappában találhatóak. Ez a réteg felelős a felhasználók hitelesítéséért, jogosultságok ellenőrzéséért és , hogy mely végpontok védettek. Emelett tartalmazza a JWT token generálást, validációt és a Spring Security filtereket.

Frontend modulok

Pages Oldal szintű komponensek a `pages/` mappában. Minden terület (foglalások, számlák, stb.) saját oldallal rendelkezik.

API kliens modulok Axios-alapú API kommunikáció az `api/` mappában. Minden backend controller-hez tartozik egy megfelelő frontend API modul.

State management Globális állapotkezelés Jotai atomokkal a `state/` mappában.

BookingService főbb metódusai

createBooking

- **Bemenő adat:** CreateBookingRequest (checkInDate, checkOutDate, name, guestIds, roomIds)
- **Kimenő adat:** Booking entitás
- **Tevékenység:**
 - Szobák lekérdezése az adatbázisból
 - BookingConflictUtil segítségével ütközések ellenőrzése
 - Foglалás létrehozása, vendégek és szobák hozzárendelése
 - Mentés az adatbázisba

updateBookingStatus

- **Bemenő adat:** Long targetId, BookingStatusEnum newStatus
- **Kimenő adat:** Booking entitás
- **Tevékenység:**
 - cansetStatus() hívása - státusz átmenet validálása
 - CHECKED_OUT státusz esetén: szobák állapotának "DIRTY"-re állítása
 - Foglалás státuszának frissítése
 - Mentés az adatbázisba

Példa: Új foglalás létrehozása

Egy konkrét használati esetet bemutatok a rétegek közötti adatáramlásról:

1. **Frontend:** A BookingPage komponensen a felhasználó kitölti a foglalási űrlapot `MultiStepBookingModal.tsx`
2. **API hívás:** `bookingApi.createBooking(data)` meghívása (POST /api/bookings)
3. **Backend Controller:** `BookingController.createBooking(@RequestBody CreateBookingRequest)`
 - Validáció (@Valid annotáció)
 - Service réteg hívása
4. **Service réteg:** `BookingService.createBooking(request)`

- Üzleti logika: szobák elérhetőségének ellenőrzése
- Foglалás adatok beállítása
- Vendégek beállítása
- Repository hívás

5. **Repository:** `BookingRepository.save(booking)`

- JPA entitás mentése MySQL-be

6. **Válasz:** Entity $\rightarrow$ DTO konverzió, visszaküldés JSON-ként

7. **Frontend:** frissül a frontend oldal és visszajelez a felhasználó felé

3.5.4. UI-tervek és navigáció

Képernyőtervezés

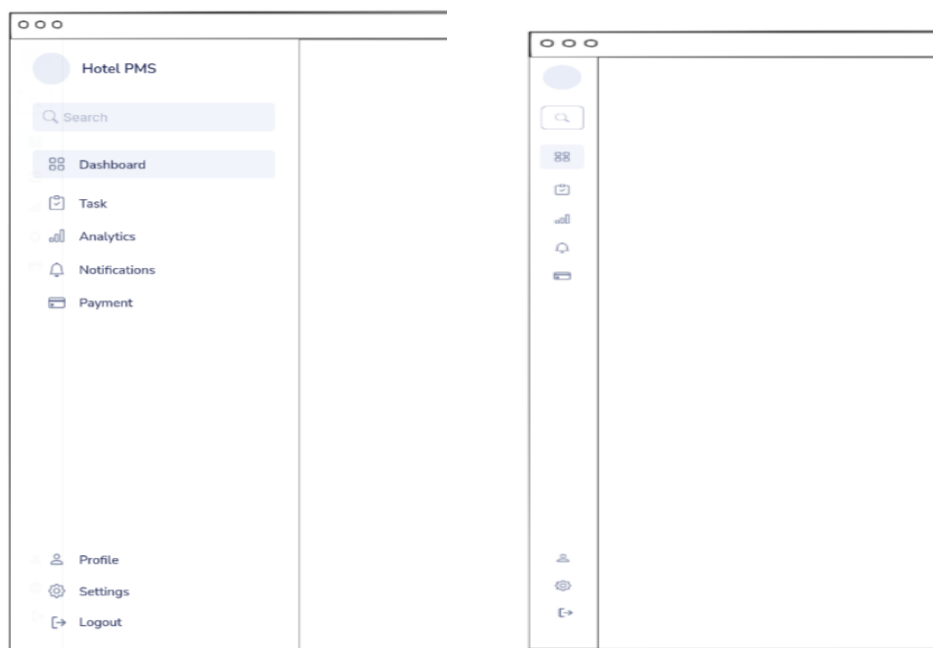
3.5.5. Felhasználói felület tervei

A rendszer webes felületet használ, Material-UI komponens könyvtárral megvalósítva. A tervezés során wireframe-eket készítettem de fontos megjegyezni, hogy ezek a kezdeti tervek voltak és a fejlesztés során tovább fejlődtek és finomításra kerültek, de jól szemléltetik a rendszer alapvető UI architektúráját.

Navigációs struktúra

A rendszer navigációja egy összezsukható oldalsó menüsorból (sidebar) áll, amely két állapotban jelenhet meg:

- **Nyitott állapot:** teljes szöveges menüpontokkal (lásd: 3.5. (a) ábra)
- **Csukott állapot:** csak ikonokkal a képernyőterület maximalizálása érdekében (lásd: 3.5. (b) ábra)

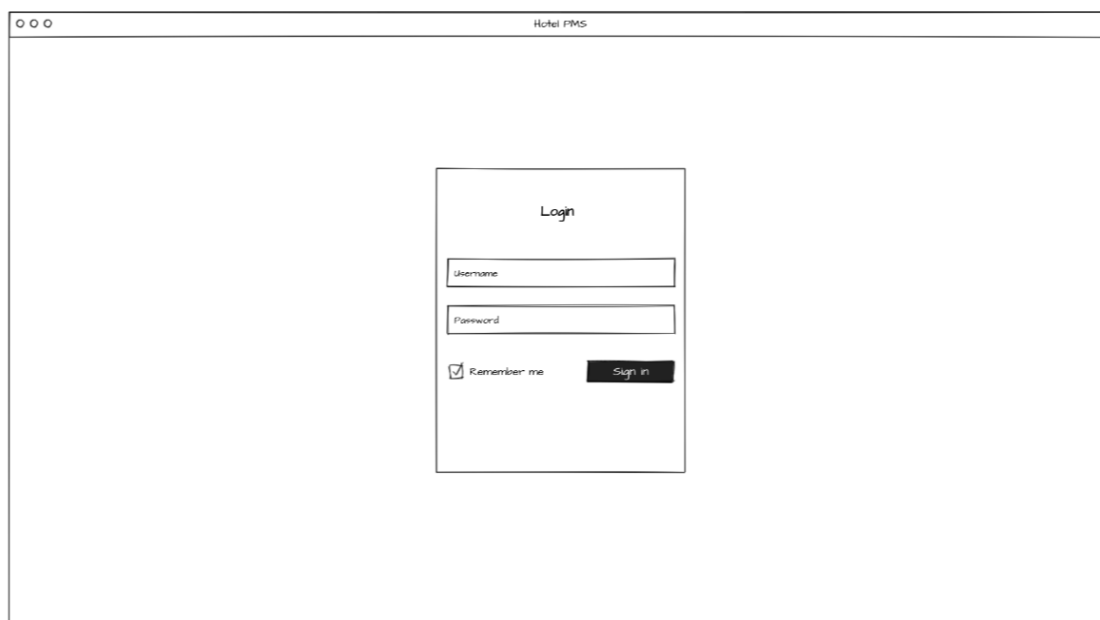


(a) Nyitott navigációs menü

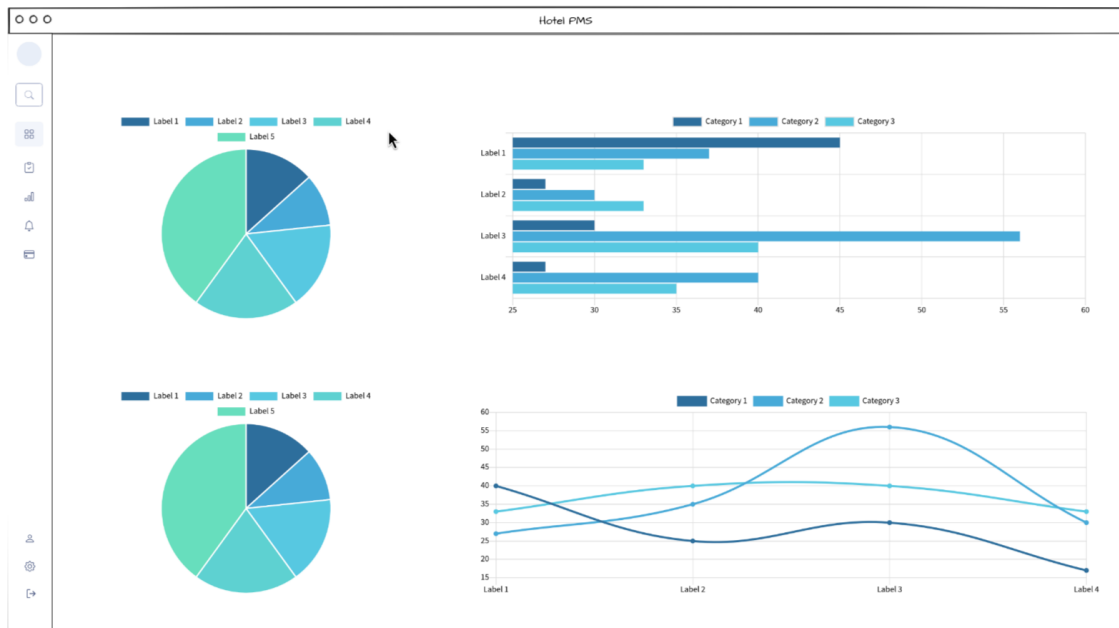
(b) Csukott navigációs menü

3.5. ábra. Navigációs menü

Kulcsfontosságú nézetek



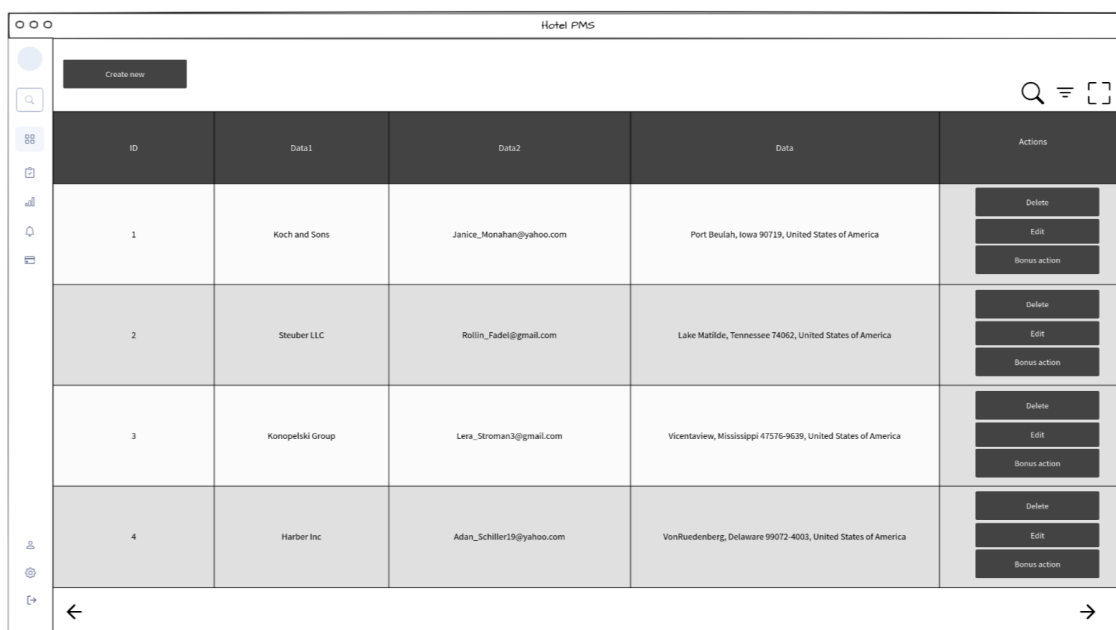
3.6. ábra. Bejelentkezési felület



3.7. ábra. Dashboard nézet statisztikákkal

Táblázatos listák Az adatok megjelenítése táblázatos formában történik (lásd: 3.8. ábra), amely biztosítja:

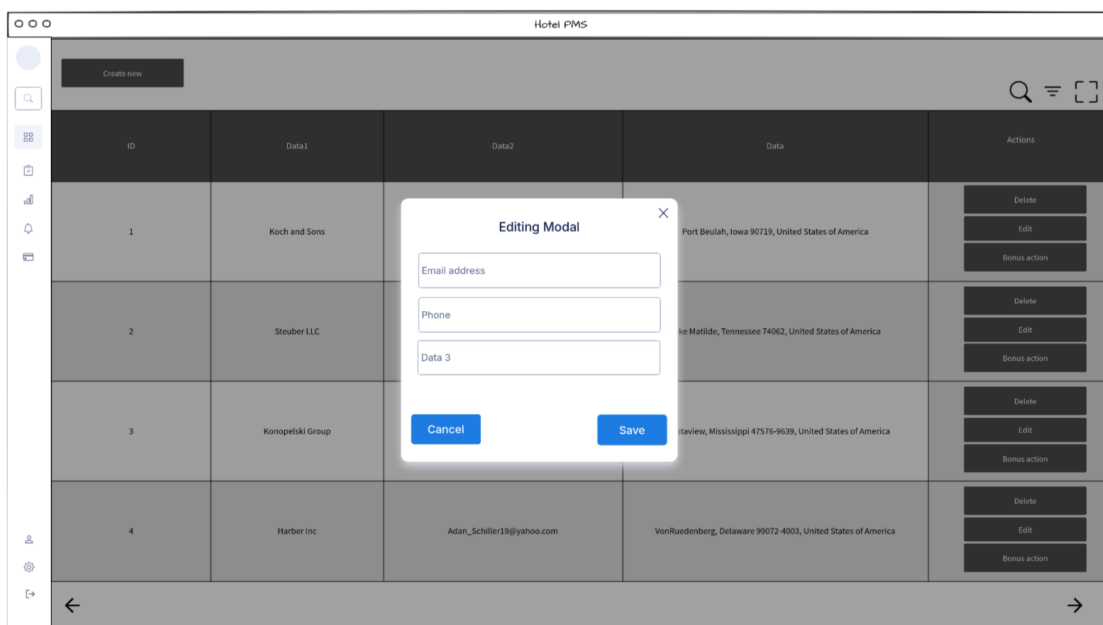
- Oszlopok szerinti rendezést
- Keresési és szűrési lehetőségeket
- Sor szintű műveleteket (Edit, Delete, Bonus action)
- Új rekord létrehozását ("Create new" gomb)
- Lapozást nagy adatmennyiség esetén
- Teljes képernyős nézetet



The screenshot shows the 'Hotel PMS' application interface. At the top, there is a 'Create new' button and a search icon. Below the header, there is a table with 5 columns: ID, Data1, Data2, Data, and Actions. The table contains 4 rows of data. Each row has three buttons in the 'Actions' column: Delete, Edit, and Bonus action. The table is surrounded by a sidebar with various icons and a bottom navigation bar with left and right arrows.

ID	Data1	Data2	Data	Actions
1	Koch and Sons	Janice_Monahan@yahoo.com	Port Beulah, Iowa 90719, United States of America	Delete Edit Bonus action
2	Steuber LLC	Rollin_Fadel@gmail.com	Lake Matilde, Tennessee 74062, United States of America	Delete Edit Bonus action
3	Konopelski Group	Lera_Stroman3@gmail.com	Vicentaview, Mississippi 47576-9639, United States of America	Delete Edit Bonus action
4	Harber Inc	Adan_Schiller19@yahoo.com	VonRuedenberg, Delaware 99072-4003, United States of America	Delete Edit Bonus action

3.8. ábra. Táblázatos lista nézet műveleti gombokkal



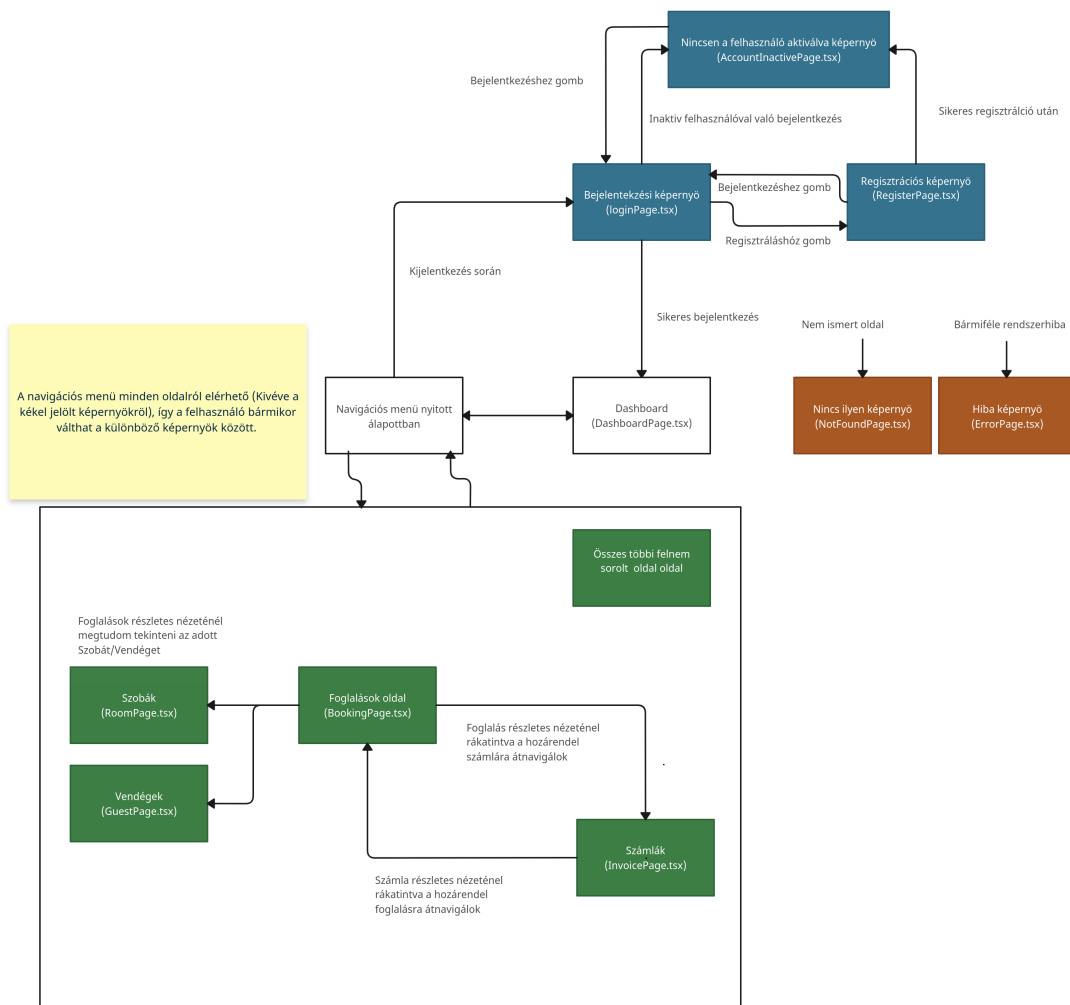
The screenshot shows the 'Hotel PMS' application interface with an 'Editing Modal' dialog box open. The modal has a title 'Editing Modal' and a close button (X). It contains three input fields: 'Email address', 'Phone', and 'Data 3'. At the bottom of the modal are 'Cancel' and 'Save' buttons. The background shows the same table as in the previous screenshot, but it is dimmed.

3.9. ábra. Modal ablak

Select building								New booking
Szobaszám	01.01	01.02	01.03	01.04	01.05	01.06	01.07	01.08
101	Booking 1				Booking 3			
102								
103								
104				Booking 2				
105	Booking 1							

3.10. ábra. Szobatükör nézet foglalások vizualizálásához

3.5.6. Navigációs diagram



3.11. ábra. Navigációs diagram

3.6. Kiemelndő kódrészletek

Itt bemutatom a rendszer implementációjának fontosabb részeit, amelyek a legnagyobb kihívást jelentették a fejlesztés során, vagy amelyek egyedi megoldásokat igényeltek.

A bemutatott kódrészletek nem tartalmazzák a teljes forráskódot hanem csak a lényegi logikát emelik ki, hogy a dokumentáció olvasható maradjon.

A fejezet két fő részre tagolódik:

- **Közös algoritmusok:** Olyan hasonló logikai egységek, amelyeket a rendszer több pontján is használtam.
- **Nem triviális megoldások:** Összetett implementációs megoldások, amelyek speciális tervezési döntéseket, külső könyvtárak integrációját vagy nem magától értetődő programozási technikákat igényeltek.

3.6.1. Közös algoritmusok

CRUD műveletek általános mintája

A rendszer backend moduljai következetes szerkezetet követnek a CRUD (Create, Read, Update, Delete) műveletek implementálásánál.

Általános felépítés: Minden CRUD modul az alábbi komponensekből áll:

- **Controller:** REST API végpontok (@RestController)
- **Service interfész + implementáció:** Üzleti logika
- **Repository:** JPA adatbázis elérés
- **Request/Response DTO-k:** Adatátviteli objektumok

Példa: AmenityController CRUD műveletek A AmenityController-el szemléltetem az általános CRUD eljárásokat

1. List (Összes elem lekérdezése):

```

1  @GetMapping
2  public ResponseEntity<ApiResponse> getAllAmenities(@ModelAttribute AmenityFilter filters) {
3      try {
4          List<Amenity> amenities = amenityService.getAmenities(filters);
5          List<AmenityDto> amenityDtos = amenities.stream()
6              .map(amenityService::convertAmenityToDto)
7              .toList();
8
9          return ResponseEntity
10             .ok(new ApiResponse(FrontEndCodes.SUCCESS.getCode(), amenityDtos));
11
12     } catch //error handling...
13     }
14     // Service reteg
15     @Transactional(readOnly = true)
16     @Override //Hiszen van egy interface
17     public List<Amenity> getAmenities(AmenityFilter filters) {
18         if (filters == null) {
19             return amenityRepository.findAll();
20         }
21
22         return amenityRepository.findAll().stream()
23             .filter(buildAmenityPredicate(filters))
24             .collect(Collectors.toList());
25     }

```

3.1. forráskód. GET /api/amenities - Lista lekérdezés

2. Create (Uj elem létrehozasa):

```

1  @PostMapping
2  public ResponseEntity<ApiResponse> createAmenity(@Valid @RequestBody CreateAmenityRequest
3      request) {
4      try {
5          Amenity createdAmenity = amenityService.createAmenity(request);
6          AmenityDto amenityDto = amenityService.convertAmenityToDto(createdAmenity);
7
8          return ResponseEntity.status(HttpStatus.CREATED)
9              .body(new ApiResponse(FrontEndCodes.SUCCESS.getCode(), amenityDto));
10
11     } catch //error handling...
12     }
13     // Service reteg
14     @Override
15     public Amenity createAmenity(CreateAmenityRequest request) {
16         Optional<Amenity> existingAmenityOpt = amenityRepository.findByAmenityName(request.
17             getAmenityName());
18
19         if (existingAmenityOpt.isPresent()) {
20             throw new AlreadyExistsException(FrontEndCodes.AMENITY_ALREADY_EXISTS.getCode());
21         }
22
23         Amenity amenity = new Amenity();
24         amenity.setAmenityName(request.getAmenityName());
25
26         return amenityRepository.save(amenity);
27     }

```

3.2. forráskód. POST /api/amenities - Uj elem létrehozasa

3. Update (Elem modositása):

```

1  @PutMapping("/{id}")
2  public ResponseEntity<ApiResponse> updateAmenity(@PathVariable Long id,
3  @Valid @RequestBody UpdateAmenityRequest request) {
4      try {
5          Amenity updatedAmenity = amenityService.updateAmenity(request, id);
6          AmenityDto amenityDto = amenityService.convertAmenityToDto(updatedAmenity);
7
8          return ResponseEntity.status(HttpStatus.CREATED)
9              .body(new ApiResponse(FrontEndCodes.SUCCESS.getCode(), amenityDto));
10
11     } catch //error handling...
12     }
13     // Service reteg
14     @Override
15     public Amenity updateAmenity(UpdateAmenityRequest request, long targetId) {
16         Optional<Amenity> existingAmenityOpt = amenityRepository.findById(targetId);
17         if (existingAmenityOpt.isEmpty()) {
18             throw new ResourceNotFoundException(FrontEndCodes.AMENITY_NOT_FOUND.getCode());
19         }
20         Amenity existingAmenity = existingAmenityOpt.get();
21         Optional<Amenity> existingName = amenityRepository.findByAmenityName(request.getAmenityName());
22         if (existingName.isPresent() && existingName.get().getId() != targetId)
23             throw new AlreadyExistsException(FrontEndCodes.AMENITY_ALREADY_EXISTS.getCode());
24         existingAmenity.setAmenityName(request.getAmenityName());
25         return amenityRepository.save(existingAmenity);
26     }

```

3.3. forráskód. PUT /api/amenities/{id} - Elem modositása

4. Delete (Elem torlese):

```

1  @DeleteMapping("/{id}")
2  public ResponseEntity<ApiResponse> deleteAmenity(@PathVariable Long id) {
3      try {
4          amenityService.deleteAmenity(id);
5          return ResponseEntity.ok(new ApiResponse(FrontEndCodes.SUCCESS.getCode(), null));
6
7      } catch //error handling...
8      }
9
10     // Service reteg
11     @Override
12     public void deleteAmenity(long targetId) {
13         amenityRepository.findById(targetId).ifPresentOrElse(
14             amenity -> {
15                 if (amenity.getRoomTypes().isEmpty()) {
16                     amenityRepository.deleteById(targetId);
17                 } else {
18                     throw new IllegalStateException(FrontEndCodes.AMENITY_HAS_ROOMS.getCode());
19                 }
20             },
21             () -> {
22                 throw new ResourceNotFoundException(FrontEndCodes.AMENITY_NOT_FOUND.getCode());
23             }
24         );
25     }

```

3.4. forráskód. DELETE /api/amenities/{id} - Elem torlese

Eltérések speciális modulokban: Vannak olyan modulok, amelyek **extra üzleti logikát** tartalmaznak a standard CRUD műveleteken túl pl.: `BookingService`: Ütközéskezelés, státusz validáció, szobák foglalása Ezek a modulok azonban **ugyanazt az alapstruktúrát használják**, csak extra metódusokkal bővítik ki.

Request és DTO

- **Request objektumok:** Bejövő adatok (pl. `CreateXxxRequest`)
- **DTO objektumok:** Kimenő adatok a frontend felé (`XxxDto`)

CreateAmenity példa

1. **Frontend:** POST `/api/amenities`, body: `{"name": "WiFi"}`
2. **Controller:** `CreateAmenityRequest` deszerializálása JSON-ból
3. **Validáció:** `@Valid` annotáció ellenőrzi a mezőket
4. **Service:** `createAmenity` üzleti logika
5. **Repository:** `Amenity` mentése adatbázisba
6. **Service:** `Amenity` → `AmenityDto` (`convertAmenityToDto`)
7. **Controller:** `AmenityDto` JSON-ná alakítása
8. **Frontend:** JSON válasz fogadása

Request Request objektumokban Bean Validation annotációk használtam, egyszerű szerver oldali ellenőrzés érdekében (`@NotNull`, `@Email`, `@Size` stb.)

```
1  @Data
2  public class CreateRoomRequest {
3      @NotNull
4      private RoomStatusEnum status;
5      private String description;
6      @NotBlank
7      private String roomNumber;
8      @NotNull
9      @Min(1)
10     private Integer floorNumber;
11     @NotNull
12     private Long roomTypeId;
13     @NotNull
14     private Long buildingId;
15 }
```

3.5. forráskód. Request validáció Bean Validation-al

DTO

```
1  @Override
2  @Transactional(readOnly = true)
3  public AmentyDto convertAmentyToDto(Amenty amenity) {
4      return modelMapper.map(amenity, AmentyDto.class);
5  }
```

3.6. forráskód. Egyszerű DTO konverzió

```
1  @Override
2  @Transactional(readOnly = true)
3  public GuestDto convertGuestToDto(Guest guest) {
4      GuestDto guestDto = modelMapper.map(guest, GuestDto.class);
5
6      List<GuestTagDto> activeGuestTagDtos = guest.getGuestTags().stream()
7          .filter(GuestTag::getActive)
8          .map(guestTag -> modelMapper.map(guestTag, GuestTagDto.class))
9          .collect(Collectors.toList());
10     guestDto.setGuestTags(activeGuestTagDtos);
11
12     return guestDto;
13 }
```

3.7. forráskód. Komplex DTO konverzió ha több entitást is vissza kell adni

Backend szűrés és lapozás - alternatív megközelítés

A rendszer fejlesztése során fontolóra vettem a backend oldali szűrés és lapozás implementálását. Végül azonban ezt a megoldást nem alkalmaztam, mert az adatmennyiség nem indokolta.

Miért NEM lett implementálva?

- **Adatmennyiség:** A szálloda mérete kisebb vagy közepes max, így a teljes adathalmaz betöltése nem vezetett teljesítményproblémához
- **Frontend képességek:** A Material React Table komponens és gyors kliens oldali szűrést és rendezést biztosít

Előkészített backend szűrési infrastruktúra: Ennek ellenére a rendszer **továbbfejleszthetőség** céljából tartalmazza a backend oldali szűrés kódját. A legtöbb Service osztályban megtalálható a szűrési logika, amely azonnal használható, ha a frontend implementálja a szűrési paraméterek küldését.

```
1  @Data
2  public class BuildingFilter {
3      private Long id;
4      private String name;
5      private String address;
6      private String city;
7      private String zipcode;
8      private String country;
9      private Boolean active;
10     private List<Long> userIds;
11 }
```

3.8. forráskód. Épületek szűrés kérés

```
1  @Transactional(readOnly = true)
2  @Override
3  public List<Building> getBuildings(BuildingFilter filters) {
4      if (filters == null) {
5          return buildingRepository.findAll();
6      }
7
8      return buildingRepository.findAll().stream()
9          .filter(buildBuildingPredicate(filters))
10         .collect(Collectors.toList());
11 }
12
13 @Override
14 public Predicate<Building> buildBuildingPredicate(BuildingFilter filters) {
15     Predicate<Building> predicate = building -> true;
16
17     if (filters.getId() != null) {
18         predicate = predicate.and(building -> building.getId().equals(filters.getId()));
19     }
20     if (filters.getName() != null && !filters.getName().isEmpty()) {
21         predicate = predicate.and(building ->
22             building.getName().toLowerCase().contains(filters.getName().toLowerCase()));
23     }
24     ...
25     if (filters.getUserIds() != null && !filters.getUserIds().isEmpty()) {
26         predicate = predicate.and(building -> building.getUsers().stream()
27             .anyMatch(user -> filters.getUserIds().contains(user.getId())));
28     }
29
30     return predicate;
31 }
```

3.9. forráskód. Szűrés függvény a BuildingService-ben

3.6.2. Nem triviális megoldások

Dinamikus űrlap léterhozás - FormFactory

A frontend fejlesztése során kihívást okozott a sok hasonló űrlap (modal ablakok) kezelése. Ahelyett, hogy minden entitáshoz (Amenity, Guest, Room, stb.) külön komponenst írtam volna, egy általános űrlap generátort hoztam létre, amely egy alap konfigurációból dinamikusan építi fel az űrlapokat.

Probléma: A rendszerben 15+ entitás van, mindegyiknek van Create és Update művelete, ami 30+ különböző űrlapot jelentene. Ez rengeteg kód duplikációhoz vezetne:

- Validációs logika ismétlődik
- Hibakezelés minden űrlapban azonos
- Stílus és elrendezés következetlensége
- Nehéz karbantarthatóság

Megoldás: FormFactory + FormConfig A FormFactory komponens egy konfigurációból generálja az űrlap mezőket automatikusan.

FormConfig típusdefiníció:

```
1  export interface FieldConfig {
2    label: string;           // Mezo cimke
3    type: FieldType;         // Mezo tipusa
4    defaultValue?: FieldValue; // Alapertelmezett ertek
5    validation?: ValidationRules; // Validacios szabalyok
6    options?: FieldOption[];  // Select/Autocomplete opciok
7    placeholder?: string;
8    helperText?: string;
9    disabled?: boolean;
10   multiple?: boolean;       // Tobbszelekcios field
11 }
12 export type FieldType =
13   | "text"
14   | "number"
15   | "autocomplete"
16   | "select"
17   | "checkbox"
18   | "datepicker"
19   | "bedTypeQuantity";
```

3.10. forráskód. FormConfig TypeScript típusok

FormFactory működése:

Először létrehozok egy `FormConfig` típusú űrlap konfigurációt, amely leírja az összes mezőt és tulajdonságait. A `FormFactory` végigmegy ezen a konfigurációs objektumon, minden `FieldConfig`-hoz generál egy megfelelő input mezőt, majd ezeket betölti egy egységes stílusú `FormModal` komponensbe.

```
1 export const FormFactory = ({...}: FormFactoryProps) => {
2   return ( {Object.keys(config).map((fieldName) => {
3     ...
4     return (
5       <FieldWrapper
6         key={fieldName}
7         fieldName={fieldName}
8         fieldConfig={fieldConfig}
9         initialValue={initialValue}
10        valuesRef={valuesRef}
11        error={errors[fieldName]}
12      />
13    );
14  })}
15 );
16 };
```

3.11. forráskód. FormFactory komponens

A `FieldWrapper` komponens a `type` alapján dönti el, hogy melyik konkrét beviteli mező komponenszt használja:

```
1 const FieldWrapper = ({ fieldName, fieldConfig, ... }) => {
2   const [value, setValue] = useState(initialValue);
3   const handleChange = (newValue) => {
4     setValue(newValue);
5     valuesRef.current[fieldName] = newValue;
6   };
7   switch (fieldConfig.type) {
8     case "text":
9     case "number": return <TextField {...fieldProps} />;
10    case "autocomplete": return <AutocompleteField {...fieldProps} />;
11    case "select": return <SelectField {...fieldProps} />;
12    case "checkbox": return <CheckboxField {...fieldProps} />;
13    case "datepicker": return <DatePickerField {...fieldProps} />;
14    case "bedTypeQuantity": return <BedTypeQuantityField {...fieldProps} />;
15    default: return null;
16  }
17 };
```

3.12. forráskód. FieldWrapper - dinamikus beviteli mező választás

Példa: Vendég címke létrehozása modal:

```
1  export const guestTagFormConfig: FormConfig = {
2    tagName: {
3      label: "forms.labels.tagName",
4      type: "text",
5      defaultValue: "",
6      validation: {
7        required: true,
8        minLength: 2,
9      },
10   },
11   active: {
12     label: "forms.labels.active",
13     type: "checkbox",
14     defaultValue: true,
15     validation: {
16       required: false,
17     },
18   },
19 };
20
21 const handleClick = () => {
22   setTitle(t("guestTag.uploadTitle"));
23   setConfig(guestTagFormConfig);
24   setInitialValues({});
25   setOnSubmit(() => handleSubmit as (values: FormValues) => Promise<void>);
26   setFormModalOpen(true);
27 };
```

3.13. forráskód. Vendég címke űrlap konfiguráció

FormModal komponens: A FormModal egy egységes modal ablak, amely a FormFactory-t használja és reaktivan kezel állapotokat Jotai atomokon keresztül

```
1  export const FormModal = <T extends FormValues = FormValues>() => {
2    const open = useAtomValue(formModalOpenAtom);
3    const title = useAtomValue(formModalTitleAtom);
4    const config = useAtomValue(formModalConfigAtom);
5    const initialValues = useAtomValue(formModalInitialValuesAtom);
6    const onSubmit = useAtomValue(formModalOnSubmitAtom);
7    const buttons = useAtomValue(formModalButtonsAtom);
8    const setOpen = useSetAtom(formModalOpenAtom);
9    const valuesRef = useRef<FormValues>(initialValues as FormValues);
10   const [errors, setErrors] = useState<FormErrors>({});
11
12   const handleSave = async () => {
13     if (!config || !onSubmit) return;
14     const validationErrors = validateForm(valuesRef.current, config);
```

```
15     setErrors(validationErrors);
16     if (!hasErrors(validationErrors)) {
17         await onSubmit(valuesRef.current as T);
18     }
19 };
20
21 useEffect(() => {
22     if (open) {
23         valuesRef.current = { ...initialValues } as FormValues;
24         setErrors({});
25     }
26 }, [open, initialValues]);
27
28 if (!config) return null;
29
30 return (
31 <Dialog open={open}>
32   <Typography sx={{ fontWeight: "bold", padding: 2 }} variant="h5">
33     {title}
34   </Typography>
35   <Box sx={{ padding: 2 }}>
36     <FormFactory
37       config={config}
38       initialValues={initialValues}
39       valuesRef={valuesRef}
40       errors={errors}
41     />
42   </Box>
43   <DialogActions>
44     <Button onClick={handleClose} label="Cancel" />
45     <ButtonUI label="Save" onClick={handleSave}/>
46   </DialogActions>
47 </Dialog>
48 );
49 };
```

3.14. forráskód. FormModal komponens (Egyszerűsített)

Automatikus validáció: A `validateForm` függvény a konfiguráció alapján automatikusan ellenőrzi a mezőket amikor mentésre kerülne az űrlap:

```
1 export const validateForm = (  
2   values: FormValues,  
3   config: { [fieldName: string]: FieldConfig },  
4 ): FormErrors => {  
5   const errors: FormErrors = {};  
6  
7   Object.keys(config).forEach((fieldName) => {  
8     const fieldConfig = config[fieldName];  
9     const value = values[fieldName];  
10    const error = validateField(value, fieldConfig.validation);  
11  
12    if (error) {  
13      errors[fieldName] = error;  
14    }  
15  });  
16  
17  return errors;  
18 };
```

3.15. forráskód. Validációs rendszer

Előnyök:

- **Típusbiztonság:** TypeScript típusok garantálják a helyes konfigurációt
- **Következetesség:** Minden űrlap azonos megjelenésű és működésű
- **Gyors fejlesztés:** Új entitás űrlap 10-15 sor JSON konfigurációval elkészül
- **Könnyű módosítás:** Egy helyen kell módosítani a validációs/styling logikát

Speciális mezőtípus: BedTypeQuantity Ez egy egyedi field típus a `bedTypeQuantity`, amely a szobatípus-ágytípus kapcsolatot kezeli mennyiséggel:

```
1   bedTypes: {  
2     label: "Bed Configuration",  
3     type: "bedTypeQuantity",  
4     defaultValue: [],  
5     validation: { required: true }  
6   }  
7   // [{ bedTypeId: 1, quantity: 1 }, { bedTypeId: 2, quantity: 2 }]
```

3.16. forráskód. BedTypeQuantity field konfiguráció

Ez a megoldás jól szemlélteti a `FormFactory` rugalmasságát: könnyen bővíthető új field típusokkal anélkül, hogy a központi logikát sérteném meg.

Dinamikus konfiguráció - backend adatokkal: A FormFactory másik előnye, hogy a konfigurációt futásidőben is módosíthatom backend adatok alapján. Például a `ServiceModel` űrlapnál a ÁFA (VAT) kulcsok listáját nem beleégetem a konfigurációba (Hiszen ezek változhatnak), hanem futásidőben lekérem a backendről és dinamikusan illesztem be.

```
1  export const serviceFormConfig: FormConfig = {
2    ...
3    vatId: {
4      label: "forms.labels.vat",
5      type: "select",
6      defaultValue: "",
7      validation: {
8        required: true,
9      },
10     options: [],
11   },
12 };
13 const [vats, setVats] = useState<{ value: number; label: string }[]>([]);
14 useEffect(() => {
15   const loadVats = async () => {
16     const vatsData = await getVats();
17     setVats(
18       vatsData.map((vat) => ({
19         value: vat.id,
20         label: `${vat.name} (${vat.percentage}%`,
21       })),
22     );
23   };
24   loadVats();
25 }, []);
26 const handleClick = () => {
27   setTitle(t("service.uploadTitle"));
28   const configWithVats = {
29     ...serviceFormConfig,
30     vatId: {
31       ...serviceFormConfig.vatId,
32       options: vats,
33     },
34   };
35   setConfig(configWithVats);
36   setInitialValues({ cost: 0 });
37   setOnSubmit(() => handleSubmit as (values: FormValues) => Promise<void>);
38   setFormModalOpen(true);
39 };
```

3.17. forráskód. Dinamikus form konfiguráció VAT opciókkal

Így az űrlap mindig az aktuális adatbázisban tárolt ÁFA kulcsokat jeleníti meg,

anélkül hogy a kódot módosítani kellene. Ugyanez a megoldás használható más entitásoknál is pl. a Room űrlapnál és RoomType űrlapnál

Virtuális szolgáltatások - szobák összesített költsége

A számlázási rendszer egyik speciális megoldása a **virtuális szolgáltatások** kezelése, amelyek csak a számlákon belül létező szolgáltatások, és a szolgáltatások képernyőn nem jelennek meg.

Probléma: Amikor egy számlához foglalást kapcsolunk, a szobák árát napok szerint kell kiszámítani és a számlához rendelni. Azonban nem szerencsés minden egyes foglalás szobáit külön-külön szolgáltatásként kezelni, mert:

- A szolgáltatások listája tele lenne ugyanolyan nevű "Szoba költség" tételekkel
- Nehéz lenne megkülönböztetni őket
- Zavaró lenne a frontend számára

Megoldás: Virtuális ServiceModel A rendszer egy **base ServiceModel**-t használ ("Rooms aggregated cost", id=1), amely sablonként szolgál. Ebből virtuális példányokat hoz létre minden számlához (foglalásonként):

```
1  @Override
2  public ServiceModel createVirtualServiceModel(
3      List<Room> rooms,
4      LocalDate checkInDate,
5      LocalDate checkOutDate
6  ) {
7      ServiceModel baseServiceModel = getOrCreateRoomsCostServiceModel();
8      double totalCost = 0.0;
9      for (Room room : rooms) {
10         long days = checkOutDate.toEpochDay() - checkInDate.toEpochDay();
11         totalCost += room.getRoomType().getPrice() * days;
12     }
13     ServiceModel virtualServiceModel = new ServiceModel();
14     virtualServiceModel.setName(baseServiceModel.getName());
15     virtualServiceModel.setDescription(baseServiceModel.getDescription());
16     virtualServiceModel.setCost(totalCost);
17     virtualServiceModel.setVirtual(true);
18     virtualServiceModel.setImmutable(false);
19     virtualServiceModel.setVat(baseServiceModel.getVat());
20     return serviceModelRepository.save(virtualServiceModel);
21 }
```

3.18. forráskód. Virtuális ServiceModel létrehozása

Base ServiceModel garantálása: A `getOrCreateRoomsCostServiceModel()` metódus biztosítja, hogy mindig létezzen a sablon `ServiceModel`:

```

1  @Override
2  public ServiceModel getOrCreateRoomsCostServiceModel() {
3      Optional<ServiceModel> existingServiceModel =
4      serviceModelRepository.findById(1L);
5      if (existingServiceModel.isPresent() &&
6      "Rooms aggregated cost".equals(existingServiceModel.get().getName())) {
7          ServiceModel serviceModel = existingServiceModel.get();
8          if (serviceModel.getImmutable() == null ||
9          !serviceModel.getImmutable()) {
10             serviceModel.setImmutable(true);
11             serviceModel = serviceModelRepository.save(serviceModel);
12         }
13         return serviceModel;
14     }
15     ServiceModel roomsCostServiceModel = new ServiceModel();
16     roomsCostServiceModel.setName("Rooms aggregated cost");
17     roomsCostServiceModel.setDescription(
18     "Base service model for room costs in invoices"
19     );
20     roomsCostServiceModel.setCost(0.0);
21     roomsCostServiceModel.setVirtual(false);
22     roomsCostServiceModel.setImmutable(true);
23     Vat defaultVat = vatRepository.findAll().stream().findFirst()
24     .orElseThrow(() -> new IllegalStateException(
25     "No VAT records found in database"
26     ));
27     roomsCostServiceModel.setVat(defaultVat);
28
29     return serviceModelRepository.save(roomsCostServiceModel);
30 }

```

3.19. forráskód. Base ServiceModel létrehozása/lekérése

Virtuális szolgáltatások szűrése: A `getServiceModels()` metódus automatikusan kiszűri a virtuális elemeket, így a frontend szolgáltatások képernyőn nem jelennek meg:

```

1  @Transactional(readOnly = true)
2  @Override
3  public List<ServiceModel> getServiceModels(ServiceModelFilter filters) {
4      if (filters == null) {
5          return serviceModelRepository.findAll().stream()
6          .filter(serviceModel -> !serviceModel.getVirtual())
7          .collect(Collectors.toList());
8      }
9
10     return serviceModelRepository.findAll().stream()

```

```
11     .filter(serviceModel -> !serviceModel.getVirtual())
12     .filter(buildServiceModelPredicate(filters))
13     .collect(Collectors.toList());
14 }
```

3.20. forráskód. Virtuális szolgáltatások szűrése

Miért fontos ez a megoldás?

- **Frontend számára:** A ServiceModel lista tiszta, csak a valódi szolgáltatásokat tartalmazza (SPA, minibar, parkolás stb.)
- **Számlán belül:** A szobák költsége egy összesített tételként jelenik meg ("Rooms aggregated cost: 45000 HUF")
- **Immutable flag:** A base ServiceModel nem törölhető.

3.7. Telepítési útmutató

A rendszer Docker konténerekben fut, így egyszerű telepíteni és konzisztens futási környezetet különböző platformokon is. A telepítéshez csak a Docker és Docker Compose szükséges.

3.7.1. Rendszerkövetelmények

Szoftver követelmények:

- **Docker Engine:** 20.10 vagy újabb
- **Docker Compose:** 2.0 vagy újabb
- **Minimum 4 GB RAM** a konténerek futtatásához
- **Minimum 5 GB szabad lemezterület**

Támogatott operációs rendszerek:

- Linux (Ubuntu 20.04+ volt csak tesztelve)
- macOS 11+ (Intel és Apple Silicon)
- Windows 10/11 (WSL2-vel)

3.7.2. Docker telepítése

- Linux (Ubuntu): docs.docker.com/engine/install/ubuntu/
- MacOS / Windows: www.docker.com/products/docker-desktop

3.7.3. Projekt letöltése és indítása

1. Repository klónozása:

```
1 git clone https://github.com/CsBenedek32/HOTEL-PMS
```

3.21. forráskód. Projekt letöltése

2. Konténerek építése és indítása:

```
1 docker compose up --build
```

3.22. forráskód. Docker Compose indítása

Ez a parancs:

- Felépíti a frontend és backend Docker image-eket
- Letölti a MySQL 8.0 image-et
- Elindítja mind a három konténert
- Létrehozza a hotelpms adatbázist

3. Háttérben futtatás (opcionális): Ha a konténereket háttérben szeretném futtatni:

```
1 docker compose up -d
```

3.23. forráskód. Docker Compose háttérben

Elérhetőség ellenőrzése

A konténerek sikeres indítása után a következő portokat elkéne tudnom érni:

Szolgáltatás	URL	Port
Frontend	http://localhost:3000	3000
Backend API	http://localhost:8080/api	8080
Swagger UI	http://localhost:8080/swagger	8080
MySQL adatbázis	localhost:3306	3306

3.3. táblázat. Szolgáltatások elérhetősége

Docker fájlok felépítése megtalálható a 6.4 függelékben

Fejlesztői mód

Production telepítéshez a fenti Docker Compose elegendő. Fejlesztési környezetben azonban érdemes a frontend-et és backend-et lokálisan futtatni hot-reloadal

3.7.4. Gyakori hibák és megoldásuk

Port már foglalt (Address already in use): Ha a 3000, 8080 vagy 3306 port már használatban van:

- Állítsd le a konfliktusban lévő alkalmazást
- Vagy módosítsd a portokat a `docker-compose.yml`-ben

MySQL healthcheck timeout: Ha a MySQL konténer nem indul el időben, növeld a healthcheck értékeket:

Backend nem kapcsolódik az adatbázishoz: Ellenőrizd a backend logokat

3.8. Tesztek

Többszintű tesztelési stratégiát alkalmaztam, amely lefedi az üzleti logika kritikus részeit. A tesztelés során hangsúlyt fektettem a rendszer központi funkcióinak ellenőrzésére, különös tekintettel a szobafoglalások konfliktuskezelésére és a számlázási folyamatokra.

3.8.1. Tesztelési módszertan

A teszteket három fő kategóriába tudom sorolni:

Unit tesztek

Az unit tesztek célja az egyes komponensek izolált tesztelése külső függőségek nélkül. Ezek a tesztek tisztán üzleti logikai számításokat és validációkat ellenőriznek, gyors futási idővel és magas kódlefedettséggel. A három fő utility osztályt teszteltem unit tesztekkel:

- **BookingConflictUtil:** A szobafoglalások időbeli konfliktusának detektálása
- **InvoiceCalculationUtil:** Számlák végösszegének kiszámítása ÁFÁ-val

- **RoomPriceCalculationUtil**: Szobák árának kalkulációja foglalási időszak alapján

com.hpms.backend.util

Element	Missed Instructions	Cov.	Missed Branches	Cov.
BookingConflictUtil		97%		83%
RoomPriceCalculationUtil		94%		100%
InvoiceCalculationUtil		93%		100%
Total	9 of 199	95%	3 of 40	92%

3.12. ábra. Kódlefedettség: Unit tesztek

Mock-alapú service tesztek

A service réteg tesztelése során a Mockito keretrendszert használtam az adatbázis és egyéb függőségek mockolására. Így tudtam tesztelni egyszerűen az üzleti logika komplex folyamatait anélkül, hogy valódi adatbázis-műveletek történnének.

Tesztelt service osztályok:

- **BookingService**: Foglalások életciklusa, státuszváltások, konfliktuskezelés
- **InvoiceService**: Számlák és foglalások szinkronizációja
- **ServiceModelService**: Service modellek kezelése

com.hpms.backend.service.implament

Element	Missed Instructions	Cov.	Missed Branches	Cov.
InvoiceService		39%		19%
BookingService		54%		37%
ServiceModelService		59%		38%

3.13. ábra. Kódlefedettség: Mock-alapú service tesztek

Megjegyzés: Csak a kritikusan fontos műveletek lettek itt tesztelve

Manuális tesztek

Az automatizált tesztek mellett manuális teszteket is végeztem, amelyek a felhasználói felület és az end-to-end folyamatok helyes működését vizsgálják.

A manuális tesztelés célja:

- Felhasználói élmény (UX) ellenőrzése
- Böngészőkompatibilitás tesztelése

- Komplex üzleti folyamatok end-to-end tesztelése
- Vizuális megjelenés és esztétika ellenőrzése
- Olyan edge case-ek felfedezése, amelyeket automatizált tesztek nem fednek le

Tesztelési eszközök

A teszteléshez az alábbi eszközöket használtam:

- **JUnit 5:** A tesztek futtatási keretrendszere
- **Mockito:** Mock objektumok létrehozása és viselkedésük definiálása
- **JaCoCo:** Kódlfedettség mérése
- **Google Chrome**
- **Mozilla Firefox**

3.8.2. Tesztesetek

Az alábbiakban a Unit- és Mock-alapú tesztesetek táblázatai kerülnek megjelenítésre, míg a manuális tesztesetek részletes ismertetése a 4. fejezetben található.

Kategória	Tesztek száma
Unit tesztek	
BookingConflictUtil	9
InvoiceCalculationUtil	7
RoomPriceCalculationUtil	11
Unit tesztek összesen	27
Mock-alapú service tesztek	
BookingService	9
InvoiceService	8
ServiceModelService	10
Mock-alapú tesztek összesen	27
Összes automatizált teszt	54
Automatizált tesztek sikerességi rátája	100%

3.4. táblázat. Tesztelési eredmények összefoglalása

Teszt neve	Leírás
BookingConflictUtil	
Overlapping dates	Átfedő dátumok esetén konfliktus detektálása
No overlap	Nem átfedő foglalások ne okozzanak konfliktust
Back to back	Egymás után következő foglalások (checkout = checkin) kezelése
Cancelled booking	CANCELLED státuszú foglalás ne blokkoljon
Checked out booking	CHECKED_OUT státuszú foglalás ne okozzon konfliktust
Exclude booking ID	Saját foglalás módosításakor ne jelezzen önmagával konfliktust
Empty room list	Üres szoba lista esetén mindig szabad
All rooms free	Minden szoba szabad esetén true visszaadása
One room has conflict	Egy szoba foglaltsága esetén false visszaadása
InvoiceCalculationUtil	
Null list	Null lista esetén 0.0 visszaadása
Empty list	Üres lista esetén 0.0 visszaadása
Single service without VAT	ÁFA nélküli service model: csak cost
Single service with VAT	100 Ft + 27% ÁFA = 127 Ft kalkuláció
Multiple services different VAT	Több service különböző ÁFA kulcsokkal: helyes összegzés
Service with null cost	Null cost kezelése (0.0-ként)
VAT with null percentage	Null VAT percentage kezelése (0%-ként)
RoomPriceCalculationUtil	
Null room list	Null szoba lista esetén 0.0
Empty room list	Üres szoba lista esetén 0.0
Null check-in date	Null checkIn esetén 0.0
Null check-out date	Null checkOut esetén 0.0
Checkout before checkin	Érvénytelen dátum (checkout < checkin) esetén 0.0
Same day check-in/out	Azonos napi checkin/checkout esetén 0.0
Single room	1 szoba $\times$ 100 Ft $\times$ 5 nap = 500 Ft
Multiple rooms same price	Több szoba azonos áron: helyes összegzés
Multiple rooms different price	Több szoba különböző árakon: $(100+150+200) \times 3 = 1350$ Ft
Room with null room type	Null roomType esetén szoba kihagyása
One day stay	Egynapi tartózkodás: 1 szoba $\times$ 100 Ft $\times$ 1 = 100 Ft

3.5. táblázat. Unit tesztesetek

Teszt neve	Leírás
Create with available rooms	Foglalás létrehozása elérhető szobákkal, RESERVED státusz beállítása, 2 szoba és 1 vendég hozzárendelése
Create without rooms	Szobák nélküli foglalás létrehozása, WAITLISTED státusz beállítása
Create with conflicting rooms	Már foglalt szoba újrafoglalási kísérlete, IllegalStateException kivétel dobása, foglalás nem mentődik
Update status: Reserved to checked-in	Booking státusz váltás RESERVED-ről CHECKED_IN-re
Update status: Checked-in to checked-out	CHECKED_IN-ről CHECKED_OUT-ra váltás, szobák DIRTY státuszba állítása
Update status: Cancelled to reserved	CANCELLED-ről RESERVED-re váltás, szobák elérhetőségének újraellenőrzése
Update booking: Change dates	Foglalás dátumainak (checkIn, checkOut) és alapadatainak (név, leírás) módosítása
Delete booking	Foglalás soft delete: active flag false-ra állítása
Update: Change to waitlisted	Státusz váltás WAITLISTED-re, szobák törlése a foglalásból

3.6. táblázat. BookingService tesztesetek

Teszt neve	Leírás
Create with initial booking	Számla létrehozása initial booking-gal, automatikus syncWithBookings() hívás
Create without booking	Üres számla létrehozása initial booking nélkül
Sync with bookings	Booking-hoz tartozó szobákból virtuális service model generálása, scynStatus SYNCED-re állítása
Add invoice bookings	Booking hozzáadása meglévő számlához, invoice referencia beállítása, szinkronizálás
Add already taken booking	Már számlázott booking (scynStatus != NO_INVOICE) hozzáadási kísérlete IllegalArgumentException-t dob
Remove invoice bookings	Booking eltávolítása számlából, invoice referencia nullázása, scynStatus NO_INVOICE-ra állítása
Update invoice services	Service model lista frissítése, totalSum automatikus újraszámolása
Delete invoice	Invoice soft delete (active=false), kapcsolt bookingok invoice és scynStatus nullázása

3.7. táblázat. InvoiceService tesztesetek

Teszt neve	Leírás
Create service model	Új service model létrehozása megadott névvel, leírással, költséggel és VAT-tal
Create virtual service model	Virtuális service model generálás: 2 szoba (100 Ft, 150 Ft) × 5 nap = 1250 Ft
Get or create: First call	Singleton első hívás: "Rooms aggregated cost" létrehozása, immutable=true, cost=0
Get or create: Already exists	Singleton második hívás: meglévő objektum visszaadása, nem hoz létre újat
Get or create: Set immutable	Meglévő objektum immutable flag-jének utólagos beállítása ha false vagy null
Update regular service	Normál service model name, description, cost és VAT frissítése
Update immutable service	Immutable service esetén csak VAT frissíthető, name/description/cost változatlan
Update virtual service	Virtual service model cost és VAT frissítése
Delete service model	Service model törlése (deleteById), ha nincs kapcsolt invoice
Delete immutable service	Immutable service törlési kísérlete IllegalStateException-t dob

3.8. táblázat. ServiceModelService tesztesetek

4. fejezet

Teszt jegyzőkönyv

4.0.1. Bejelentkezési képernyő

Funkció	Lépések	Elvárt eredmény
Sikeres bejelentkezés helyes adatokkal	1. Beírom az admin@hotel.com címet az email mezőbe 2. Beírom az admin123 jelszót a jelszó mezőbe 3. Rányomok a bejelentkezés gombra	Sikeresen bejelentkezek a rendszerbe, a vezérlőpult fogad
Sikertelen bejelentkezés rossz jelszóval	1. Beírom az admin@hotel.com címet az email mezőbe 2. Beírom a rossz123 jelszót 3. Rányomok a bejelentkezés gombra	Hibaüzenet: "Érvénytelen bejelentkezési adatok"
Üresen hagyott mezőkkel való bejelentkezés	1. Üresen hagyom az email mezőt 2. Üresen hagyom a jelszó mezőt 3. Megpróbálok rányomni a bejelentkezés gombra	A gomb inaktív marad

Folytatás a következő oldalon

táblázat 4.1 – *Folytatás az előző oldalról*

Funkció	Lépések	Elvárt eredmény
Sikertelen bejelentkezés inaktív fiók miatt	1. Beírom a <code>teszt@hotel.com</code> címet az email mezőbe 2. Beírom a helyes jelszót 3. Rányomok a bejelentkezés gombra	"A fiók nem aktív" oldal fogad

4.1. táblázat. Bejelentkezési képernyő manuális tesztjei

4.0.2. Regisztrációs képernyő

Funkció	Lépések	Elvárt eredmény
Sikeres regisztráció helyes adatokkal	1. Beírom a keresztnévet: Teszt 2. Beírom a vezetéknévet: Béla 3. Beírom az emailt: <code>teszt.bela@hotel.com</code> 4. Beírom a telefont: +36301234567 5. Beírom a jelszót: Jelszo123 6. Beírom a jelszó megerősítést: Jelszo123 7. Rányomok a regisztráció gombra	Sikeresen regisztrálok a rendszerbe, "A fiók még nem aktív" oldal fogad
Sikertelen regisztráció eltérő jelszavakkal	1. Kitöltöm az összes mezőt helyesen 2. Jelszó: Jelszo123 3. Jelszó megerősítés: Jelszo456 4. Rányomok a regisztráció gombra	Hibaüzenet: "A jelszavak nem egyeznek"

Folytatás a következő oldalon

táblázat 4.2 – *Folytatás az előző oldalról*

Funkció	Lépések	Elvárt eredmény
Sikertelen regisztráció már létező emaillel	1. Kitöltöm az összes mezőt 2. Email: admin@hotel.com (létező) 3. Rányomok a regisztráció gombra	Hibaüzenet: "Az erőforrás már létezik"
Üresen hagyott kötelező mezők	1. Néhány mezőt üresen hagyok 2. Megpróbálok rányomni a regisztráció gombra	Validációs üzenetek jelennek meg

4.2. táblázat. Regisztrációs képernyő manuális tesztjei

4.0.3. Foglalások oldal

Funkció	Lépések	Elvárt eredmény
Foglalások listázása	1. Navigálás a Foglalások menüpontra 2. Az oldal betöltődik	Megjelenik a foglalások listája táblázatos formában

Folytatás a következő oldalon

táblázat 4.3 – Folytatás az előző oldalról

Funkció	Lépések	Elvárt eredmény
Új foglalás teljes adatokkal (szobával és vendéggel)	1. lépés: Alapadatok: 1. Rányomok az "Új foglalás" gombra 2. Beírom a nevet: Teszt Foglalás 3. Dátumok: 2025-11-15 - 2025-11-20 4. Leírás: Teszt leírás (opcionális) 5. "Következő" gomb 2. lépés: Vendégek: 6. Kiválasztok egy létező vendéget: Teszt Béla 7. "Következő" gomb 3. lépés: Szobák: 8. Kiválasztom a szobát: 101 9. "Mentés" gomb	Sikeres mentés üzenet, foglalás megjelenik "Foglalt" státusszal és "Nincs számla" számla státusszal
Új foglalás vendég nélkül	1. lépés: Alapadatok: 1. Rányomok az "Új foglalás" gombra 2. Kitöltöm a kötelező mezőket 3. "Következő" gomb 2. lépés: Vendégek: 4. Nem adok hozzá vendéget 5. "Következő" gomb 3. lépés: Szobák: 6. Kiválasztom a szobát: 102 7. "Mentés" gomb	Sikeres mentés, foglalás létrejön vendég nélkül, "Foglalt" státusz, "Nincs számla" számla státusz

Folytatás a következő oldalon

táblázat 4.3 – *Folytatás az előző oldalról*

Funkció	Lépések	Elvárt eredmény
Új foglalás szoba nélkül	1. lépés: Alapadatok: 1. Rányomok az "Új foglalás" gombra 2. Kitöltöm a kötelező mezőket 3. "Következő" gomb 2. lépés: Vendégek: 4. Hozzáadok vendéget 5. "Következő" gomb 3. lépés: Szobák: 6. Nem választok szobát 7. "Mentés" gomb	Sikeres mentés, foglalás létrejön szoba nélkül, "Várólistás" státusz, "Nincs számla" számla státusz
Új foglalás sem vendég, sem szoba nélkül	1. lépés: Alapadatok: 1. Rányomok az "Új foglalás" gombra 2. Kitöltöm a kötelező mezőket 3. "Következő" gomb 2 és 3. lépés: 4. Átlépkedek vendég és szoba hozzáadása nélkül 5. "Mentés" gomb	Sikeres mentés, foglalás létrejön, "Várólistás" státusz, "Nincs számla" számla státusz

Folytatás a következő oldalon

táblázat 4.3 – *Folytatás az előző oldalról*

Funkció	Lépések	Elvárt eredmény
Új foglalás új vendég hozzáadásával	1. lépés: Alapadatok: 1. Kitöltöm az alapadatokat 2. "Következő" gomb 2. lépés: Vendégek: 3. Elkezdem gépelni egy rendszerben nem létező vendég nevét 4. Kiválasztom az "Új vendég hozzáadása" opciót a dropdown-ból 5. Kitöltöm: Név, Email, Telefon 6. Elmentem az új vendéget 7. Az új vendég megjelenik a listában, automatikusan kiválasztva 8. "Következő" gomb 3. lépés: Szobák: 9. Kiválasztok szobát 10. "Mentés" gomb	Sikeres mentés, új vendég létrejön és hozzárendelődik a foglaláshoz
Alapadatok lépés hibaellenőrzés: üres név	1. lépés: Alapadatok: 1. Rányomok az "Új foglalás" gombra 2. Név mezőt üresen hagyom 3. Dátumok: 2025-11-15 - 2025-11-20 4. Megpróbálok rányomni a "Következő" gombra	Validálciós üzenet: "A név mező kitöltése kötelező", nem lehet továbblépni

Folytatás a következő oldalon

táblázat 4.3 – Folytatás az előző oldalról

Funkció	Lépések	Elvárt eredmény
Alapadatok lépés hibaellenőrzés: üres dátumok	1. lépés: Alapadatok: 1. Rányomok az "Új foglalás" gombra 2. Név: Teszt 3. Dátumokat üresen hagyom 4. Megpróbálok rányomni a "Következő" gombra	Validálciós üzenet: "A dátumok megadása köte- lező", nem lehet tovább- lépni
Alapadatok lépés hibaellenőrzés: ér- vénytelen dátumok	1. lépés: Alapadatok: 1. Rányomok az "Új foglalás" gombra 2. Név: Teszt 3. Érkezés: 2025-11-20 4. Távozás: 2025-11-15 (koráb- bi dátum) 5. Megpróbálok rányomni a "Következő" gombra	Validálciós üzenet: "A távozás dátuma nem le- het korábbi, mint az ér- kezés", nem lehet tovább- lépni
Szobák lépés: csak elérhető szobák lát- szanak	1 és 2. lépés: 1. Alapadatok: 2025-11-15 - 2025-11-20 2. Vendég hozzáadása, "Következő" 3. lépés: Szobák: 3. Megjelenik a szobák listája	Csak azok a szobák je- lennek meg, amelyek a megadott dátumok kö- zött szabadok
Foglalás informáci- ók megtekintése	1. Meglévő foglalás sorában rányomok a "..." gombra 2. Kiválasztom az "Információ" opciót	Megjelenik a foglalás részletes információi

Folytatás a következő oldalon

táblázat 4.3 – *Folytatás az előző oldalról*

Funkció	Lépések	Elvárt eredmény
Foglalás szerkesztése: alapadatok módosítása	1. Foglalás sorában "..." gomb 2. "Szerkesztés" opció kiválasztása 1. lépés: Alapadatok: 3. Név módosítása: Módosított Foglalás 4. Dátumok módosítása: 2025-12-01 - 2025-12-05 5. "Következő" gomb 2 és 3. lépés: 6. Végigmegyek a lépéseken 7. "Mentés" gomb	Sikeres módosítás, foglalás frissül az új adatokkal. A számla státusz átállítódik "Nincs szinkronizálva"-ra
Foglalás szerkesztése: szoba nélkül mentés	1. Foglalás sorában "..." gomb 2. "Szerkesztés" opció 1 és 2. lépés: 3. Alapadatok és vendégek lépések 3. lépés: Szobák: 4. Nem választok szobát (vagy eltávolítom az összeset) 5. "Mentés" gomb	Sikeres mentés, foglalás státusza automatikusan "Várólistás"-ra vált
Foglalás szerkesztése: vendég hozzáadása	1. Foglalás sorában "..." gomb 2. "Szerkesztés" opció 2. lépés: Vendégek: 3. További vendég hozzáadása: Kiss Anna 4. "Következő", "Mentés"	Sikeres módosítás, új vendég hozzárendelődik a foglaláshoz

Folytatás a következő oldalon

táblázat 4.3 – Folytatás az előző oldalról

Funkció	Lépések	Elvárt eredmény
Foglalás szerkesztése: vendég eltávolítása	1. Foglalás sorában "..." gomb 2. "Szerkesztés" opció 2. lépés: Vendégek: 3. Meglévő vendégnél "X" gomb 4. "Következő", "Mentés"	Sikeres módosítás, vendég eltávolítódik a foglalásból
Foglalás szerkesztése: szoba hozzáadása	1. Foglalás sorában "..." gomb 2. "Szerkesztés" opció 3. lépés: Szobák: 3. További szoba kiválasztása: 103 4. "Mentés"	Sikeres módosítás, új szoba hozzárendelődik, státusz "Foglalt"-ra vált ha korábban "Várólistás" volt
Foglalás szerkesztése: szoba eltávolítása	1. Foglalás sorában "..." gomb 2. "Szerkesztés" opció 3. lépés: Szobák: 3. Meglévő szobánál "Eltávolítás" gomb 4. "Mentés"	Sikeres módosítás, szoba eltávolítódik, ha nincs több szoba akkor státusz "Várólistás"-ra vált
Foglalás státusz módosítása: Foglalt → Bejelentkezett	1. Foglalás sorában "..." gomb 2. "Státusz módosítása" opció 3. Kiválasztom: "Bejelentkezett" 4. "Mentés" gomb	Sikeres módosítás, foglalás státusza "Bejelentkezett"-re vált
Foglalás státusz módosítása: Bejelentkezett → Kijelentkezett	1. Foglalás sorában "..." gomb 2. "Státusz módosítása" opció 3. Kiválasztom: "Kijelentkezett" 4. "Mentés" gomb	Sikeres módosítás, foglalás státusza "Kijelentkezett"-re vált, szobák státusza "Piszkos"-ra vált

Folytatás a következő oldalon

táblázat 4.3 – *Folytatás az előző oldalról*

Funkció	Lépések	Elvárt eredmény
Foglalás státusz módosítása: Törölt → Foglalt	1. Törölt foglalás sorában "..." gomb 2. "Státusz módosítása" opció 3. Kiválasztom: "Foglalt" 4. "Mentés" gomb	Sikeres módosítás (Ha más aktiv foglalás nem használja már foglalásban lévő szobákat), foglalás státusza "Foglalt"-ra vált, szobák elérhetőségének újraellenőrzése történik
Foglalás státusz módosítása: Várólistás-ra váltás	1. Foglalás sorában "..." gomb 2. "Státusz módosítása" opció 3. Kiválasztom: "Várólistás" 4. "Mentés" gomb	Sikeres módosítás, foglalás státusza "Várólistás"-ra vált, szobák törölődnek a foglalásból
Számla szinkronizálása (ha van számla)	1. Foglalás sorában "..." (foglalásnak van számlája) gomb 2. "Számla szinkronizálása" opció	Számla szinkronizálódik a foglalással, megjelenik dialóg: "Szeretné megtekinteni a számlát?"
Számla megtekintése (ha van számla)	1. Foglalás sorában "..." (foglalásnak van számlája) gomb 2. "Számla megtekintése" opció	Átírányít a számla részletes nézetéhez
Számla létrehozása (ha nincs számla)	1. Foglalás sorában "..." (foglalásnak nincs számlája) gomb 2. "Számla létrehozása" opció	Új számla létrejön, megjelenik dialóg: "Szeretné megtekinteni a számlát?"
Számla létrehozása után megtekintés	1. Számla létrehozása 2. Dialógban "Igen" gomb	Átírányít az újonnan létrehozott számla részletes nézetéhez
Számla létrehozása után megtekintés elutasítása	1. Számla létrehozása 2. Dialógban "Nem" gomb	Maradok a foglalások listázó oldalon, dialóg bezárul

Folytatás a következő oldalon

táblázat 4.3 – *Folytatás az előző oldalról*

Funkció	Lépések	Elvárt eredmény
Foglalás törlése megerősítéssel	1. Foglalás sorában "..." gomb 2. "Törlés" opció 3. Megerősítő dialógban "Igen" gomb	Sikeres törlés üzenet, foglalás active flag false-ra áll (soft delete)
Foglalás törlése megerősítés nélkül	1. Foglalás sorában "..." gomb 2. "Törlés" opció 3. Megerősítő dialógban "Nem" gomb	Törlés megszakad, dialóg bezárul, foglalás változatlan marad

4.3. táblázat. Foglалások oldal manuális tesztjei

4.0.4. Vendégek oldal

Funkció	Lépések	Elvárt eredmény
Vendégek listázása	1. Navigálás a Vendégek menüpontra 2. Az oldal betöltődik	Megjelenik a vendégek listája táblázatos formában
Új vendég létrehozása teljes adatokkal	1. Rányomok az "Új vendég" gombra 2. Keresztnév: Teszt 3. Vezetéknév: Béla 4. Email: teszt.bela2@email.com 5. Telefon: +36301234267 6. Típus: Felnőtt 7. Aktív: Igen 8. Ország: Magyarország 9. Címkék: VIP, Törzsvásárló 10. "Mentés" gomb	Sikeres mentés üzenet, új vendég megjelenik a listában az összes megadott adattal

Folytatás a következő oldalon

táblázat 4.4 – *Folytatás az előző oldalról*

Funkció	Lépések	Elvárt eredmény
Új vendég típus: Gyerek	1. Rányomok az "Új vendég" gombra 2. Kitöltöm a kötelező mezőket 3. Típus: Gyerek 4. "Mentés" gomb	Sikeres mentés, vendég típusa "Gyerek"
Új vendég hibael- lenőrzés: Üres ke- resztnév	1. Rányomok az "Új vendég" gombra 2. Keresztnév mezőt üresen hagyom 3. Kitöltöm a többi kötelező mezőt 4. "Mentés" gomb	Validálciós üzenet: "Ez a mező kötelező"
Új vendég hibael- lenőrzés: Üres veze- téknév	1. Rányomok az "Új vendég" gombra 2. Vezetéknév mezőt üresen hagyom 3. Kitöltöm a többi kötelező mezőt 4. "Mentés" gomb	Validálciós üzenet: "Ez a mező kötelező"
Új vendég hi- baellenőrzés: Érvénytelen email	1. Rányomok az "Új vendég" gombra 2. Email: tesztEmail (érvényte- len formátum) 3. Kitöltöm a többi kötelező mezőt 4. "Mentés" gomb	Validálciós üzenet: "Érvénytelen formátum"

Folytatás a következő oldalon

táblázat 4.4 – Folytatás az előző oldalról

Funkció	Lépések	Elvárt eredmény
Új vendég hibael- lenőrzés: Már léte- ző email	1. Rányomok az "Új vendég" gombra 2. Email: teszt.bela@email.com (már létezik) 3. Kitöltöm a többi kötelező mezőt 4. "Mentés" gomb	Rendszer üzenet: "Ezzel az e-mail címmel már lé- tezik vendég"
Új vendég hi- baellenőrzés: Érvénytelen tele- fonszám	1. Rányomok az "Új vendég" gombra 2. Telefon: 123abc (érvénytelen formátum) 3. Kitöltöm a többi kötelező mezőt 4. "Mentés" gomb	Validálciós üzenet: "Érvénytelen formátum"
Vendég szerkeszté- se	1. Vendég sorában "..." gomb 2. "Szerkesztés" opció 3. Módosítom az adatokat (pl. telefon, ország, címkék) 4. "Mentés" gomb	Sikeres módosítás, ven- dég adatai frissülnek
Vendég aktív stá- tuszt váltása	1. Vendég sorában "..." gomb 2. "Szerkesztés" opció 3. Aktív: Nem 4. "Mentés" gomb	Sikeres módosítás, ven- dég inaktív státuszra vált
Vendég törlése:	1. Vendég sorában "..." gomb (vendégnek nincs foglalása) 2. "Törlés" opció 3. Megerősítő dialógban "Igen"	Sikeres törlés, vendég eltávolítódik a listából (Inaktív lesz)

Folytatás a következő oldalon

táblázat 4.4 – *Folytatás az előző oldalról*

Funkció	Lépések	Elvárt eredmény
Vendég törlése megerősítés nélkül	1. Vendég sorában "..." gomb (nincs foglalása) 2. "Törlés" opció 3. Megerősítő dialógban "Nem"	Törlés megszakad, dialóg bezárul, vendég változat- lan marad
Vendég címkék hoz- záadása	1. Vendég sorában "..." gomb 2. "Szerkesztés" opció 3. Címkék mezőben hozzá- adok: VIP, Törzsvendég 4. "Mentés" gomb	Sikeres mentés, címkék megjelennek a vendég adatainál
Vendég címkék el- távolítása	1. Vendég sorában "..." gomb 2. "Szerkesztés" opció 3. Meglévő címkéknél "X" gomb (törlés) 4. "Mentés" gomb	Sikeres mentés, címkék eltávolítódnak a vendég adataiból

4.4. táblázat. Vendégek oldal manuális tesztjei

4.0.5. Számlák oldal

Funkció	Lépések	Elvárt eredmény
Számlák listázása	1. Navigálás a Számlák menü- pontra 2. Az oldal betöltődik	Megjelenik a számlák lis- tája táblázatos formában bal oldalon, jobb oldal üres
Számla részleteinek megtekintése	1. Számlák listájában rákattin- tok egy számla sorára	A számla részletei megje- lennek a jobb oldali pane- len

Folytatás a következő oldalon

táblázat 4.5 – Folytatás az előző oldalról

Funkció	Lépések	Elvárt eredmény
Másik számla kiválasztása	1. Egy számla már meg van nyitva a jobb oldalon 2. Rákattintok egy másik számla sorára	A korábban megnyitott számla bezárul, az új számla részletei jelennek meg a jobb oldalon
Új számla létrehozása	1. Rányomok az "Új számla" gombra 2. Név: Teszt Számla 3. "Mentés" gomb	Sikeres mentés, üres számla létrejön "Függőben" fizetési státusszal, megjelenik a listában
Új számla hibael-lenőrzés: Üres név	1. Rányomok az "Új számla" gombra 2. Név mezőt üresen hagyom 3. "Mentés" gomb	Validálciós üzenet: "Ez a mező kötelező"
Számla alapadatok szerkesztése	1. Számla kiválasztása 2. Jobb oldali panelen "Szerkesztés" gomb 3. Név módosítása: Módosított Számla 4. Leírás: Új leírás 5. Státusz: Fizetve 6. "Mentés" gomb	Sikeres módosítás, számla alapadatai frissülnek
Számla címzett adatok szerkesztése	1. Számla kiválasztása 2. "Szerkesztés" gomb 3. Címzett adatok szerkesztése: Név, Cím, Adószám... 4. "Mentés" gomb	Sikeres módosítás, címzett adatok frissülnek
Számla alapadatok és címzett változtatások eldobása	1. Számla kiválasztása 2. "Szerkesztés" gomb 3. Módosítom az alapadatokat és címzett adatokat 4. "Visszaállítás" gomb	Változtatások eldobódnak, eredeti adatok visszaállnak

Folytatás a következő oldalon

táblázat 4.5 – *Folytatás az előző oldalról*

Funkció	Lépések	Elvárt eredmény
Új szolgáltatás hozzáadása számlához	1. Számla kiválasztása 2. Szolgáltatások szekciónál "+" gomb 3. Új sor jelenik meg a szolgáltatások táblázat alján 4. Kiválasztok egy szolgáltatást a legördülőből 5. Sorvégi "Mentés" ikon	Sikeres mentés, új szolgáltatás hozzáadódik a számlához, összeg újra-számolódik
Szolgáltatás hozzáadásának eldobása	1. Számla kiválasztása 2. Szolgáltatások szekció alatt "+" gomb 3. Új sor jelenik meg 4. Kiválasztok egy szolgáltatást 5. Sorvégi "X" gomb	Új szolgáltatás hozzáadása megszakad, sor eltűnik, változatlan marad a számla
Szolgáltatás eltávolítása számlából	1. Számla kiválasztása 2. Meglévő szolgáltatás sorában "X" gomb 3. Megerősítő dialógban "Igen"	Sikeres törlés, szolgáltatás eltávolítódik, összeg újraszámolódik
Új foglalás hozzáadása számlához	1. Számla kiválasztása 2. Foglalások szekció alatt "+" kártyára kattintás 3. Legördülőből kiválasztok egy foglalást (csak olyan foglalások látszanak, amik nem tartoznak más számlához) 4. "Hozzáadás" gomb	Sikeres hozzáadás, foglalás hozzárendelődik a számlához, megjelenik kártyaként
Foglalás eltávolítása számlából	1. Számla kiválasztása 2. Foglalás kártya sarkában "X" gomb 3. Megerősítő dialógban "Igen"	Sikeres eltávolítás

Folytatás a következő oldalon

táblázat 4.5 – *Folytatás az előző oldalról*

Funkció	Lépések	Elvárt eredmény
Foglalás eltávolítása megerősítés nélkül	1. Számla kiválasztása 2. Foglalás kártya sarkában "X" gomb 3. Megerősítő dialógban "Nem"	Eltávolítás megszakad, dialóg bezárul, foglalás változatlan marad
Foglalás hozzáadása: Csak szabad foglalások	1. Számla kiválasztása 2. "+" kártya 3. Legördülő megnyílik	Csak azok a foglalások jelennek meg, amelyek nem tartoznak más számlához
Számla összeg automatikus számítása	1. Számla kiválasztása 2. Szolgáltatás vagy foglalás hozzáadása/eltávolítása	A számla végösszege automatikusan újraszámolódik
Számla nyomtatása (PDF letöltés)	1. Számla kiválasztása 2. A számla neve mellett "Nyomtatás" ikon gomb	PDF fájl letöltődik a számlával
Számla törlése	1. Számla kiválasztása 2. A számla neve mellett "Törlés" gomb 3. Megerősítő dialógban "Igen"	Sikeres törlés, számla active flag false-ra áll (inaktív), soft delete
Számla törlése megerősítés nélkül	1. Számla kiválasztása 2. A számla neve mellett "Törlés" gomb 3. Megerősítő dialógban "Nem"	Törlés megszakad, dialóg bezárul, számla változatlan marad

4.5. táblázat. Számlák oldal manuális tesztjei

4.0.6. Takarítás oldal

Funkció	Lépések	Elvárt eredmény
Takarítási feladatok listázása	1. Navigálás a Takarítás menüpontra 2. Az oldal betöltődik	Megjelenik a takarítási feladatok listája
Új takarítási feladat létrehozása dolgozó nélkül	1. Rányomok az "Új takarítás" gombra 2. Szoba: 101 3. Prioritás: Magas 4. Megjegyzés: Alapos takarítás szükséges 5. Hozzárendelt dolgozó: üresen hagyom 6. "Mentés" gomb	Sikeres mentés, új feladat létrejön dolgozó és hozzárendelés dátuma nélkül, státusz: "Teendő"
Új takarítási feladat létrehozása dolgozóval	1. Rányomok az "Új takarítás" gombra 2. Szoba: 102 3. Prioritás: Normál 4. Megjegyzés: Gyors takarítás 5. Hozzárendelt dolgozó: Kiss Anna 6. "Mentés" gomb	Sikeres mentés, új feladat létrejön hozzárendelt dolgozóval, aktuális dátum hozzárendelve, státusz: "Teendő"
Új takarítási feladat hibael-lenőrzés: Üres szoba	1. Rányomok az "Új takarítás" gombra 2. Szoba mezőt üresen hagyom 3. Kitöltöm a többi mezőt 4. "Mentés" gomb	Validálciós üzenet: "Ez a mező kötelező"
Feladat szerkesztése: Dolgozó hozzárendelése	1. Feladat sorában "..." gomb 2. "Szerkesztés" opció 3. Hozzárendelt dolgozó: Kovács Béla 4. "Mentés" gomb	Sikeres módosítás, dolgozó hozzárendelődik, aktuális dátum hozzárendelődik a feladathoz

Folytatás a következő oldalon

táblázat 4.6 – Folytatás az előző oldalról

Funkció	Lépések	Elvárt eredmény
Gyors státusz váltás: Teendő → Folyamatban	1. Feladat sorában státusz dropdown 2. Kiválasztom: "Folyamatban"	Státusz azonnal frissül "Folyamatban"-ra, befejezési dátum nincs
Gyors státusz váltás: Folyamatban → Kész	1. Feladat sorában státusz dropdown 2. Kiválasztom: "Kész"	Státusz azonnal frissül "Kész"-re, aktuális dátum hozzárendelődik befejezési dátumként
Gyors státusz váltás: Kész → Folyamatban	1. Befejezett feladat sorában státusz dropdown 2. Kiválasztom: "Folyamatban"	Státusz frissül "Folyamatban"-ra, befejezési dátum törlődik
Gyors státusz váltás: Kész → Teendő	1. Befejezett feladat sorában státusz dropdown 2. Kiválasztom: "Teendő"	Státusz frissül "Teendő"-re, befejezési dátum törlődik
Gyors státusz váltás: Folyamatban → Teendő	1. Folyamatban lévő feladat sorában státusz dropdown 2. Kiválasztom: "Teendő"	Státusz azonnal frissül "Teendő"-re
Feladataim megjelenítése: Kapcsoló bekapcsolása	1. "Feladataim megjelenítése" kapcsoló bekapcsolása	Csak azok a feladatok jelennek meg, amelyekhez az aktuális felhasználó van hozzárendelve dolgozóként
Feladataim megjelenítése: Kapcsoló kikapcsolása	1. "Feladataim megjelenítése" kapcsoló kikapcsolása	Minden takarítási feladat megjelenik a listában
Feladataim szűrés: Nincs hozzárendelt feladat	1. "Feladataim megjelenítése" toggle bekapcsolása (felhasználóhoz nincs feladat hozzárendelve)	Üres lista jelenik meg és üzenet: "Nincs találat"

Folytatás a következő oldalon

táblázat 4.6 – *Folytatás az előző oldalról*

Funkció	Lépések	Elvárt eredmény
Prioritás módosítása	1. Feladat sorában "..." gomb 2. "Szerkesztés" opció 3. Prioritás: Alacsony → Magas 4. "Mentés" gomb	Sikeres módosítás, prioritás frissül
Megjegyzés módosítása	1. Feladat sorában "..." gomb 2. "Szerkesztés" opció 3. Megjegyzés módosítása: "Sürgős takarítás" 4. "Mentés" gomb	Sikeres módosítás, megjegyzés frissül
Feladat törlése	1. Feladat sorában "..." gomb 2. "Törlés" opció 3. Megerősítő dialógban "Igen"	Sikeres törlés, feladat eltávolítódik a listából
Feladat törlése megerősítés nélkül	1. Feladat sorában "..." gomb 2. "Törlés" opció 3. Megerősítő dialógban "Nem"	Törlés megszakad, dialóg bezárul, feladat változatlan marad

4.6. táblázat. Takarítás oldal manuális tesztjei

5. fejezet

Összegzés

Szakdolgozatom célja egy modern, egyszerű, de hatékony webalapú hotelmenedzsment és foglalási rendszer készítése volt. Elkészítettem egy átfogó hotelmenedzsment rendszert, amelyben lehet kezelni a foglalásokat, szobákat, számlákat, takarításokat és a hotel alapvető adatait.

Technológiailag felhasználtam az egyetemen elsajátított React és TypeScript tudásomat a frontend megvalósításához és ugyancsak az egyetemről megszerzett MySQL tudást az adatbázis felépítéséhez. Újdonságként megtanultam a projekthez a Java Spring Boot keretrendszert a backend elkészítéséhez.

Ezek mellett sok apróbb új technológiát használtam a szakdolgozat során, például Jotai state management-et Redux helyett, vagy OpenPDF-et nyomtatványok generálásához.

Úgy gondolom, a rendszer mostani állapota megfelelően használható kisebb szállodák igényei számára, de sajnos nem lett olyan komplex, mint szerettem volna. Ez leginkább abban látszik meg, hogy nincs beépített channel manager. (Ez tenné lehetővé az automatikus kommunikálást a rendszer és más foglalási platformok között, például Booking.com.) Azonban a rendszer moduláris mivolta miatt továbbfejlesztési lehetőségként nem kellene akkora kihívást okoznia egy channel manager kiépítése.

Összességében sikerült egy működőképes, modern szállodamenedzsment rendszert létrehoznom, amely valós problémára nyújt megoldást.

6. fejezet

Függelék

6.1. Szerepkörök

Szerepkör neve	Jogosultságok
Admin	Teljes rendszer hozzáférés: felhasználók kezelése, szerepkörök módosítása, minden adat megtekintése és szerkesztése
Receptionist	Recepciók műveletek: foglalások megtekintése/létrehozása/módosítása, számlák megtekintése, vendégek megtekintése/létrehozása/módosítása és nyomtatása , Számlák szinkronizálása foglalásokkal, szobák megtekintése
Housekeeper	Takarítási feladatok: housekeeping feladatok megtekintése/létrehozása/módosítása
Invoice Manager	Pénzügyi műveletek: számlák megtekintése/létrehozása/módosítása és nyomtatása, Számlák szinkronizálása foglalásokkal. ÁFA kulcsok megtekintése/létrehozása/módosítása és Szolgáltatások megtekintése/létrehozása/módosítása
Data Maintainer	Adatbázisban található adatok létrehozása/szerkesztése (kivéve a felhasználói adatok és szerepkörök)
Alap Felhasználó	Vezérlőpult megtekintése, alapadatok megtekintése. Minden felhasználó rendelkezik ezekkel a jogosultságokkal az egyéni szerepkörétől függetlenül

6.1. táblázat. Felhasználói szerepkörök és jogosultságaik

6.2. Részletes entitás leírások

Itt bemutatom a rendszer legfontosabb entitásait részletesen, csak a kritikus táblákra térek ki. **Automatizmusok:** A legtöbb entításban szerepelnek `created_at` és `updated_at` mezők. Ezek mindenhol ugyanúgy működnek:

- @PrePersist: `createdAt` és `updatedAt` beállítása aktuális időre
- @PreUpdate: `updatedAt` frissítése módosításkor

6.2.1. User tábla

A `users` tábla tárolja a rendszer felhasználóit (recepciósk, adminisztrátorok, stb.).

Mező	Típus	Megszorítás	Leírás
<code>id</code>	BIGINT	PK, AUTO_INC	Elsődleges kulcs
<code>email</code>	VARCHAR(255)	NOT NULL, UNIQUE	Email cím (természetes kulcs)
<code>first_name</code>	VARCHAR(255)	NULL	Keresztnév
<code>last_name</code>	VARCHAR(255)	NULL	Vezetéknév
<code>phone</code>	VARCHAR(255)	NOT NULL, UNIQUE	Telefonszám
<code>password</code>	VARCHAR(255)	NOT NULL	BCrypt hash jelszó
<code>active</code>	BOOLEAN	DEFAULT TRUE	Aktív-e a felhasználó?
<code>created_at</code>	TIMESTAMP	NULL	Létrehozás időpontja
<code>updated_at</code>	TIMESTAMP	NULL	Utolsó módosítás

6.2. táblázat. User tábla szerkezete

Kapcsolatok:

- N:M kapcsolat `Role` táblával: Milyen jogosultságokkal rendelkezik a felhasználó (`user_roles` kapcsolótábla)
- N:M kapcsolat `Building` táblával: Melyik épülethez tartozik a felhasználó (mappedBy: users)

6.2.2. Guest tábla

A `guests` tábla a vendégek személyes adatait tárolja.

Mező	Típus	Megszorítás	Leírás
id	BIGINT	PK, AUTO_INC	Elsődleges kulcs
first_name	VARCHAR(255)	NOT NULL	Keresztnév
last_name	VARCHAR(255)	NOT NULL	Vezetéknév
email	VARCHAR(255)	NOT NULL, UNIQUE	Email cím
phone_number	VARCHAR(255)	NOT NULL, UNIQUE	Telefonszám
home_country	VARCHAR(255)	NULL	Származási ország
type	VARCHAR(20)	NOT NULL	ENUM: GuestType
active	BOOLEAN	DEFAULT TRUE	Aktív-e a vendég?
created_at	TIMESTAMP	NULL	Létrehozás időpontja
updated_at	TIMESTAMP	NULL	Utolsó módosítás

6.3. táblázat. Guest tábla szerkezete

Kapcsolatok:

- N:M kapcsolat Booking táblával (guest_booking)
- N:M kapcsolat GuestTag táblával (guest_guest_tags)

Enumok:

- type: GuestTypeEnum értékei: Adult (Felnőtt), Child (Gyerek)

6.2.3. Booking tábla

A bookings tábla kezeli a szállodai foglalásokat.

Mező	Típus	Megszorítás	Leírás
id	BIGINT	PK, AUTO_INC	Elsődleges kulcs
active	BOOLEAN	NOT NULL	Aktív-e a foglalás?
status	VARCHAR(20)	NULL	ENUM: BookingStatus
scyn_status	VARCHAR(20)	NULL	ENUM: BookingInvoice
check_in_date	DATE	NOT NULL	Bejelentkezés dátuma
check_out_date	DATE	NOT NULL	Kijelentkezés dátuma
name	VARCHAR(255)	NOT NULL	Foglalás neve
description	TEXT	NULL	Leírás, megjegyzések
invoice_id	BIGINT	FK(Invoice.id)	Kapcsolódó számla
created_at	TIMESTAMP	NULL	Létrehozás időpontja
updated_at	TIMESTAMP	NULL	Utolsó módosítás

6.4. táblázat. Booking tábla szerkezete

Kapcsolatok:

- N:M kapcsolat **Guest** táblával (`guest_booking`)
- N:M kapcsolat **Room** táblával (`room_booking`)
- N:1 kapcsolat **Invoice** táblával (`invoice_id` FK)

Enumok:

- `status`: `BookingStatusEnum` (pl. `RESERVED`, `CHECKED_IN`, `CHECKED_OUT`, `CANCELLED`, `NO_SHOW`, `WAITLISTED`, `BLOCKED`)
- `scynStatus`: `BookingInvoiceEnum` (`NOT_SYNCED`, `NO_INVOICE`, `SYNCED`)

6.2.4. Room tábla

A `rooms` tábla a szálloda szobáit kezeli.

Mező	Típus	Megszorítás	Leírás
<code>id</code>	<code>BIGINT</code>	PK, <code>AUTO_INC</code>	Elsődleges kulcs
<code>room_number</code>	<code>VARCHAR(255)</code>	NOT NULL	Szobaszám
<code>floor_number</code>	<code>INT</code>	NULL	Emelet száma
<code>status</code>	<code>VARCHAR(20)</code>	NULL	ENUM: <code>RoomStatus</code>
<code>description</code>	<code>TEXT</code>	NULL	Szoba leírása
<code>room_type_id</code>	<code>BIGINT</code>	FK(<code>RoomType.id</code>), NOT NULL	Szobatípus
<code>building_id</code>	<code>BIGINT</code>	FK(<code>Building.id</code>), NOT NULL	Épület
<code>created_at</code>	<code>TIMESTAMP</code>	NULL	Létrehozás időpontja
<code>updated_at</code>	<code>TIMESTAMP</code>	NULL	Utolsó módosítás

6.5. táblázat. Room tábla szerkezete

Kapcsolatok:

- N:1 kapcsolat **RoomType** táblával (`room_type_id` FK)
- N:1 kapcsolat **Building** táblával (`building_id` FK)
- 1:N kapcsolat **Housekeeping** táblával
- N:M kapcsolat **Booking** táblával (`room_booking`)

Enumok:

- `status`: `RoomStatusEnum` (`OUT_OF_SERVICE`, `CLEAN`, `DIRTY`)

6.2.5. Invoice tábla

Az `invoice` tábla a számlák adatait tárolja.

Mező	Típus	Megszorítás	Leírás
id	BIGINT	PK, AUTO_INC	Elsődleges kulcs
name	VARCHAR(255)	NOT NULL	Számla megnevezése
description	VARCHAR(255)	NULL	Leírás
payment_status	VARCHAR(20)	NULL	ENUM: PaymentStatus
payment_fulfilled_at	TIMESTAMP	NULL	Fizetés teljesítésének időpontja
total_sum	DOUBLE	NULL	Végösszeg
recipient_name	VARCHAR(255)	NULL	Címzett neve
recipient_company_name	VARCHAR(255)	NULL	Címzett cég neve
recipient_address	VARCHAR(255)	NULL	Címzett címe
recipient_city	VARCHAR(255)	NULL	Címzett városa
recipient_postal_code	VARCHAR(255)	NULL	Címzett irányítószáma
recipient_country	VARCHAR(255)	NULL	Címzett országa
recipient_tax_number	VARCHAR(255)	NULL	Címzett adószáma
recipient_email	VARCHAR(255)	NULL	Címzett emailje
recipient_phone	VARCHAR(255)	NULL	Címzett telefonszáma
active	BOOLEAN	DEFAULT TRUE	Aktív-e számla?
created_at	TIMESTAMP	NULL	Létrehozás időpontja
updated_at	TIMESTAMP	NULL	Utolsó módosítás

6.6. táblázat. Invoice tábla szerkezete

Kapcsolatok:

- 1:N kapcsolat Booking táblával (mappedBy: invoice)
- N:M kapcsolat ServiceModel táblával (invoice_serviceModels)

Enumok:

- paymentStatus: PaymentStatusEnum (PENDING, FULFILLED, CANCELLED)

6.3. Osztályok és modulok

6.3.1. Controller osztályok

- `AuthController`: Bejelentkezés, regisztráció kezelése
- `InitController`: Rendszer inicializálás
- `UserController`: Felhasználók kezelése
- `RoleController`: Szerepkörök kezelése
- `GuestController`: Vendégek kezelése
- `GuestTagController`: Vendég címkék (VIP, törzsvendég, stb.)
- `BuildingController`: Épületek kezelése
- `RoomController`: Szobák kezelése
- `RoomTypeController`: Szobatípusok kezelése
- `BedTypeController`: Ágytípusok kezelése
- `AmenityController`: Szoba szolgáltatások (WiFi, klíma, stb.)
- `BookingController`: Foglalások kezelése
- `InvoiceController`: Számlák kezelése
- `ServiceModelController`: Szolgáltatások kezelése
- `VatController`: ÁFA kulcsok kezelése
- `HousekeepingController`: Takarítási feladatok
- `StatisticsController`: Statisztikák és riportok
- `CompanyInfoController`: Cég adatok kezelése
- `DevLogController`: Verzió történet

6.3.2. Service osztályok

- `IAmenityService` / `AmenityService`: Szolgáltatások üzleti logikája
- `IAuthService` / `AuthService`: Autentikáció és jogosultságkezelés
- `IBedTypeService` / `BedTypeService`: Ágytípusok üzleti logikája
- `IBookingService` / `BookingService`: Foglalási validáció, ütközéskezelés
- `IBuildingService` / `BuildingService`: Épületek kezelése
- `ICompanyInfoService` / `CompanyInfoService`: Cég adatok kezelése
- `IDevLogService` / `DevLogService`: Verzió történet kezelése
- `IGuestService` / `GuestService`: Vendégek üzleti logikája
- `IGuestTagService` / `GuestTagService`: Vendég címkék kezelése

- `IHousekeepingService` / `HousekeepingService`: Takarítási feladatok logikája
- `IInitService` / `InitService`: Rendszer inicializálás
- `IInvoiceService` / `InvoiceService`: Számlázási logika
- `IRoleService` / `RoleService`: Szerepkörök kezelése
- `IRoomService` / `RoomService`: Szobák üzleti logikája
- `IRoomTypeService` / `RoomTypeService`: Szobatípusok kezelése
- `IServiceModelService` / `ServiceModelService`: Szolgáltatási modellek
- `IStatisticsService` / `StatisticsService`: Statisztikák és riportok
- `IUserService` / `UserService`: Felhasználók kezelése
- `IVatService` / `VatService`: ÁFA kulcsok kezelése

6.4. Docker konfiguráció

6.4.1. Docker Compose konfiguráció

A `docker-compose.yml` fájl három szolgáltatást definiál:

```

1  services:
2  frontend:
3  build: ./frontend
4  ports:
5  - "3000:3000"
6  depends_on:
7  - backend
8  restart: on-failure
9
10 backend:
11 build: ./backend
12 ports:
13 - "8080:8080"
14 environment:
15 - SPRING_DATASOURCE_URL=jdbc:mysql://mysql:3306/hotelpms
16 - SPRING_DATASOURCE_USERNAME=root
17 - SPRING_DATASOURCE_PASSWORD=rootpass123
18 - SPRING_JPA_HIBERNATE_DDL_AUTO=update
19 depends_on:
20 mysql:
21 condition: service_healthy
22 restart: on-failure
23
24 mysql:
25 image: mysql:8.0
26 environment:
27 - MYSQL_ROOT_PASSWORD=rootpass123
28 - MYSQL_DATABASE=hotelpms
29 command: --authentication-policy=mysql_native_password --host-cache-size=0
30 ports:
31 - "3306:3306"
32 volumes:
33 - mysql_data:/var/lib/mysql
34 healthcheck:
35 test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p", "rootpass123"]
36 timeout: 20s
37 retries: 10
38 interval: 10s
39 start_period: 40s
40
41 volumes:
42 mysql_data:

```

6.1. forráskód. `docker-compose.yml` struktúra

6.4.2. Frontend Dockerfile

A frontend konténer Node.js 22 Alpine image-et használ:

```
1 FROM node:22-alpine
2 WORKDIR /app
3 COPY package.json .
4 RUN npm install
5 RUN npm i -g serve
6 COPY . .
7 RUN npm run build
8 EXPOSE 3000
9 CMD [ "serve", "-s", "dist" ]
```

6.2. forráskód. Frontend Dockerfile

Build lépések:

1. Node.js 22 Alpine base image
2. Függőségek telepítése (`npm install`)
3. Serve telepítése (statikus fájl kiszolgáláshoz)
4. Forráskód másolása
5. Vite production build (`npm run build`)
6. 3000-es port expose-olása
7. Serve indítása a `dist` mappából

6.4.3. Backend Dockerfile

A backend konténer Eclipse Temurin JDK 17 image-et használ:

```
1 FROM eclipse-temurin:17-jdk-alpine
2 WORKDIR /app
3 COPY mvnw .
4 COPY mvnw.cmd .
5 COPY .mvn .mvn
6 COPY pom.xml .
7 RUN chmod +x ./mvnw
8 RUN ./mvnw dependency:go-offline -B
9 COPY src src
10 RUN ./mvnw clean package -DskipTests
11 EXPOSE 8080
12 CMD [ "java", "-jar", "target/backend-0.0.1-SNAPSHOT.jar" ]
```

6.3. forráskód. Backend Dockerfile

Build lépések:

1. Eclipse Temurin JDK 17 Alpine base image
2. Maven wrapper fájlok másolása
3. Függőségek előzetes letöltése (build gyorsítás)
4. Forráskód másolása
5. Maven clean package (tesztek kihagyásával)
6. 8080-as port expose-olása
7. Spring Boot JAR indítása

6.4.4. Adatbázis inicializálás

Az adatbázis adatai a `mysql_data` Docker volume-ban tárolódnak, így a konténer újraindítása után is megmaradnak. A MySQL konténer első indításkor:

- Létrehozza a `hotelpms` adatbázist
- A backend Hibernate `ddl-auto=update` miatt automatikusan létrehozza a táblákat
- A `DataInitFlag` kapcsoló alapján minta adatok töltődnek be, ha ez az első indítás

6.4.5. Környezeti változók

A backend környezeti változói a `docker-compose.yml`-ben módosíthatók:

- `SPRING_DATASOURCE_URL`: Adatbázis kapcsolat URL
- `SPRING_DATASOURCE_USERNAME`: Adatbázis felhasználónév
- `SPRING_DATASOURCE_PASSWORD`: Adatbázis jelszó
- `SPRING_JPA_HIBERNATE_DDL_AUTO`: Séma kezelés módja

Ábrák jegyzéke

2.1. Alapértelmezetten megjelenő bejelentkezési képernyő	8
2.2. Új felhasználó létrehozását dokumentáló képek	9
2.3. Alapértelmezett admin fiók törlése	10
2.4. Vezérlőpult megjelenése	11
2.5. Bevételi gráf időszak beállítása	12
2.6. Táblaműveletek	13
2.7. Oszlopszűrés és rendezés felület	13
2.8. Példa: Új épület hozzáadásának lépései	14
2.9. Példa: Épület szerkesztésének lépései	14
2.10. Példa: Épület törlése gomb	15
2.11. Épületek felület	15
2.12. Szoba szolgáltatások felület	16
2.13. Vendég címkék felület	16
2.14. Ágytípusok felület	17
2.15. Szerepkörök felület	18
2.16. Felhasználók felület	18
2.17. Szobatípusok felület	19
2.18. Szobák felület	19
2.19. Szoba műveletek	20
2.20. Cég információ felület	21
2.21. Vendégek felület	22
2.22. Foglalások felület	22
2.23. Új foglalás létrehozásának lépései	25
2.24. Szobatükör felület	27
2.25. ÁFA kulcsok felület	28
2.26. Szolgáltatások felület	29
2.27. Számlák felület teljes nézet	30

2.28. Számla nyomtatás gomb	31
2.29. Számla törlése gomb	32
2.30. Takarítások felület	32
2.31. Takarítás felület műveletei	33
2.32. Feladat szerkesztése	34
2.33. Példa űrlap validációs hibákra	35
2.34. Művelet végrehajtási üzenetek	35
3.1. Rendszer háromrétegű architektúrája	37
3.2. Adatbázis Entitás-Kapcsolat Diagram	42
3.3. Rendszer architektúra áttekintése	45
3.4. Booking modul osztálydiagramja	48
3.5. Navigációs menü	51
3.6. Bejelentkezési felület	51
3.7. Dashboard nézet statisztikákkal	52
3.8. Táblázatos lista nézet műveleti gombokkal	53
3.9. Modal ablak	53
3.10. Szobátükör nézet foglalások vizualizálásához	54
3.11. Navigációs diagram	54
3.12. Kódlefedettség: Unit tesztek	72
3.13. Kódlefedettség: Mock-alapú service tesztek	72

Táblázatok jegyzéke

2.1. Alapértelmezett adminisztrátori fiók	8
3.1. guest_booking kapcsolótábla	44
3.2. room_type_bed_type kapcsolótábla szerkezete	44
3.3. Szolgáltatások elérhetősége	70
3.4. Tesztelési eredmények összefoglalása	73
3.5. Unit tesztesetek	74
3.6. BookingService tesztesetek	75
3.7. InvoiceService tesztesetek	75
3.8. ServiceModelService tesztesetek	75
4.1. Bejelentkezési képernyő manuális tesztjei	77
4.2. Regisztrációs képernyő manuális tesztjei	78
4.3. Foglalások oldal manuális tesztjei	86
4.4. Vendégek oldal manuális tesztjei	89
4.5. Számlák oldal manuális tesztjei	92
4.6. Takarítás oldal manuális tesztjei	95
6.1. Felhasználói szerepkörök és jogosultságaik	97
6.2. User tábla szerkezete	98
6.3. Guest tábla szerkezete	99
6.4. Booking tábla szerkezete	99
6.5. Room tábla szerkezete	100
6.6. Invoice tábla szerkezete	101

Forráskódjegyzék

3.1. GET /api/amenities - Lista lekérdezés	56
3.2. POST /api/amenities - Új elem létrehozása	56
3.3. PUT /api/amenities/{id} - Elem modositasa	57
3.4. DELETE /api/amenities/{id} - Elem torlese	57
3.5. Request validáció Bean Validation-al	58
3.6. Egyszerű DTO konverzió	59
3.7. Komplex DTO konverzió ha több entitást is vissza kell adni	59
3.8. Épületek szűrés kérés	60
3.9. Szűrés függvény a BuildingService-ben	60
3.10. FormConfig TypeScript típusok	61
3.11. FormFactory komponens	62
3.12. FieldWrapper - dinamikus beviteli mező választás	62
3.13. Vendég címke űrlap konfiguráció	63
3.14. FormModal komponens (Egyszerűsített)	63
3.15. Validációs rendszer	65
3.16. BedTypeQuantity field konfiguráció	65
3.17. Dinamikus form konfiguráció VAT opciókkal	66
3.18. Virtuális ServiceModel létrehozása	67
3.19. Base ServiceModel létrehozása/lekérése	68
3.20. Virtuális szolgáltatások szűrése	68
3.21. Projekt letöltése	70
3.22. Docker Compose indítása	70
3.23. Docker Compose háttérben	70
6.1. docker-compose.yml struktúra	104
6.2. Frontend Dockerfile	105
6.3. Backend Dockerfile	105